# Genomic Prediction in Black Beans

A beginner-friendly course that reproduces a real GWAS & genomic-selection study

Built around Izquierdo, Wright & Cichy (2025), *G3* jkaf007

19 documents · toy-first worked examples · reproduced figures · glossary

## Contents

# Genomic Prediction in Black Beans — A Course for Beginners

> A step-by-step, beginner-friendly course built around one real study: **Izquierdo, Wright & Cichy (2025), *G3* — "GWAS-assisted and multitrait genomic prediction for improvement of seed yield and canning quality traits in a black bean breeding panel."** (DOI: 10.1093/g3journal/jkaf007)

We are going to **reproduce this study** and, along the way, explain *every* moving part — the **biology**, the **breeding**, the **statistics**, and the **math** — in language a 2nd/3rd-year BSc student can follow. By the end you will not just "know what GWAS and genomic selection are"; you will have a **mental map** of the whole research question and be able to run the analysis yourself.

---

## Who this is for

- BSc students in plant breeding, genetics, agronomy, biology, or data science.
- Anyone who has heard "GWAS", "GBLUP", "genomic selection", "BLUP", "kinship matrix" and wants to *really* understand them, not just nod along.

**Assumed background:** high-school algebra, the idea of a mean and a correlation, and a willingness to meet a little matrix notation (we explain it as we go). No prior genetics or R required — we build it up.

---

## How to read this course

Each lesson follows the same rhythm:

1. **The question** — what are we trying to find out, and *why now* in the pipeline?
2. **Intuition first** — a picture/analogy before any symbol.
3. **The math, gently** — every symbol defined, small worked numbers.
4. **In the data** — real numbers/figures reproduced from *this* study.
5. **Why they did it this way** — the breeding logic behind the choice.

Boxes you'll see:

- 🧠 **Intuition** — the plain-language mental model.
- 🧱 **Math** — the equation, with every term defined.
- 🧸 **Toy example** — a tiny made-up dataset (a handful of lines/SNPs) where we compute *every number by hand and visualize it*, **before** touching the real 415-line data.
- 🔭 **Zoom out** — the bridge: "now picture that toy stretched to the real 415 × 2,315 scale."
- 🌱 **Breeding logic** — why a breeder cares.
- 🔬 **In the data** — a real, reproduced result from the black bean dataset.
- 🔧 **In practice ®** — which package computes this for you in real work.
- ⚠️ **Common confusion** — a trap to avoid.

> **The core teaching move:** every hard idea appears first on a 🧸 **toy** you can hold in your head (e.g. 5 lines × 10 SNPs), then 🔭 **zooms out** to the real data. Toy → real, every time.

## Syllabus

| # | Lesson | What you'll be able to do |
|---|---|---|
| 0 | The Mental Map | Draw the whole study as one flowchart and state the research question |
| 1 | Beans, Breeding & the Traits | Explain the crop, the traits, and *why* yield vs. quality fight each other |
| 2 | The Data: anatomy of `GB_BLB` | Open the real dataset and know what every piece is |
| 3 | Phenotyping & Spatial BLUPs | Turn messy field plots into one clean number per line; understand heritability |
| 4 | Genotyping by Sequencing & SNPs | Explain how DNA becomes a 0/1/2 marker matrix, and every filter applied |
| 5 | Quantitative Genetics Foundations | Define breeding value, additive effect, and heritability with equations |
| 6 | The Genomic Relationship Matrix **G** | Build VanRaden's **G** by hand and read a kinship heatmap |
| 7 | GBLUP — genomic prediction's workhorse | Derive GBLUP, see it = ridge regression, and reproduce accuracy 0.64 |
| 8 | RKHS & Gaussian Kernels | Explain kernels, bandwidth, and kernel averaging (KA) |
| 9 | GWAS with FarmCPU | Run/understand a GWAS, PCs, Bonferroni, Manhattan plots |
| 10 | GWAS-assisted Genomic Prediction | Add markers as fixed effects — and explain why it *hurt* here |
| 11 | NIRS & Regularized Selection Indices | Use spectra as a cheap secondary trait via penalized regression |
| 12 | Multi-Trait Models | Borrow strength from correlated traits; why MT beat ST across cycles |
| 13 | Cross-Validation & Prediction Accuracy | Measure "how good is the prediction?" honestly; read Table 1 |
| 14 | Across Breeding Cycles & Updating | Predict a *new* cycle and explain why the training set must keep growing |
| 15 | Results & Take-home Messages | Summarize what the study found and what it means for breeders |
| 16 | Reproduce It Yourself | Run the scripts in `code/` and regenerate the figures |

## What's in this folder

```
GWAS_GS/
├── ref_paper.pdf          # the study (G3, 2025)
├── repo/                  # the authors' original code + data (cloned from GitHub)
│   ├── GB_BLB.RData       # phenotypes, genotypes, NIRS, SNP positions
│   └── *.R                # their analysis scripts
├── course/                # ← you are here: the lessons
├── code/                  # our clean, runnable reproduction scripts
└── figures/               # figures we generate from the real data
```

> **Reproducibility note.** Every number quoted in 🔬 *In the data* boxes is produced by the scripts in `code/`. We re-implement the core methods *from scratch* in base R first (so you can see the machinery), then confirm against the same packages the authors used (`BGLR`, `rrBLUP`). Where we ran the real reproduction, you'll see the exact figure name.

Start with **Lesson 0: The Mental Map**.

# Lesson 0 — The Mental Map

> **Goal of this lesson:** before any equations, get the *whole study* into your head as a single picture. Everything else in the course is a zoom-in on one box of this map.

## 0.1 The one-sentence research question

> **Can we use a plant's DNA (and cheap light-scan data) to predict how good its beans will be — for *yield* and for *canning quality* — accurately enough to pick winners *before* we grow and taste them?**

That's it. If we can predict well from DNA, a breeder can **select** the best lines using only a leaf sample, skipping years of field trials and expensive taste panels. Faster selection = faster **genetic gain** = better cultivars sooner.

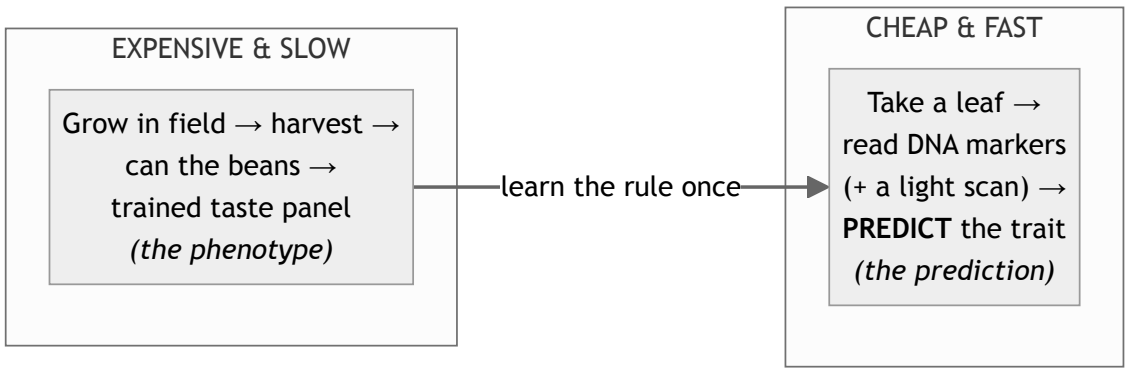
## 0.2 Why this question is *hard* (and therefore interesting)

Three honest difficulties, which the whole study is designed around:

1. **The two goals fight each other.** High **seed yield** and high **canning quality** tend to be *negatively* associated in beans. Selecting hard for one can drag the other down. (We'll confirm this in the real data: yield ↔ seed-weight is positive, but seed-weight ↔ texture is *negative*.)
2. **Quality is painful to measure.** Canning appearance and texture need special equipment and a *trained human taste/look panel* — slow, costly, low-throughput. Most breeding programs simply can't measure it on thousands of lines.
3. **These are *complex* (polygenic) traits.** Yield and appearance are controlled by *many* genes of small effect, not one or two. So you can't just find "the yield gene" and select on it.

These three facts explain *every* method choice in the paper. Keep them in mind.

## 0.3 The big idea: prediction instead of measurement

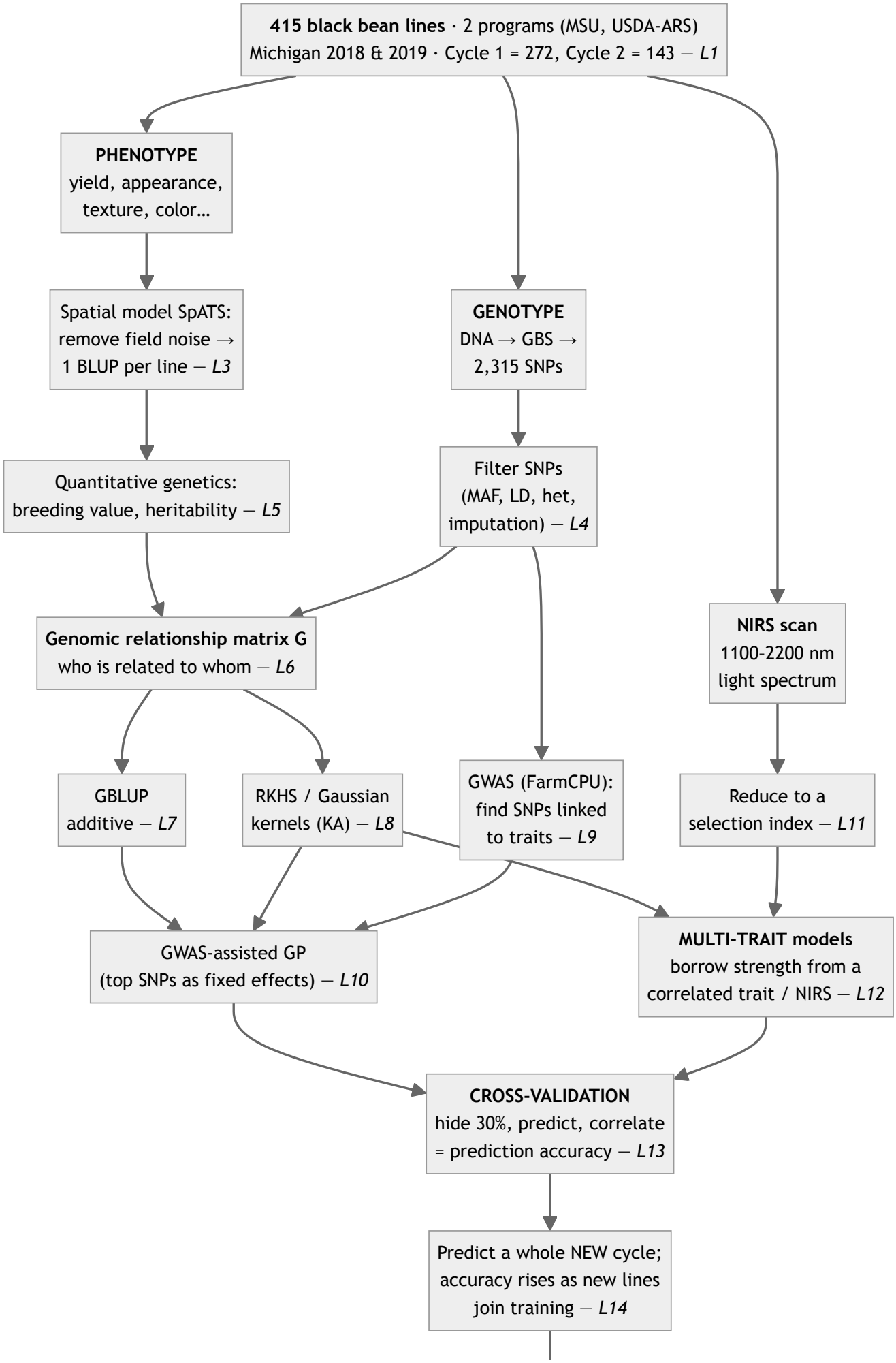


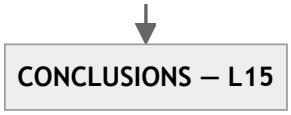
We measure *both* DNA and phenotype on a **training set** of lines, learn the statistical relationship, then for **new** lines we measure *only* DNA and **predict** the phenotype. This is **genomic prediction (GP)**; using those predictions to choose parents/lines is **genomic selection (GS)**.

---

## 0.4 The whole study as one flowchart

Read this top to bottom. Each box is (roughly) one lesson.


**415 black bean lines** · 2 programs (MSU, USDA-ARS)
Michigan 2018 & 2019 · Cycle 1 = 272, Cycle 2 = 143 — *L1*
**PHENOTYPE**
yield, appearance, texture, color…
Spatial model SpATS: remove field noise → 1 BLUP per line — *L3*
Quantitative genetics: breeding value, heritability — *L5*
**GENOTYPE**
DNA → GBS → 2,315 SNPs
Filter SNPs (MAF, LD, het, imputation) — *L4*
**NIRS scan**
1100–2200 nm light spectrum
**Genomic relationship matrix G**
who is related to whom — *L6*
GBLUP additive — *L7*
RKHS / Gaussian kernels (KA) — *L8*
GWAS (FarmCPU): find SNPs linked to traits — *L9*
Reduce to a selection index — *L11*
GWAS-assisted GP (top SNPs as fixed effects) — *L10*
**MULTI-TRAIT models**
borrow strength from a correlated trait / NIRS — *L12*
**CROSS-VALIDATION**
hide 30%, predict, correlate = prediction accuracy — *L13*
Predict a whole NEW cycle; accuracy rises as new lines join training — *L14*




## 0.5 The four objectives (the study's own words, decoded)

The paper states four objectives. Here they are in plain language, mapped to lessons:

| # | Paper's objective | In plain words | Lessons |
|---|---|---|---|
| 1 | Compare **single-trait (ST)** vs **multi-trait (MT)** GP for yield & quality | Is it better to predict each trait alone, or to let correlated traits help each other? | 7, 12, 13 |
| 2 | Assess **NIRS-based selection indices** as secondary traits in MT models | Can a cheap light scan stand in as a helpful extra trait? | 11, 12 |
| 3 | Assess adding **GWAS-significant SNPs** to GP models | Does telling the model "these specific SNPs matter" help? | 9, 10 |
| 4 | Measure how many **new-cycle genotypes** you must add to keep predicting well | How often, and how much, must you refresh the training set? | 14 |

> **The punchlines (so you know where we're heading):**
>
> - MT models **did not** beat ST *within* a cycle, but **did** beat ST *across* cycles (up to **+63% for yield**, **+41% for appearance**). → *Correlated traits help most when prediction is otherwise hard.*
> - Adding NIRS or GWAS SNPs **did not** help (often slightly *hurt*). → *More information is not automatically better.*
> - Accuracy **rose** as lines from the new cycle were added to training. → *Keep your training set fresh.*

## 0.6 The mental model to carry through the whole course

Hold these three layers in your head; every lesson slots into one:

1. **Biology layer** — genes → traits, in a real crop with real field noise. (L1, L3–L5)
2. **Relationship layer** — DNA tells us *who resembles whom* (matrix **G**) and *which loci matter* (GWAS). (L4, L6, L9)
3. **Prediction layer** — a statistical model uses the relationship layer to guess phenotypes for unseen lines, and we *honestly* score it. (L7–L8, L10–L14)

When you feel lost later, ask: *"Which layer am I in, and which flowchart box?"*

👉 Next: **Lesson 1 — Beans, Breeding & the Traits.**

# Lesson 1 — Beans, Breeding & the Traits

> **The question:** *What* are we breeding, *which* traits matter, and *why* do they fight each other? Without this, the statistics later will feel arbitrary.

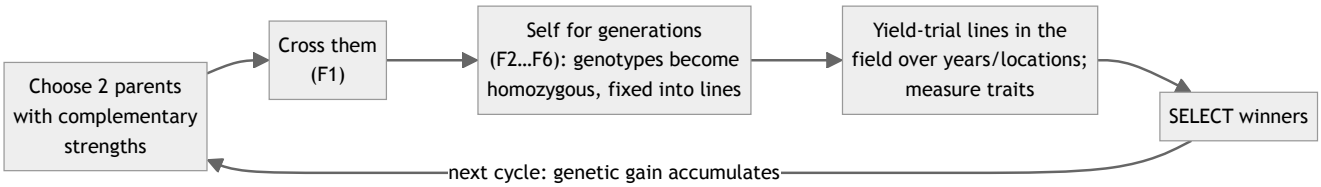
## 1.1 The crop: *Phaseolus vulgaris*, the common bean

- **Common bean** (*Phaseolus vulgaris* L.) is the most important grain legume for direct human consumption worldwide — a cheap, storable source of **protein, fiber, iron and zinc**.
- It comes in **market classes** defined by seed color/shape: navy, pinto, kidney, great northern… and **black bean**, the focus of this study. Black beans are popular and *rising* in the U.S.
- Genetically, common bean is **diploid** with **2n = 22 → 11 chromosomes**. (Remember this number: when we look at SNPs, they'll fall on chromosomes 01–11. We confirmed exactly 11 chromosomes in the data.)
- It is largely **self-pollinating**. That matters for breeding (next section) and for the genetics: lines are highly **homozygous** ("true-breeding"), so a "line" is essentially a reproducible genotype you can grow again and again.

🌱 **Breeding logic.** Because beans self-pollinate, a breeder can fix a genotype into a near-homozygous **line**, multiply its seed, and test the *same* genetic individual across years and locations. This is *why* we can speak of "the breeding value of line X" as a stable thing worth predicting.

## 1.2 How a bean cultivar gets made (the breeding cycle)

A simplified dry-bean breeding cycle:




In this study the lines are at the **F6 generation or later** — i.e. essentially fixed, homozygous lines ready for yield testing. They come from **two programs**:

| Program | Emphasis |
|---|---|
| **MSU** (Michigan State University) | seed **yield** and canning quality |
| **USDA-ARS** | **nutritional** and canning quality |

Because the two programs select for *different* things from *different* parents, their lines form somewhat **distinct genetic groups** — this is **population structure**, and it will come back to bite us in GWAS (Lesson 9) and is visible in the relationship matrix (Lesson 6).

### Breeding *cycles* in this dataset (memorize this — it structures everything)

- **Cycle 1 = 272 lines.** Advanced/preliminary trial lines, grown in **both 2018 and 2019**.
- **Cycle 2 = 143 lines.** A *newer* set, grown **only in 2019**.

- Total = **415 lines** (rows of every data matrix you'll meet).

> 🔬 **In the data.** We confirmed it directly: rows 1–272 have a 2018 yield value (272 of them); rows 273–415 do **not** (0), but 142 of them have a 2019 yield value. So the row order *encodes* the cycle. This single fact is what makes Objective 4 ("predict the new cycle") possible.

🧠 **Intuition — why "cycle" matters for prediction.** Cycle 2 is the *future* relative to cycle 1. If we train a model on cycle 1 and predict cycle 2, we are simulating exactly what a breeder does: *use last year's data to pick this year's winners*. Lesson 14 is entirely about this.

---

## 1.3 The traits — and the words breeders actually use

The study measures **agronomic** traits and **canning-quality** traits. You don't need to memorize all of them, but you must understand the two *stars* (yield & appearance) and why the *supporting cast* (seed weight, texture) matters.

### Agronomic traits (measured in the field)

| Code | Trait | What it is | Why measured |
|---|---|---|---|
| **YD** | Seed yield (kg/ha) | total seed weight per area, at 18% moisture | the #1 breeding target |
| **SW** | 100-seed weight (g) | weight of 100 seeds | a yield *component*, easy & fast to measure |
| **DF** | Days to flowering | days from planting to 50% plants flowering | maturity/adaptation |
| **DPM** | Days to maturity | days until pods start drying | maturity/adaptation |

### Canning-quality traits (measured in the lab, after cooking)

Beans are cooked in a standardized **canning protocol** (soaked, blanched, brined, retorted), then evaluated:

| Code | Trait | How measured | Scale |
|---|---|---|---|
| **App** | Canning **appearance** | a *trained human panel* (8–12 people) rates each sample | 1 (worst) – 5 (best) |
| **Col** | Color (darkness) | panel rating | 1 (lightest) – 5 (darkest) |
| **Text** | **Texture** | a machine (texture analyzer) measures peak force to compress 100 g | force (objective) |
| **L*** | Lightness | Hunter colorimeter | 0 black → 100 white |
| **a*** | green↔red | colorimeter | – green / + red |
| **b*** | blue↔yellow | colorimeter | – blue / + yellow |

⚠️ **Common confusion.** *Appearance and color are NOT the same.* "App" is the panel's holistic judgment of how good the canned beans *look* (intact, glossy, dark). "Col" and "L*a*b*" are specific measures of darkness/hue. For black beans, **staying dark after canning** ("color retention") is prized — pale, faded beans look unappetizing.

🌱 **Breeding logic — human panel vs machine.** The panel ratings (App, Col) are *subjective* and *expensive* (you must assemble and train people, then cook samples for them). The colorimeter (L*a*b*) and texture analyzer are *objective* and cheaper. A recurring theme of the paper: **can a cheaper, objective measurement (or a DNA prediction, or a light scan) stand in for the expensive panel?**

---

## 1.4 The central tension: yield vs. quality

This is the heart of *why* the study needed clever multi-trait methods.

🧠 **Intuition.** Imagine two dials: **YIELD** and **CANNING QUALITY**. In beans, turning up one tends to turn down the other. A famous example: bigger/heavier seeds (high SW, which helps yield) can be *softer* after canning (worse texture). So a breeder who blindly selects for yield may accidentally ship beans that turn to mush in the can.

🔬 **In the data (2018 BLUPs, reproduced in `code/01_explore_data.R`):**

| Pair | Correlation *r* | Reading |
|---|---|---|
| Yield ↔ 100-seed weight | **+0.41** | bigger seeds → more yield (makes sense) |
| Yield ↔ days to maturity | +0.43 | later-maturing lines yield more |
| **Seed weight ↔ texture** | **−0.36** | **bigger seeds → softer (lower-force) texture** ← the tension |
| Yield ↔ appearance | ≈ 0.00 | no simple field-yield/quality link |

> Figure: `figures/02_trait_correlations.png` (full correlation heatmap).

That **−0.36** is the antagonism in miniature: the very thing that boosts yield (seed size) costs you canning texture. **Selecting for both at once requires a method that can hold two traits in view simultaneously** — which is exactly the *multi-trait* and *selection index* machinery in Lessons 11–12.

⚠️ **Common confusion — "correlation isn't always strong."** Note yield ↔ appearance was ~0. Trait antagonisms are *tendencies*, not laws, and they vary by year (2018 was wetter, so correlations differed from 2019). The study measured them across **two seasons** precisely because single-year correlations can mislead.

---

## 1.5 Heritability — the trait property that decides "is prediction even possible?"

We'll define heritability mathematically in Lesson 5, but plant the seed now:

🧠 **Intuition. Heritability ($h^2$)** answers: *of the differences we see between lines, what fraction is due to their genes* (as opposed to weather, soil patch, measurement error)? It runs from 0 (all noise) to 1 (all genetics).

- **Color** in this study is *highly* heritable → easy to predict from DNA (accuracies up to **0.93**).
- **Yield** and **appearance** are *lower* heritability → harder to predict (accuracies ~0.4–0.6).

🌱 **Breeding logic.** You can only predict from DNA the part of a trait that *is* genetic. A trait that's 80% weather noise has little signal for the genome to grab. So **heritability sets a ceiling on prediction accuracy.** Keep this rule of thumb: *higher* $h^2$ → *higher achievable accuracy*. It explains almost every accuracy ranking in the results.

---

## 1.6 What you should now be able to say

- Black bean is a self-pollinating, diploid (2n=22, **11 chromosomes**) grain legume; lines are near-homozygous reproducible genotypes.
- The dataset = **415 lines** in **2 cycles** (272 + 143) from **2 programs**, grown in Michigan 2018–2019.
- The breeding targets are **yield** and **canning quality (appearance, color, texture)**; quality is **expensive and subjective** to measure.
- Yield and quality are **antagonistic** (we saw SW ↔ texture = −0.36), which *motivates* multi-trait methods.
- **Heritability** caps how well any trait can be predicted.

👉 Next: **Lesson 2 — The Data: anatomy of `GB_BLB`** — we open the actual file and look at every object inside.

# Lesson 2 — The Data: Anatomy of `GB_BLB`

> **The question:** before any modeling, *what exactly do we have?* A model is only as good as your understanding of its inputs. We open the real file and name every part.

---

## 2.1 One file to rule them all

The authors ship a single R data file, `repo/GB_BLB.RData`. Loading it gives one object, `GB_BLB`, which is a **list** (think: a labeled drawer with 5 compartments):

```
load("repo/GB_BLB.RData")
names(GB_BLB)
#> "pheno"  "geno"  "nirs18"  "nirs19"  "SNP_Position"
```

🔬 **In the data** (from `code/01_explore_data.R`), the five compartments are:

| Compartment | Shape | What it holds |
|---|---|---|
| `pheno` | **415 × 21** | one row per line; the cleaned trait values (BLUPs) for 2018 & 2019 |
| `geno` | **415 × 2315** | one row per line; **2,315 SNP markers** coded 0/1/2 |
| `nirs18` | **415 × 551** | near-infrared **spectrum** per line, 2018 (one column per wavelength) |
| `nirs19` | **415 × 551** | same, 2019 |
| `SNP_Position` | **2315 × 3** | for each SNP: its name, chromosome, and base-pair position |

The magic that makes everything work: **all matrices share the same 415 rows, in the same order.** Row 7 of `pheno`, row 7 of `geno`, row 7 of `nirs18` are *the same bean line*. This alignment is what lets a model connect "this DNA" ↔ "this phenotype."

⚠️ **Common confusion — "415 rows but only 272 have 2018 yield?"** Yes. The matrix has a row for *every* line, but cycle-2 lines (rows 273–415) were not grown in 2018, so their 2018 cells are `NA` (missing). Missingness is *information*, not an error — it encodes the breeding-cycle design (Lesson 1).

---

## 2.2 `pheno` — the trait table

21 columns: a `taxa` (line name) column plus 20 trait columns. The naming convention is `trait` + `year`:

```
taxa | yd18 yd19 | sw18 sw19 | dm18 dm19 | df18 df19 | app18 app19 |
       text18 text19 | col18 col19 | l18 l19 | a18 a19 | b18 b19
```

So `yd18` = yield in 2018, `app19` = canning appearance in 2019, etc. These are **not** raw field measurements — they are **BLUPs**: one cleaned, field-noise-adjusted value per line per year. *How* raw plot data became BLUPs is the entire subject of **Lesson 3**.

🧠 **Why store BLUPs, not raw data?** Because every downstream model wants *one number per line* ("the genetic value we're trying to predict"), not a tangle of replicate plots contaminated by which corner of the field they sat in.

---

## 2.3 `geno` — the marker matrix

- 2,315 columns, each a **SNP** (single-nucleotide polymorphism — a single DNA position where lines differ, e.g. some have an **A**, others a **G**).
- Column names like `S01_79737` mean *chromosome 01, position 79,737 bp*.
- Values are **0, 1, 2** = the count of one of the two alleles a line carries at that SNP.
    - For a homozygous self-pollinating bean, you mostly see **0** (zero copies) or **2** (two copies); **1** (heterozygous) is rarer.

🧱 **What the numbers mean.** Pick a SNP with alleles A/G. Code the "G" allele:

- `AA` → 0 copies of G → **0**
- `AG` → 1 copy → **1**
- `GG` → 2 copies → **2**

This 0/1/2 encoding turns DNA into something we can do arithmetic on — average it, correlate it, multiply it by effect sizes. That arithmetic *is* genomic prediction.

> 🔬 **In the data.** Allele coding range is exactly 0–2; SNPs are spread across all **11 chromosomes** (166–273 SNPs each). Minor-allele frequencies (Lesson 4) range from ~0.007 up to 0.5. See `figures/03_maf_histogram.png`.

---

## 2.4 `nirs18` / `nirs19` — the light fingerprint

Near-infrared spectroscopy (NIRS) shines infrared light (1100–2200 nm) at dry beans and records how much is absorbed at each wavelength. The result is a **spectrum**: 551 numbers per line (one every 2 nm). Different chemistry (water, starch, protein, seed-coat compounds) absorbs differently, so the spectrum is an indirect *chemical fingerprint* of the bean.

🌱 **Breeding logic.** A NIRS scan is **fast, cheap, and non-destructive** — you keep the seed. *If* the spectrum carries information about canning quality, it could be a cheap stand-in for the expensive taste panel. Lesson 11 tests exactly this idea (spoiler: it didn't help much here, but it could help in other studies and the *method* is important to understand).

⚠️ The column names differ slightly between years (`nm18_1100` vs `nm19_nm19_1100`) — a tiny real-world data-cleaning wrinkle you'd fix before combining them.

---

## 2.5 `SNP_Position` — the map

A 3-column table — `SNP`, `Chromosome`, `Position` — giving each marker's physical location on the *P. vulgaris* v2.1 reference genome. You need this for two things:

1. **GWAS** (Lesson 9): to draw a **Manhattan plot** you must know where each SNP sits.
2. **Interpreting hits**: a significant SNP at chr03:5,000,000 can be looked up against known genes.

---

## 2.6 What's *not* in the file (and where it lives)

- **Raw plot data** (individual field plots with row/column positions) — used by the spatial analysis (Lesson 3). The repo's `0.Spatial_Analysis.r` reads a separate Excel file (`BBL_phenotype_2020.xlsx`) not included here; the *output* of that step (the BLUPs) is what lives in `pheno`. So we **explain** Lesson 3 fully but reproduce it conceptually.
- **Raw DNA sequence reads** — these are huge; they live on NCBI (BioProject **PRJNA1138671**) and were processed into `geno` by the bioinformatics pipeline in Lesson 4.

---

## 2.7 A 30-second hands-on

```
load("repo/GB_BLB.RData")
str(GB_BLB, max.level = 1)            # see the 5 compartments
GB_BLB$geno[1:5, 1:6]                 # a corner of the marker matrix (0/1/2)
summary(GB_BLB$pheno$yd18)            # distribution of 2018 yield BLUPs
sum(!is.na(GB_BLB$pheno$yd18))        # 272  -> cycle-1 lines
```

You now know what every input is. Next we earn the right to call `pheno` "clean".

👉 Next: **Lesson 3 — Phenotyping & Spatial BLUPs.**

# Lesson 3 — Phenotyping & Spatial BLUPs

> **The question:** A field is not a spreadsheet. One corner is wetter, one edge gets more sun, a tractor compacted one strip. How do we turn *messy plot measurements* into **one fair number per line** that reflects its *genetics*, not its lucky/unlucky spot? Answer: a **mixed model** that produces a **BLUP**. This is repo script `0.Spatial_Analysis.r`.

---

## 3.1 Why raw plot values lie

Each line is grown in small **plots**, arranged in a grid of **rows ®** and **columns/passes (P)**. Two identical lines in two different plots can yield differently *purely because of where they sat*. This spatial nuisance is **field heterogeneity**.

🧠 **Intuition.** If I judge students' "ability" by exam scores, but some students sat next to a noisy window, I should *adjust* for the window before ranking them. The field's "windows" are moisture gradients, fertility patches, edge effects. We want each line's score *after* removing the seat it happened to get.

🌱 **Breeding logic.** Select on raw plot values and you'll sometimes promote a mediocre line that sat in a fertile patch, and cull a great line stuck in a bad one. Spatial adjustment protects genetic gain from being wasted on noise.

---

## 3.2 The experimental design (so the model has something to lean on)

- **2018:** an **incomplete block design**; 200 lines replicated twice, 72 lines (seed-limited) once.
- **2019:** a **complete block design**, 2 replications.
- Three **check** cultivars — **Eclipse, Zorro, Zenith** — are planted repeatedly throughout. Checks are *known* genotypes grown in many spots, so they act like **rulers**: by seeing how the *same* check varies across the field, the model learns the field's gradient.
- Plots are **4-row, 50-cm spacing, trimmed to 4.5 m**; the center 2 rows are harvested.

⚠️ **Common confusion — replication vs. checks.** *Replication* (growing a line more than once) averages out noise for *that* line. *Checks* (a few genotypes grown many times everywhere) map the *shape* of the field's gradient so it can be subtracted from *all* lines. You need both.

---

## 3.3 The model: a linear mixed model with a 2-D spatial surface

The authors use the R package **SpATS** (Spatial Analysis of field Trials with Splines). The model for a trait $y$ measured on each plot is:

🧱 **The mixed model.**

$$y_{ijk} = \underbrace{\mu}_{\text{overall mean}} + \underbrace{c_k}_{\text{check vs. non-check (fixed)}} + \underbrace{g_i}_{\text{genotype (random)}} + \underbrace{r_j + p_l}_{\text{row, column (random)}} + \underbrace{f(P,R)}_{\text{smooth 2-D field surface}} + \underbrace{e_{ijk}}_{\text{leftover error}}$$

Term by term, in plain words:

- $\mu$ — the grand average of the trait.
- $c_k$ — a **fixed effect** distinguishing the check cultivars from the breeding lines.
- $g_i$ — the **genotype effect**: *this is the thing we want* — line $i$'s genetic deviation from the mean. Fitted as **random** (see box below).
- $r_j, p_l$ — random row and column effects (discrete field structure).
- $f(P, R)$ — a **smooth 2-D spline surface** over the field (the PSANOVA term). Think of draping a flexible sheet over the field to capture the gradual fertility/moisture trend.
- $e_{ijk}$ — what's left: pure noise.

In the actual SpATS call (from the repo):

```
fit.yd18 <- SpATS(response = "yield",
                  spatial  = ~ PSANOVA(P, R, nseg = c(20, 10)),  # the 2-D surface
                  genotype = "ID", genotype.as.random = TRUE,    # we want BLUPs of genotype
                  fixed    = ~ CheckStatus,                       # checks vs. lines
                  random   = ~ row_f + col_f,                     # row & column effects
                  data = pheno2018)
```

---

## 3.4 Fixed vs. random — the single most important distinction here

🧠 **Intuition.** A **fixed** effect is a category you care about *specifically* and want the exact estimate for (e.g., "is it a check?"). A **random** effect is one of *many exchangeable* levels drawn from a population, where you believe the levels shouldn't stray too far from the mean — so the model **shrinks** extreme estimates toward 0.

We treat **genotype as random**. Why? Because:

1. Our lines are a *sample* from the breeding program's gene pool.
2. **Shrinkage is desirable.** A line grown in only one (lucky) plot shouldn't get an extreme score on thin evidence — the model pulls it back toward the mean *in proportion to how little we know about it*. Well-replicated lines get shrunk less.

This shrinkage is exactly what makes the output a **BLUP** rather than a raw mean.

---

## 3.5 BLUP — what the letters mean

**BLUP = Best Linear Unbiased Prediction.** The genotype estimates $\hat{g}_i$ from the random model are BLUPs. Decoding the name:

- **Prediction** — because random effects are *predicted*, not "estimated" (a technical distinction; treat them as the model's best guess of each line's genetic value).
- **Best / Unbiased / Linear** — among all linear, unbiased predictors, BLUP has the smallest prediction-error variance. (You don't need the proof; you need the consequence: *it is the statistically optimal one-number summary of a line's genetics given this messy field.*)

🧱 **The shrinkage formula (intuition version).** For a line with mean $\bar{y}_i$ over $n_i$ plots, the BLUP looks roughly like

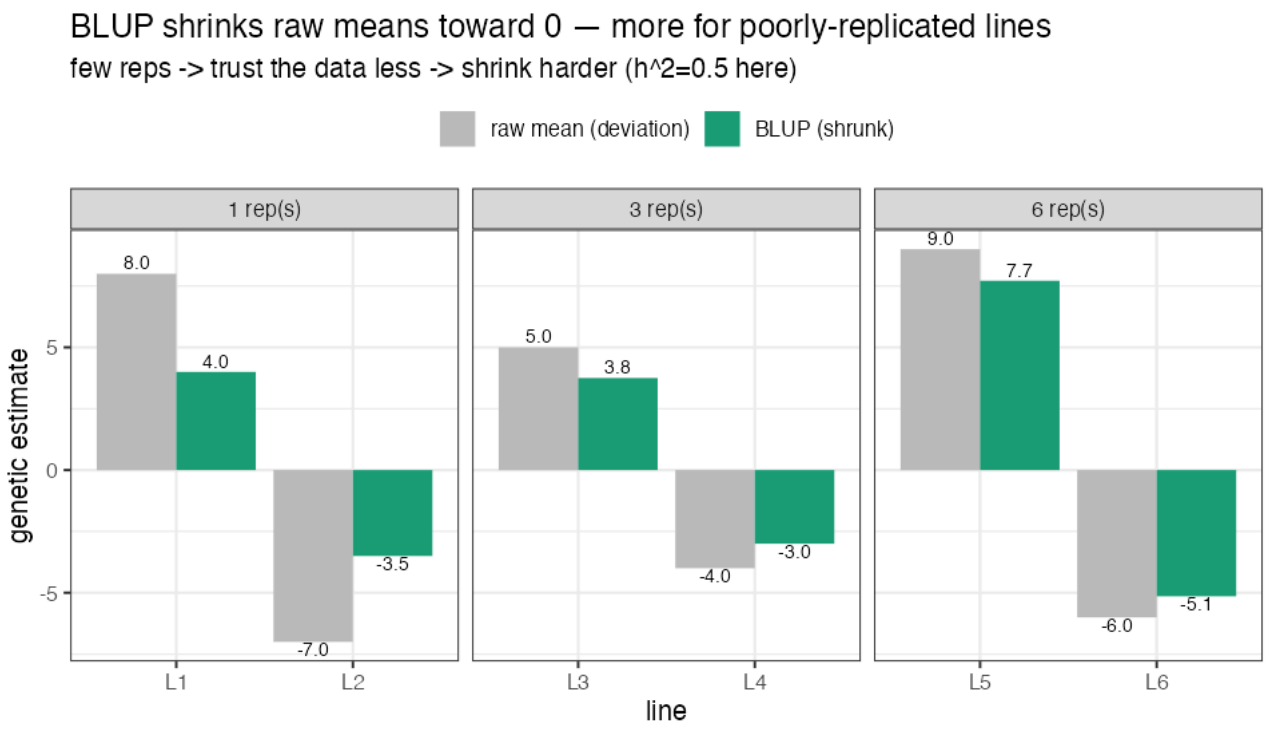
$$\hat{g}_i \approx w_i(\bar{y}_i - \mu), \qquad w_i = \frac{n_i \sigma_g^2}{n_i \sigma_g^2 + \sigma_e^2}$$

where $\sigma_g^2$ is genetic variance and $\sigma_e^2$ is error variance. Read it:

- More replicates ($n_i\uparrow$) → $w_i \to 1$ → trust the data, little shrinkage.
- More noise ($\sigma_e^2\uparrow$) → $w_i\downarrow$ → shrink harder toward the mean.
- More genetic signal ($\sigma_g^2\uparrow$) → $w_i\uparrow$ → trust the data more.

That ratio $\sigma_g^2/(\sigma_g^2+\sigma_e^2)$ is precisely **heritability** — which is why heritability and BLUP shrinkage are two sides of one coin.

🧸 **Toy — watch shrinkage depend on replication.** Six lines, each with a raw mean deviation, but grown a different number of times. With $h^2 = 0.5$, the shrinkage weight $w_i = n_i h^2/(n_i h^2 + (1-h^2))$ pulls each raw value toward 0 — *hard* for 1-rep lines, *barely* for 6-rep lines:




A 1-rep line at raw +8.0 is shrunk to **+4.0** (we don't trust thin evidence); a 6-rep line at +9.0 stays **+7.7**. In the real study, lines with 2 reps (and the spatial model's borrowing of strength from neighbors) get exactly this treatment across *all* traits — turning raw plots into the trustworthy BLUPs in `GB_BLB$pheno`.

The repo extracts these BLUPs and stacks them into the table we met in Lesson 2:

```
yield18 <- predict(fit.yd18, which = "ID")[, c(1,7)]   # line name + its BLUP
# ... repeated for every trait & year, then join_all() into BBL_2020  ->  GB_BLB$pheno
```

---

## 3.6 Heritability, properly

🧱 **Definition (broad-sense, plot basis).**

$$H^2 = \frac{\sigma_g^2}{\sigma_g^2 + \sigma_e^2}$$

the fraction of total variance that is genetic. SpATS also reports a **generalized heritability** suited to spatial models. Either way:

🌱 **Why a breeder lives by $h^2$:**

- It tells you **how much of what you see is real (heritable)** and thus *selectable*.
- It **caps prediction accuracy**: a trait that's mostly noise has little genetic signal for the genome (Lesson 7) to recover.
- In this study, **color** had high $h^2$ (→ accuracy up to 0.93) while **yield** and **appearance** had lower $h^2$ (→ accuracy ~0.4–0.6), and **days-to-maturity** $h^2$ even dropped from 0.77 (2018) to 0.49 (2019) because of weather — directly lowering its accuracy.

⚠️ **Common confusion — $H^2$ vs $h^2$ vs genomic $h_g^2$.** *Broad-sense* $H^2$ = all genetic variance / total. *Narrow-sense* $h^2$ = only the **additive** part / total (additive = the part that "breeds true" and that GBLUP models — Lesson 5). The paper also reports a **genomic heritability** $h_g^2$ = the share of variance explained by the markers via a linear model. For near-homozygous self-pollinated lines, most genetic variance *is* additive, so these are close.

---

## 3.6b See it work (real SpATS run) — `code/06_spatial_demo.R`

The study's **raw plot data** ( `BBL_phenotype_2020.xlsx` ) was never released publicly — only its *output*, the BLUPs in `GB_BLB$pheno` , and the supplement holds figures/tables, not the field file. So we demonstrate the method on a **simulated field with known truth**: 150 genotypes (whose true genetic values we *set*), 2 reps on a 20×15 grid, plus a smooth fertility gradient and noise.

🔬 **What SpATS recovered:**

- Raw plot value vs. the *true* genotype effect: **r = 0.46** — field noise badly blurs the genetics.
- **SpATS BLUP** vs. the true genotype effect: **r = 0.84** — the spatial gradient is estimated and removed, and the genetics re-emerges.
- Generalized heritability: **0.65**.

> Figure `figures/09_spats_demo.png` : (left) the field gradient SpATS estimates and subtracts; (right) BLUPs lining up with the truth. This is L3 in one picture — *the model turns "where the plot sat" noise back into "what the genes do" signal.*

---

## 3.7 Why this is Step 0 of the pipeline

Notice the script is numbered `0.` That's deliberate. **Everything downstream predicts the BLUPs.** If the BLUPs are contaminated by field position, every fancy genomic model is just learning to predict *where the plot was*, not *what the genes do*. Clean phenotypes first; clever models second. This ordering — *phenotype hygiene before genomics* — is a habit worth internalizing.

> 🔬 **In the data.** The `pheno` table you'll model in Lessons 7+ is the *output* of this step. We reproduced the trait distributions across years ( `figures/01_trait_distributions.png` ) — note 2018 vs 2019 differ because 2018 was wetter, a year effect the per-year BLUPs preserve.

---

> 🔧 **In practice ®.** Spatial field BLUPs: **`SpATS`** (used here, 2-D splines) or `breedR` , `sommer` , `asreml` . General mixed-model BLUPs: `lme4` ( `lmer` ), `sommer` ( `mmer` ). Heritability helpers: the `heritability` package and `SpATS::getHeritability()` .

## 3.8 What you should now be able to say

- Field data is spatially biased; a **linear mixed model** with a **2-D spline** + row/column effects + checks removes that bias.
- **Genotype is fit as random** so its estimates are **shrunk** → those shrunken estimates are **BLUPs**, the optimal one-number genetic summary per line.
- The shrinkage weight *is* heritability; **heritability caps downstream prediction accuracy**.
- This is Step 0 because all later models predict these BLUPs.

👉 Next: **Lesson 4 — Genotyping by Sequencing & SNPs** — the other input: turning DNA into the 0/1/2 matrix.

# Lesson 4 — Genotyping by Sequencing & SNPs

> **The question:** A leaf is not a number. How do we go from *DNA in a tube* to the tidy **415 × 2,315 matrix of 0/1/2** we met in Lesson 2 — and *why* each cleaning step?
>
> **This lesson is special: we run the real pipeline on the study's own sequencing data** and watch a 0/1/2 matrix appear from raw reads. Scripts: `code/05a_demux.py`, `05b_pipeline.sh`, `05c_call_genotypes.py`.

---

## 4.1 What is a SNP, really?

DNA is a string from a 4-letter alphabet: **A, C, G, T**. Line up the same chromosome region from two bean lines and they're identical almost everywhere — *except* at scattered single positions where one has, say, **A** and another has **G**. Such a position is a **SNP** (Single-Nucleotide Polymorphism). Its two versions are **alleles**; we count copies of one allele → **0, 1, 2** (a diploid has two chromosome copies).

🧠 **Intuition.** A SNP is a *fork in the genetic road* shared across the population. Knowing which fork each line took, at thousands of positions, gives a high-resolution **genetic fingerprint** that tracks which chromosome segments a line inherited. Genes ride on those segments, so SNPs are **signposts** for genes we can't see directly.

---

## 4.2 Genotyping-by-Sequencing (GBS): the cheap way to find thousands of SNPs

Whole-genome sequencing every line is expensive. **GBS** sequences only a *reproducible ~1% slice* of the genome — the same slice in every plant:

1. **Cut the DNA with a restriction enzyme** (here **ApeKI**, which recognizes `GCWGC`). The enzyme slices only at its sites → the same fragments in every plant.
2. **Attach barcodes** — each sample gets a short unique DNA "name tag"; 96 samples are pooled ("multiplexed") into one sequencing lane (huge cost saving). This is what `repo/GBS_barcode_plate_info_blb.txt` records.
3. **Sequence** on Illumina → hundreds of millions of short reads, each tagged by sample.

🌱 **Breeding logic.** GBS is *affordable* — the whole premise of putting genomics into a routine breeding pipeline depends on cheap genotyping. GBS makes genomic selection *practical*.

---

## 4.2b 🧸 Toy first — the file formats, and one read → one genotype

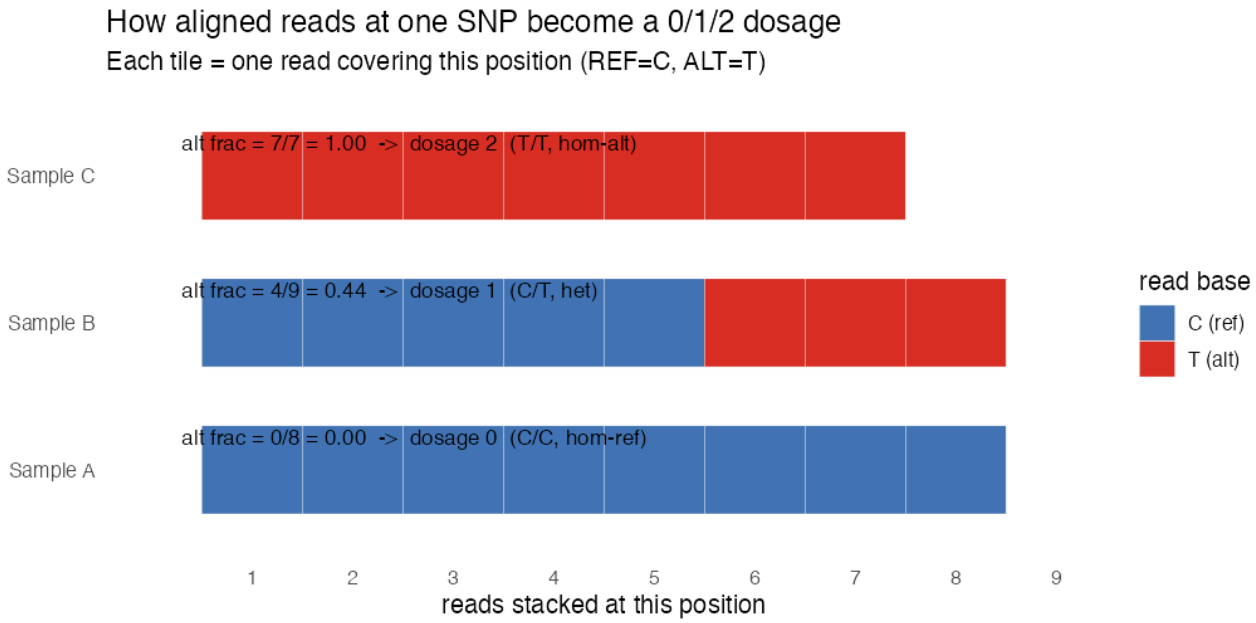
Before the real lane (millions of reads), let's meet the **file formats** on toy data so the real files aren't intimidating.

**(a) FASTQ — one sequencing read = 4 lines.** This is what comes off the machine:

```
@READ_001   ApheaKI read from sample BL1728-5      ← line 1: read name (starts with @)
TTCAAGT CAGC ACGGTTAGCCATTACGGGATACCAGTTACGAC      ← line 2: the DNA bases
+                                                  ← line 3: a separator (+)
IIIIIII IIII IIIIIIIIIIFFFFFFFFAAAAA<<<<<7777      ← line 4: per-base QUALITY (ASCII; I=great, 7=poor)
```

- Line 2 decodes (Lesson 4.3) as `[barcode TTCAAGT][ApeKI remnant CAGC][genomic ...]`.
- Line 4: each character is a quality score for the base above it — this is how "genotype quality < 30" filtering knows which calls to trust.

**(b) Align, then pile up.** Each read is matched to its place on the reference (Lesson 4.3). Now look at **one SNP position** where the reference says **C** but some lines carry **T**. Stack the reads from each sample and *count votes*:




**The rule (same as §4.3), applied by eye:**

- **Sample A:** 8 C, 0 T → alt fraction 0.00 → **dosage 0** (C/C).
- **Sample B:** 5 C, 4 T → alt fraction 0.44 → **dosage 1** (C/T, heterozygous).
- **Sample C:** 0 C, 7 T → alt fraction 1.00 → **dosage 2** (T/T).

**© VCF — where the calls are stored.** The genotype caller writes a **VCF** (Variant Call Format). One SNP = one row; samples are columns:

```
#CHROM  POS      REF  ALT  ...  FORMAT   SampleA   SampleB   SampleC
chr01   1081554  C    T    ...  GT:DP    0/0:8     0/1:9     1/1:7
```

- `GT` = **genotype**: `0/0` =two ref, `0/1` =one each (het), `1/1` =two alt. `DP` =read depth.
- **The 0/1/2 dosage you model is just the count of "1"s in GT:** `0/0` →**0**, `0/1` →**1**, `1/1` →**2**. *That single mapping is the entire bridge from sequencer to the* `geno` *matrix.*

**Now the real scale.** The `geno` matrix (415 × 2,315) is just this VCF, turned on its side (lines as rows, SNPs as columns) with every `GT` replaced by its 0/1/2 count — built from **millions** of the 4-line FASTQ records above, piled up at **thousands** of positions. Next we do exactly this on the study's *real* reads.

---

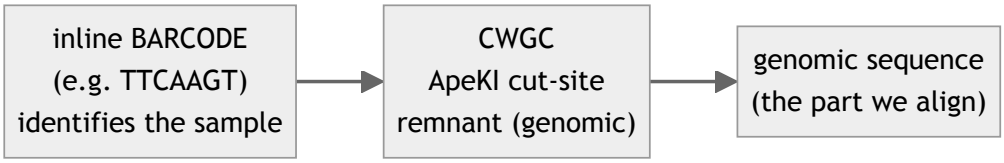
## 4.3 We did it for real — reads → 0/1/2

The study's raw reads live at **NCBI BioProject PRJNA1138671** (mirrored at ENA). There are 5 sequencing lanes, 10–26 GB each. We took the **smallest lane** ( `SRR29913416` ) and — because this laptop had ~2 GB free — **streamed a 6-million-read subset** rather than the full 361-million -read file, and aligned to **chromosome 1 only**. The *logic is identical at full scale*; only the size differs. Here is exactly what happened.

### Step 0 — look at a raw read

```
@SRR29913416.1 ...
CNGGCCGTACAGCCTTTTCGCAGCCAAATCAACTGCTTTCCCCGCTAGTG
```

A 50 bp read. Reading it left to right we *discover its structure*:




⚠️ Real-data wrinkle we *saw*: sequencing **cycle 2 failed** in the first reads (position 2 = `N`), so naive exact barcode-matching breaks. We matched barcodes *tolerantly* (ignoring position 2).

## Step 1 — demultiplex (`05a_demux.py`)

Sort reads back to samples by their barcode, then strip the barcode (keep the genomic part):

```
reads=6,000,000   matched=5,871,321 (97.9%)   samples found=91
top samples: BL1728-5 (241,537 reads), BL1709-3 (205,288), BL1728-2 (190,559), ...
```

🔬 **97.9% of reads carried a recognizable barcode** — a healthy library. We kept the 8 best- covered samples for the demo.

## Step 2 — trim → align → sort (`05b_pipeline.sh`)

Trim low-quality bases/adapters (`fastp`), align to chr01 (`bwa mem`), sort (`samtools`). Real output:

| sample | trimmed reads | mapped to chr01 | map rate |
|---|---|---|---|
| BL1728-5 | 239,332 | 40,950 | 17.1% |
| BL1709-3 | 201,757 | 37,706 | 18.7% |
| BL1704-4 | 185,201 | 33,319 | 18.0% |
| … | … | … | … |

🧠 Only ~17% map because we used **1 of 11 chromosomes** — the rest map to chromosomes 2–11 we didn't download. At full scale you'd align to all 11 and map ~90%+.

## Step 3 — call genotypes (`05c_call_genotypes.py`): *this is where 0/1/2 is born*

At each covered position we pile up the reads from all samples and, **per sample**, count how many reads support the **reference** allele vs an **alternative** allele, then convert the **allele fraction** to a dosage:

🧱 **The genotype rule (the simplest honest version):**

First the **alt fraction** $= \dfrac{\text{alt reads}}{\text{ref reads} + \text{alt reads}}$ (each term is a *count* of reads), then read off the **dosage**:

| alt fraction | dosage | genotype |
|---|---|---|
| ≤ 0.15 | **0** | hom-ref (AA) |
| 0.15 – 0.85 | **1** | heterozygous (Aa) |
| ≥ 0.85 | **2** | hom-alt (aa) |

(too few reads → call it **missing**). Real callers like **NGSEP** (the paper) or `bcftools` use a genotype-*likelihood* model, but the underlying idea is exactly this: *reads voting on alleles*.

🔬 **The real output — an actual 0/1/2 matrix from the study's DNA:**

```
sample       S01_1081554  S01_2327642  S01_2327647  S01_2327729 ...
BL1728-5     1            1            1            0
BL1709-3     1            0            0            0
BL1728-2     0            1            0            0
BL1704-4     NA           0            1            0
```

That is the **same shape and coding as** `GB_BLB$geno` (Lesson 2) — samples × SNPs, entries 0/1/2. *We built it from raw reads.* (`bioinfo/genotypes_012.tsv`, figure `figures/08_real_genotype_matrix.png`.)

---

## 4.4 Our demo *accidentally proves why filtering is essential*

Our naive caller reported **~30% heterozygous (1) calls** — and this stayed ~30% even when we demanded 15 reads per call:

| min depth | SNPs | het-rate |
|---|---|---|
| 5 | 90 | 30.0% |
| 10 | 19 | 29.0% |
| 15 | 7 | 29.5% |

🧠 **Why this is impossible — and therefore informative.** Beans **self-pollinate**, so lines are near-**homozygous**: real het should be **rare** (~0–5%), not 30%. Persistent high het that *doesn't* go away with more depth is the fingerprint of **repetitive / paralogous regions** — reads from two near-identical genome copies collapse onto one position and *look* like a constant 50/50 "heterozygote." This is an **artifact**, not biology.

🌱 **This is exactly what the paper's filters remove.** Now the QC chain makes visceral sense:

| Filter (paper) | Threshold | The artifact it kills |
|---|---|---|
| Remove **repetitive regions** | — | the paralog-collapse het cloud we just saw |
| **Biallelic** only | 2 alleles | the 0/1/2 model needs exactly two alleles |
| **Genotype quality** | < 30 dropped | low-confidence calls |
| **Heterozygosity** | obs. > 0.05 removed | **directly** removes our 30%-het artifact SNPs |
| **Missingness** | > 20% per SNP | SNPs with too many gaps |
| **MAF** | < 0.01 removed | ultra-rare variants (can't estimate effects) |
| **LD pruning** | $r^2 > 0.9$ | redundant near-duplicate SNPs |
| **Imputation** (Beagle) | — | *fills* remaining gaps using genetic neighbors |

After this chain on the *full* data: **2,315 clean SNPs** — the matrix you model in Lessons 6+.

⚠️ **Common confusion — "doesn't filtering throw away real data?"** Yes, deliberately. The goal is *maximum reliable signal per unit noise*, not maximum SNPs. Our experiment is the proof: skip the het/repeat filter and 30% of your "signal" is garbage.

---

## 4.5 Two filters worth the math

🧱 **Minor allele frequency (MAF).** With 0/1/2 coding, the alt-allele frequency is just (column mean)/2, and $\mathbf{MAF} = \min(p, 1-p) \in [0, 0.5]$. A SNP at MAF 0.005 means ~1 line in 200 carries the rare allele — too rare to estimate an effect, and a magnet for false GWAS hits.

> 🔬 We recomputed MAF on the final `geno`: range ~0.007–0.50, median ≈ 0.03 (`figures/03_maf_histogram.png`), consistent with the ≥0.01 filter.

🧱 **Linkage disequilibrium (LD).** Nearby SNPs are inherited together → correlated ($r^2$ high). Keeping 50 SNPs that say the same thing wastes computation; LD pruning keeps one **tag** SNP per $r^2 > 0.9$ block, losing little because the tag stands in for its neighbors.

---

## 4.6 The result, and why its *shape* drives everything next

We end with **M**, a 415 × 2,315 matrix in {0,1,2}. Two structural facts shape every later method:

1. **Wide, not tall.** Far more markers (2,315) than lines (415): $p \gg n$. Ordinary regression is impossible (more unknowns than equations). This *forces* the shrinkage/ relationship methods of Lessons 6–8.
2. **Rows are comparable.** Every line scored at the *same* SNPs → we can measure genetic *similarity* between any two lines → the genomic relationship matrix (Lesson 6).

🌱 You don't need a marker *in* every gene — just markers *near enough* to ride along (be in LD) with the genes. That "riding along" is the entire basis of marker-based prediction.

---

## 4.7 Run it yourself

```
# (needs bwa, samtools, fastp + python3; see Lesson 16)
python3 code/05a_demux.py 8        # stream-subset already fetched -> demultiplex top 8 samples
bash    code/05b_pipeline.sh        # trim -> align(chr01) -> sort
python3 code/05c_call_genotypes.py  # pileup -> 0/1/2 matrix + het diagnostics
```

> Full-scale notes are in `code/05b_pipeline.sh`: the paper used **Cutadapt + Bowtie2 + NGSEP**; we used the equivalent **fastp + bwa + a transparent caller** (and chr01 + a read subset) to fit a laptop. Same five logical steps: demultiplex → trim → align → pileup → genotype.

---

> 🔧 **In practice (tools).** Demultiplex/trim: `process_radtags` (Stacks), `Cutadapt`, `fastp`; align: `Bowtie2` / `BWA`; call genotypes: **`NGSEP`** (the paper), `bcftools`, or `GATK`; filter (MAF, missingness, het, biallelic): `VCFtools`, `PLINK`, `bcftools`; LD pruning & MAF: `PLINK`, R's `SNPRelate`; impute: `Beagle`. In R, read the final VCF with `vcfR` and build dosages.

## 4.8 What you should now be able to say

- A **SNP** is a single-position DNA difference, coded as allele-count **0/1/2**; **GBS** cheaply finds thousands by cutting (ApeKI) + barcoding + sequencing a reproducible genome slice.

- The pipeline is **demultiplex → trim → align → pileup → call**; we ran it on the study's own reads and watched a real 0/1/2 matrix form (`figures/08_real_genotype_matrix.png`).
- Calling 0/1/2 = **reads voting on alleles** (allele fraction → dosage); real callers add a likelihood model.
- A naive run gave **30% het** — a repeat/paralog artifact — which is *exactly why* the paper's **het/repeat/quality/MAF/LD** filters exist; after them, **2,315** clean SNPs remain.
- The matrix is **wide** ($p \gg n$), which forces the shrinkage methods coming next.

👉 Next: **Lesson 5 — Quantitative Genetics Foundations**.

# Lesson 5 — Quantitative Genetics Foundations

> **The question:** Genomic prediction predicts a line's **genetic value**. But what *is* a genetic value, in equations? This short lesson gives you the four ideas — **additive effect, breeding value, genetic variance, heritability** — that every later model rests on.

You met heritability intuitively in Lesson 3. Now we make it precise enough to *use*.

---

## 5.1 One gene, written as a number

Take a single SNP with alleles, count copies of one allele → $x \in \{0, 1, 2\}$ (Lesson 2). Suppose each copy adds, on average, a fixed amount $\alpha$ to the trait. Then that locus contributes:

$$\text{contribution} = \alpha \cdot x$$

- $x = 0$: contributes $0$
- $x = 1$: contributes $\alpha$
- $x = 2$: contributes $2\alpha$

🧠 **Intuition.** $\alpha$ is the **additive effect** — the "price per copy" of the allele. "Additive" because copies just *add up*. This linear, additive picture is the backbone of classical quantitative genetics and of GBLUP.

⚠️ **Common confusion — what about dominance/epistasis?** Real genetics also has **dominance** (the heterozygote isn't exactly halfway) and **epistasis** (genes interacting). The *additive* part is special because it's the part that **passes reliably from parent to offspring** — so it's what breeders select on. GBLUP models the additive part; the **kernel** methods of Lesson 8 are a way to *also* capture some non-additive effects.

---

## 5.2 Many genes → the breeding value

A complex trait (yield, appearance) is influenced by **many** loci. Sum each locus's additive contribution over all $p$ markers:

🧱 **The genetic (breeding) value of line $i$:**

$$g_i = \sum_{j=1}^{p} \alpha_j x_{ij}$$

- $x_{ij}$ — line $i$'s genotype (0/1/2) at marker $j$.
- $\alpha_j$ — additive effect of marker $j$.
- $g_i$ — line $i$'s total **breeding value**: the sum of all its allele "prices".

In matrix form for all lines at once: $\mathbf{g} = \mathbf{M}\boldsymbol{\alpha}$, where $\mathbf{M}$ is the 415 × 2,315 marker matrix and $\boldsymbol{\alpha}$ the vector of effects.

### 🧸 Toy example first — 5 lines, 10 SNPs (so $i$, $j$, $\alpha$, $g$ become concrete)

Symbols are slippery until you compute one by hand. Let's shrink the real 415 × 2,315 matrix to a toy **$\mathbf{M}$ of 5 lines × 10 SNPs** and *invent* the effects $\boldsymbol{\alpha}$ (in reality $\boldsymbol{\alpha}$ is unknown — estimating it is what Lessons 7–8 do; here we hand it to you to see the machinery). Run `code/toy_05_breeding_value.R`:

**Step 1 — the genotypes $x_{ij}$** (row = line $i$, column = SNP $j$, value = copies 0/1/2):

| line\SNP | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| **L1** | 2 | 0 | 1 | 2 | 0 | 2 | 0 | 0 | 1 | 2 |
| **L2** | 0 | 0 | 2 | 2 | 0 | 0 | 1 | 2 | 1 | 0 |
| **L3** | 2 | 1 | 0 | 0 | 2 | 2 | 0 | 0 | 0 | 2 |
| **L4** | 0 | 2 | 2 | 1 | 0 | 0 | 2 | 2 | 0 | 0 |
| **L5** | 1 | 0 | 0 | 2 | 1 | 2 | 0 | 1 | 2 | 2 |

**Step 2 — the effect per copy $\alpha_j$** (one number per SNP; + helps the trait, − hurts, 0 = no effect):

| SNP $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| $\alpha_j$ | **+2** | 0 | **−1** | +1 | 0 | **+3** | 0 | **−2** | +1 | 0 |

**Step 3 — breeding value $g_i = \sum_j \alpha_j x_{ij}$.** This sum *is* a **matrix × vector** multiplication, $\mathbf{g} = \mathbf{M}\boldsymbol{\alpha}$. Written out in full, the 5×10 genotype matrix times the 10×1 effect vector gives the 5×1 breeding-value vector:

Reading it as $\mathbf{M}\,(5 \times 10) \times \boldsymbol{\alpha}\,(10 \times 1) = \mathbf{g}\,(5 \times 1)$ — each line's genotype **row** times the shared **effect column** gives that line's breeding value:

| line $i$ | genotype row of $\mathbf{M}$ (SNP 1 → 10) | $g_i = \sum_j \alpha_j x_{ij}$ |
|---|---|---|
| **L1** | `2 0 1 2 0 2 0 0 1 2` | **12** |
| **L2** | `0 0 2 2 0 0 1 2 1 0` | **−3** |
| **L3** | `2 1 0 0 2 2 0 0 0 2` | **10** |
| **L4** | `0 2 2 1 0 0 2 2 0 0` | **−5** |
| **L5** | `1 0 0 2 1 2 0 1 2 2` | **10** |

…with the **same** effect column $\boldsymbol{\alpha} = (2,\ 0,\ -1,\ 1,\ 0,\ 3,\ 0,\ -2,\ 1,\ 0)$ applied to every row.

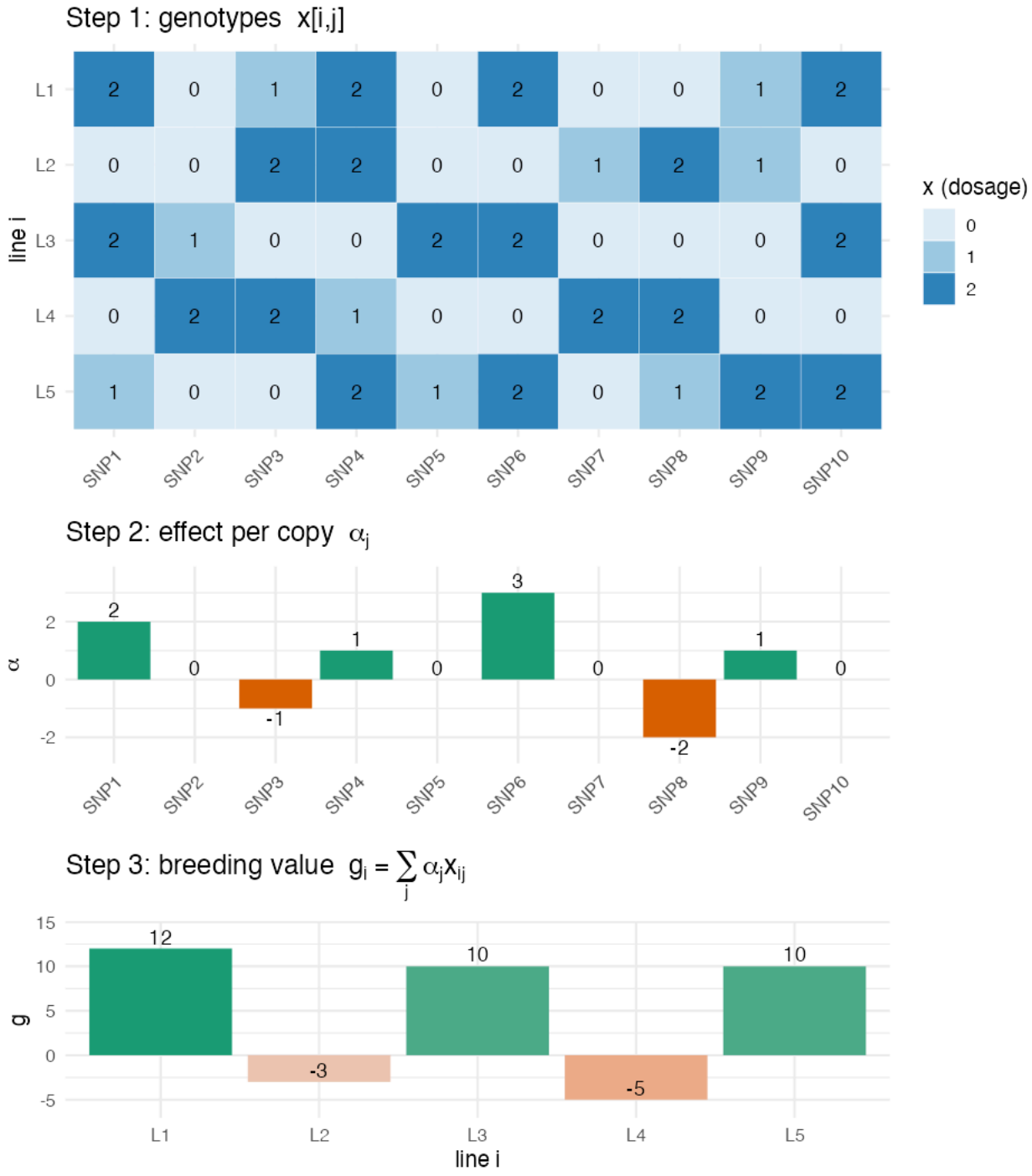
**How a matrix times a vector actually works:** each entry of $\mathbf{g}$ is the **dot product** of *one row* of $\mathbf{M}$ with the *whole column* $\boldsymbol{\alpha}$ — multiply them position-by-position, then add. Row **L1** with $\boldsymbol{\alpha}$ (showing only the nonzero terms):

$$g_{\text{L1}} = \sum_{j=1}^{10} \alpha_j x_{\text{L1},j} = \underbrace{2\cdot 2}_{\text{SNP1}} + \underbrace{(-1)\cdot 1}_{\text{SNP3}} + \underbrace{1\cdot 2}_{\text{SNP4}} + \underbrace{3\cdot 2}_{\text{SNP6}} + \underbrace{1\cdot 1}_{\text{SNP9}} = 4 - 1 + 2 + 6 + 1 = \mathbf{12}$$

And row **L4**, to see a *negative* breeding value appear (its alt alleles sit mostly on the penalty SNPs, $\alpha_3 = -1$, $\alpha_8 = -2$):

$$g_{\text{L4}} = \underbrace{(-1)\cdot 2}_{\text{SNP3}} + \underbrace{1\cdot 1}_{\text{SNP4}} + \underbrace{(-2)\cdot 2}_{\text{SNP8}} = -2 + 1 - 4 = \mathbf{-5}$$

Repeat for all five rows and you read off the result vector $\mathbf{g} = [12,\ -3,\ 10,\ -5,\ 10]^{\top}$. **That vector is the ranking a breeder wants** — L1, L3, L5 are the keepers; L2 and L4 are culls.




🔭 **Now zoom out to the real data.** Picture that little 5×10 grid stretched to **415 rows × 2,315 columns**, and the $\boldsymbol{\alpha}$ bar-chart to **2,315 bars** (most tiny, a few bigger). The operation is *identical* — multiply genotypes by effects and sum across the row — just with more of everything. The single hard difference: in the toy we *knew* $\boldsymbol{\alpha}$; with real data we must **estimate** all 2,315 effects from only 415 lines ($\boldsymbol{p \gg n}$), which is impossible by ordinary regression. That impossibility (§5.5) is the doorway to GBLUP.

🌱 **This is the target of genomic prediction.** A line's **breeding value** $\boldsymbol{g_i}$ is exactly what a breeder wants to rank lines by — it's the heritable, transmissible merit. The phenotype we observe is $\boldsymbol{y_i = \mu + g_i + e_i}$: breeding value plus environmental noise $\boldsymbol{e_i}$. **We see $y$; we want $g$.** Every model in this course is a different strategy for recovering $\boldsymbol{g}$ from $\boldsymbol{y}$ and the markers.

---

## 5.3 The phenotype = signal + noise

### 🧱 The decomposition.

$$\underbrace{y_i}_{\text{what we measure}} = \underbrace{\mu}_{\text{mean}} + \underbrace{g_i}_{\text{genetics (signal)}} + \underbrace{e_i}_{\text{environment (noise)}}$$

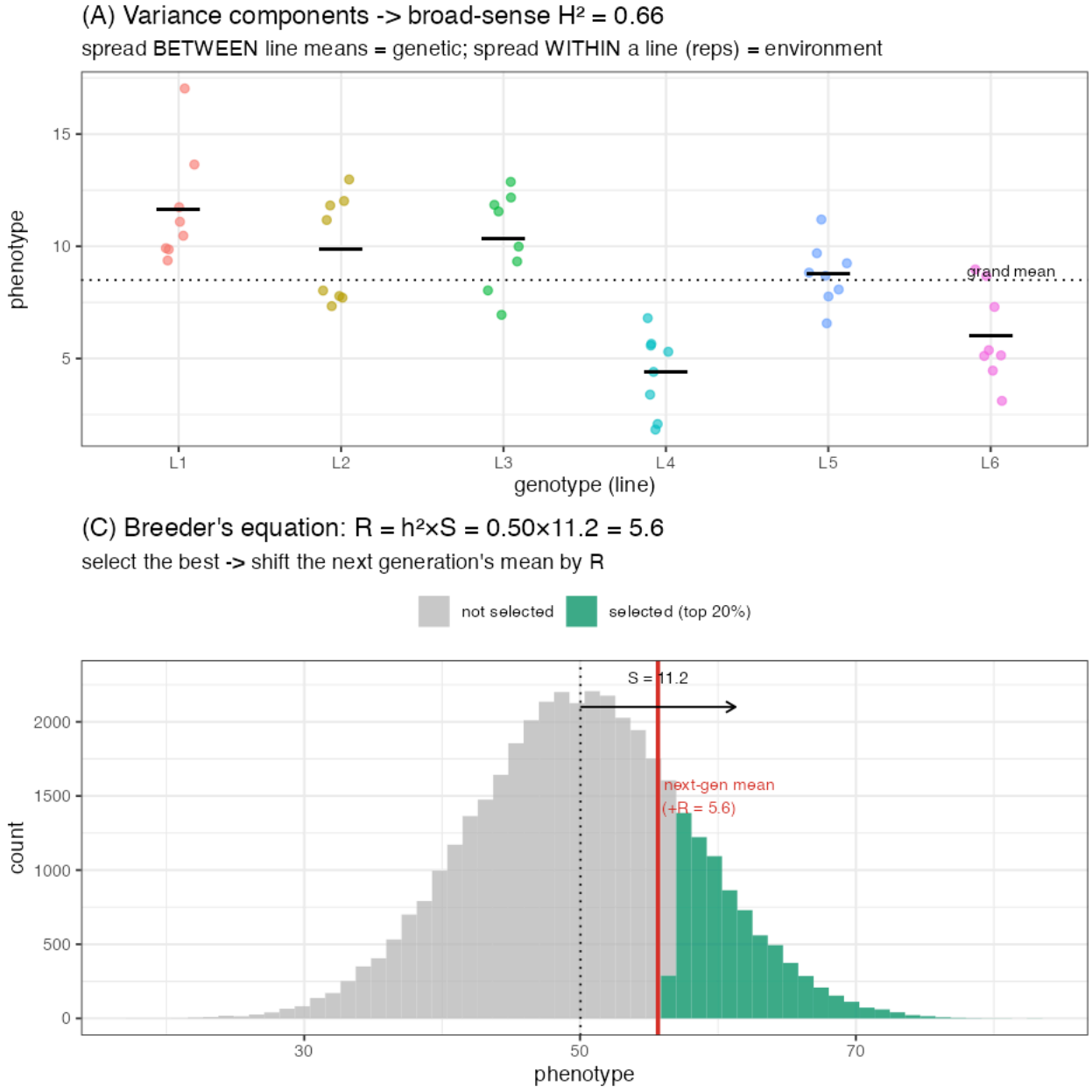
Taking variances across lines (signal and noise independent):

$$\sigma^2_P = \sigma^2_g + \sigma^2_e$$

total **phenotypic** variance = **genetic** variance + **environmental** variance.

### 🧸 Toy first — split the variance with replicated lines ( `code/toy_05b_quantgen.R` )

How do we actually *get* $\sigma^2_g$ and $\sigma^2_e$? Grow each genotype **several times** (reps). Then the spread **between** line means is genetic, and the spread **within** a line (same genes, different plots) is environmental. With 6 toy lines × 8 reps:




🔬 From the toy (panel A): $\sigma^2_g = 8.4$, $\sigma^2_e = 4.3$, so $\sigma^2_P = 12.7$ and

$$H^2 = \frac{\sigma_g^2}{\sigma_g^2 + \sigma_e^2} = \frac{8.4}{12.7} = 0.66.$$

🔭 **Zoom out:** the study's SpATS model (Lesson 3) does exactly this partition for 415 lines across a noisy field — replication + checks let it separate genetics from "which plot you sat in," then reports a (generalized) heritability per trait.

---

## 5.4 Heritability, defined three ways (all the same idea)

🧱 **Narrow-sense heritability:**

$$h^2 = \frac{\sigma_A^2}{\sigma_P^2} = \frac{\text{additive genetic variance}}{\text{total phenotypic variance}}$$

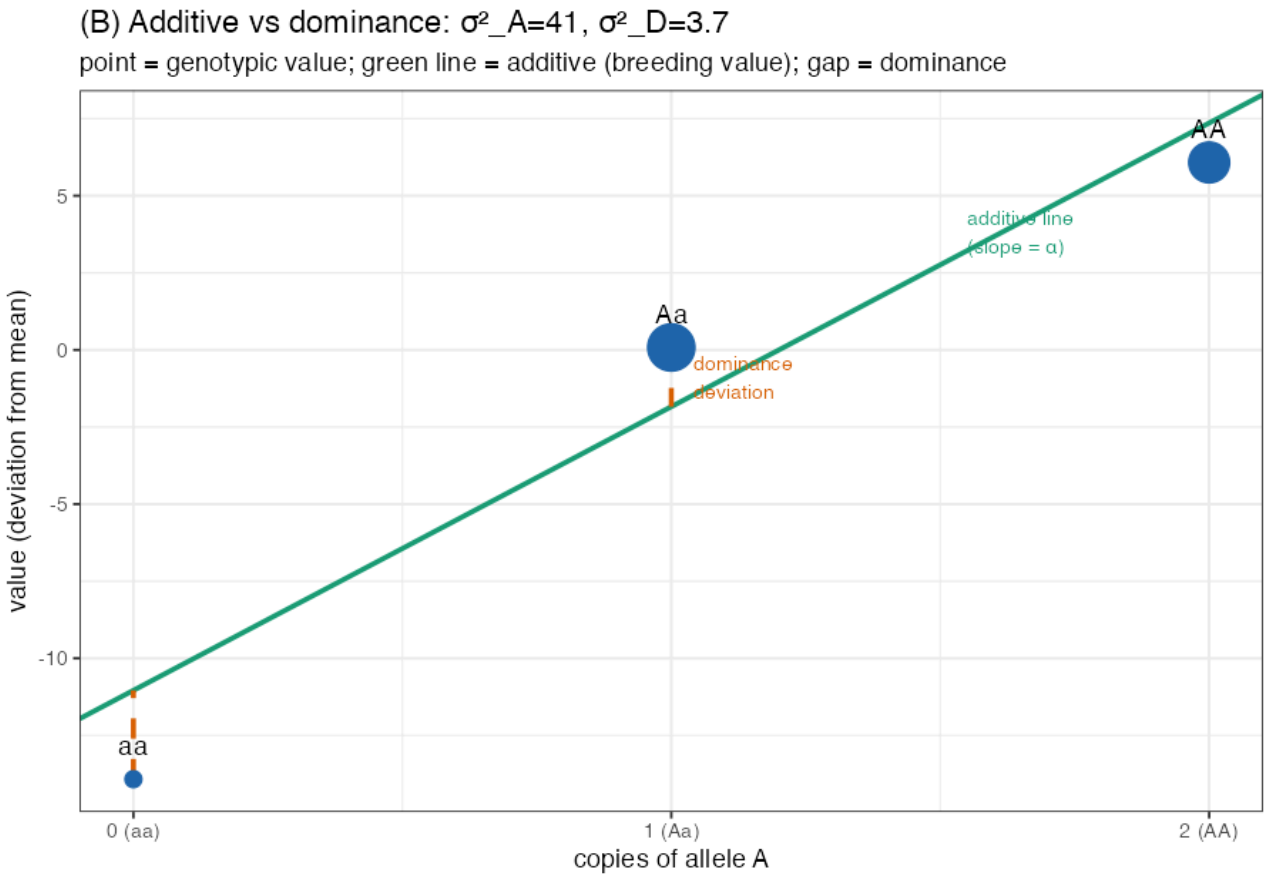
**The three heritabilities, side by side:**

| symbol | name | numerator | answers |
|---|---|---|---|
| $H^2$ | broad-sense | *all* genetic variance $\sigma_g^2 = \sigma_A^2 + \sigma_D^2 (+ \ldots)$ | "how much is genetic at all?" |
| $h^2$ | narrow-sense | only **additive** $\sigma_A^2$ | "how much breeds true to offspring?" |
| $h_g^2$ | genomic | variance the **markers** capture | "how much can the SNPs explain?" |

All three share the denominator $\sigma_P^2$. For near-homozygous selfed bean lines, $\sigma_D^2$ is tiny, so $H^2 \approx h^2 \approx h_g^2$ — which is *why* additive GBLUP works so well here.

### 🧸 Toy first — where does $\sigma_D^2$ come from, and why $h^2 \leq H^2$? ( `code/toy_05b_quantgen.R` )

Take **one** locus (alleles A/a). Falconer's model assigns genotypic values $-a, d, +a$ to $aa, Aa, AA$ ($a$ = additive size, $d$ = dominance: $d \neq 0$ means the heterozygote isn't halfway). Fit the **best straight line** of genotypic value on *number of A alleles* — its slope is the **average effect** $\alpha$, and the line gives each genotype's **breeding value** (the additive, heritable part). The **gaps** between the points and the line are **dominance deviations**:




🔬 With $a = 10$, $d = 4$, allele freq $p = 0.6$:

$$\sigma^2_A = 2pq[a + d(q - p)]^2 = 40.6, \qquad \sigma^2_D = (2pqd)^2 = 3.7$$

So the locus is **92% additive**. The additive part ($\sigma^2_A$) goes into $h^2$ and **passes to offspring**; the dominance part ($\sigma^2_D$) inflates $H^2$ but **doesn't breed true** (it depends on getting the *pair* of alleles together again). That gap is precisely why $h^2 \leq H^2$.

> 🔧 **In practice ®.** Variance components & heritabilities come from mixed-model packages: `sommer` (`mmer`, with additive **and** dominance kernels via `A.mat` / `D.mat`), `BGLR`, or `lme4` / `heritability`. The paper's genomic $h^2_g$ is the marker-explained share from a GBLUP fit.

Read $h^2$ three equivalent ways:

1. **Fraction of variance that's heritable** (the definition).
2. **The BLUP shrinkage weight** (Lesson 3): how much we trust the data vs. shrink to the mean.
3. **A ceiling on prediction accuracy**: you cannot predict the noise; you can at best recover the genetic part, so accuracy is bounded by (a function of) $h^2$.

🌱 **Breeding payoff — the breeder's equation.** Genetic gain per cycle is

$$\Delta G = ih^2\sigma_P \quad \text{(mass selection form)} \quad \Rightarrow \quad \Delta G = ir\sigma_A \text{ (general form)}$$

where $i$ = selection intensity (how picky you are), $r$ = accuracy of selection, $\sigma_A$ = additive SD. **This equation is the *reason genomic prediction exists*:** genomic selection raises $\Delta G$ by (a) increasing **accuracy** $r$ on hard-to-measure traits and (b) letting you select *earlier/faster*, shortening the cycle so $\Delta G$ accumulates per *year* faster. Everything in this study is ultimately an attempt to push $r$ up.

### 🧸 Toy first — selection differential $S$ → response $R$ (`code/toy_05b_quantgen.R`)

The breeder's equation in its most tangible form is $R = h^2 S$: select the best, and the next generation moves by the *heritable fraction* of how special your selections were. Toy: a population with mean 50, $h^2 = 0.5$; keep the **top 20%** (panel C of the figure above):

$$S = \bar{y}_{\text{selected}} - \bar{y}_{\text{all}} = 11.2, \qquad R = h^2 S = 0.5 \times 11.2 = 5.6$$

so the next generation's mean rises from **50 → 55.6**. ⚠️ Note the lever: with $h^2 = 0.5$ you only *keep half* of the 11.2 you selected for — **low heritability throttles response**, which is the breeder's daily reality and the reason genomic prediction (raising the *accuracy* term $r$) matters.

> 🔬 **In the data — we measured it (`code/07_heritability_accuracy.R`).** For every 2018 trait we estimated genomic heritability $h^2_g$ (from GBLUP variance components) *and* the GBLUP prediction accuracy, then plotted one against the other: **the correlation across traits is r = 0.80** — high-$h^2$ color/L*/b* predict at 0.72–0.79, while the lowest-$h^2$ trait (days-to-flowering) sits at 0.46. See `figures/10_heritability_vs_accuracy.png`. The paper sees the same: **days-to-maturity**'s $h^2$ fell 0.77→0.49 from 2018 to 2019 and its accuracy fell with it. **Heritability is not abstract — it *is* (most of) the accuracy ceiling.** (These $h^2_g$ are estimated on the cleaned BLUPs, so they run a bit higher than the paper's plot-basis values; the *relationship* is the point.)

---

## 5.5 The crucial pivot: from effects ($\alpha$) to relationships (G)

Here is the conceptual hinge of the whole course. We wrote breeding value as $\mathbf{g} = \mathbf{M}\boldsymbol{\alpha}$. To predict, it seems we must estimate all 2,315 effects $\boldsymbol{\alpha}$ — but with only 415 lines ($p \gg n$, Lesson 4) we *can't* do that by ordinary least squares.

**Two escape routes, and they turn out to be the same math:**

- **Route A (markers):** estimate all $\alpha_j$ but *shrink* them toward 0 (penalize big effects). This is **ridge regression / RR-BLUP** (Lesson 7).
- **Route B (relationships):** never estimate individual $\alpha_j$; instead use the fact that *genetically similar lines have similar breeding values*. Summarize similarity in one **genomic relationship matrix G** (Lesson 6) and predict via that. This is **GBLUP**.

🧠 **Intuition for why B works.** You don't need to know *which* genes a champion line carries to predict its relatives are good — you just need to know *who its relatives are*. Pedigree breeders did this for a century with family trees; markers let us measure relatedness directly, even between lines with no recorded pedigree.

It is a beautiful and load-bearing fact that **Route A and Route B give identical predictions.** We'll see *why* in Lesson 7. For now, internalize the pivot:

> **Predicting breeding values ≠ finding genes. It = exploiting relatedness.**

This is also exactly why **GWAS** (finding genes, Lesson 9) and **GP** (predicting, Lesson 7) are *different jobs*: GWAS asks "*which* loci?", GP asks "*how good* is this line?" — and the study's Objective 3 is the surprising discovery that doing the first does **not** automatically help the second.

---

## 5.6 What you should now be able to say

- An **additive effect** $\alpha_j$ is the per-copy value of an allele; a **breeding value** $g_i = \sum_j \alpha_j x_{ij}$ is the sum over all loci — the heritable merit we want to rank.
- Phenotype = mean + breeding value + environment; **heritability** $h^2 = \sigma^2_A / \sigma^2_P$ is the genetic fraction, and it **caps prediction accuracy** and drives **genetic gain** ($\Delta G = ir\sigma_A$).
- Because $p \gg n$, we predict breeding values via **shrinkage on markers (ridge)** or, equivalently, via **relatedness (G matrix / GBLUP)** — *not* by finding individual genes.

👉 Next: **Lesson 6 — The Genomic Relationship Matrix G** — we build the relatedness matrix by hand.

# Lesson 6 — The Genomic Relationship Matrix G

> **The question:** Lesson 5 said prediction = *exploiting relatedness*. So we need a precise, number-for-number answer to **"how genetically related is line $i$ to line $j$?"** for all pairs. That answer is the matrix **G**. We build it by hand on 4 lines, then on the real 415.

---

## 6.1 What G is

**G** is a square table, one row and one column per line (415 × 415 here). Entry $G_{ij}$ measures the **genomic relationship** (realized kinship) between lines $i$ and $j$:

- **Large positive $G_{ij}$** → lines share many alleles → closely related.
- **Near zero** → unrelated.
- **Negative** → *less* related than two random lines (more genetically opposite than average).
- The **diagonal $G_{ii}$** ≈ how inbred/typical line $i$ is (≈1 on average).

🧠 **Intuition.** G is a *friendship-by-DNA* map. Pedigree breeders drew family trees to guess relatedness ("these two are cousins → ~12.5% shared"). Markers let us **measure** it directly — and catch surprises a pedigree misses (two "unrelated" lines that happen to share a lot, or full sibs that drifted apart). G is the *data-driven* upgrade of the pedigree relationship matrix.

---

## 6.2 The formula (VanRaden 2008) — exactly what the paper used

🧱 **Step by step.** Start from the marker matrix $\mathbf{M}$ (415 × 2,315, entries 0/1/2).

1. **Center & scale each SNP column** → matrix $\mathbf{Z}$. For SNP $j$ with mean $2p_j$ and SD $s_j$:

$$Z_{ij} = \frac{M_{ij} - 2p_j}{s_j}$$

   Centering removes the "average dose" so we measure *deviation* from the population; scaling puts every SNP on a comparable footing.

2. **Cross-multiply and average over markers:**

$$\boxed{\mathbf{G} = \frac{\mathbf{Z}\mathbf{Z}^{\top}}{p}} \qquad (p = \text{number of markers})$$

In R this is literally the line the authors wrote (and we reproduced):

```
Z <- scale(M)              # center + scale every SNP
G <- tcrossprod(Z) / ncol(M)   # Z Z' / p
```

🧠 **Why this *is* relatedness.** $G_{ij} = \frac{1}{p}\sum_j Z_{ij}Z_{kj}$ is just the **average product of standardized genotypes** across all SNPs — i.e. how *correlated* lines $i$ and $j$ are across the genome. If they tend to be high together and low together at the same SNPs, the products are positive and pile up → big $G_{ij}$. It's a correlation in disguise.

## 6.3 Build it by hand — 4 lines, 3 SNPs

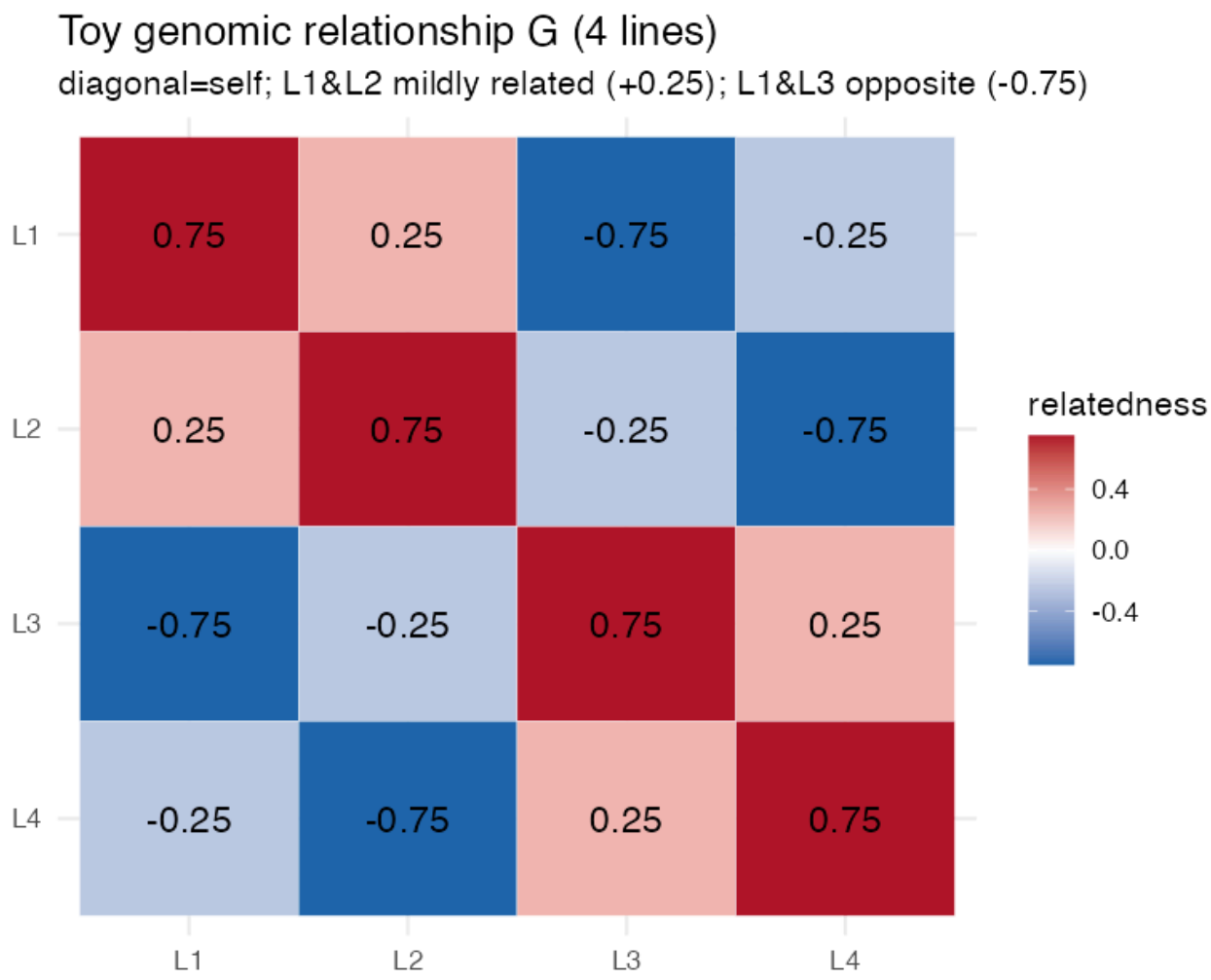
Take a toy with 4 lines and 3 SNPs (we ran this in R):

```
M (0/1/2):              Z (centered+scaled):        G = ZZ'/p:
    SNP1 SNP2 SNP3           SNP1  SNP2  SNP3           L1    L2    L3    L4
L1   2    2    0      L1   0.87  0.87 -0.87     L1   0.75  0.25 -0.75 -0.25
L2   2    0    0      L2   0.87 -0.87 -0.87     L2   0.25  0.75 -0.25 -0.75
L3   0    0    2      L3  -0.87 -0.87  0.87     L3  -0.75 -0.25  0.75  0.25
L4   0    2    2      L4  -0.87  0.87  0.87     L4  -0.25 -0.75  0.25  0.75
```

Read the result like a breeder:

- $G_{12} = 0.25$ (L1 vs L2): they match at SNP1 (both 2) and SNP3 (both 0) but differ at SNP2 → mildly related. ✔ makes sense.
- $G_{13} = -0.75$ (L1 vs L3): opposite at *every* SNP → strongly *un*alike, below average. ✔
- $G_{11} = 0.75$ (diagonal): L1's relationship with itself = how far it sits from the population average across SNPs.

That's the whole idea. The same toy as a **color heatmap** (run via `code/`):




🔭 **Zoom out:** the real G is this exact picture at **415 × 415** — a giant heatmap whose red blocks are families of related lines and blue regions are unrelated pairs (we draw the real one in §6.4). The real G just does this over 2,315 SNPs and 415 lines.

🔬 **In the data.** For the real matrix ( `code/02_gblup_from_scratch.R` ): mean diagonal ≈ **0.998** (≈1, as designed) and mean off-diagonal ≈ **−0.002** (≈0 — the *average* pair is unrelated, by construction of centering). Individual pairs depart from 0 — and *that variation* is the signal prediction feeds on.

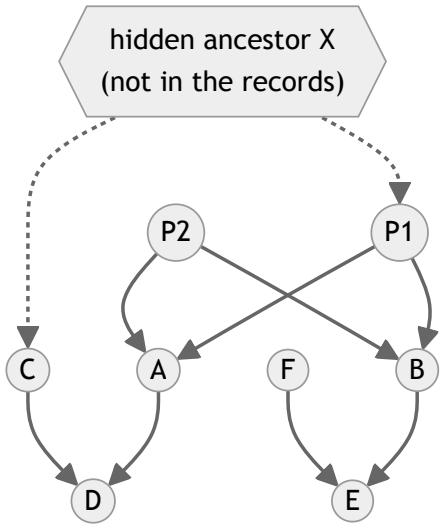
---

## 6.3b 🧸 Pedigree kinship vs. marker reality — and the two glitches markers fix

Before markers, breeders measured relatedness from a **pedigree** (family tree). It's worth seeing this older method *and why DNA improves on it* — it's the whole reason the paper uses a marker **G**. Run `code/toy_06_pedigree_kinship.R`.

### The toy pedigree

Founders **P1, P2, C, F**; full sibs **A, B** (= P1×P2); and **D** (= A×C), **E** (= B×F):




### Pedigree expectation: the relationship matrix A (by hand)

The **numerator relationship matrix A** is built by one rule applied top-down (the *tabular method*): for individual $i$ with parents $s, d$,

$$A_{ij} = \tfrac{1}{2}(A_{js} + A_{jd}), \qquad A_{ii} = 1 + \tfrac{1}{2}A_{sd}$$

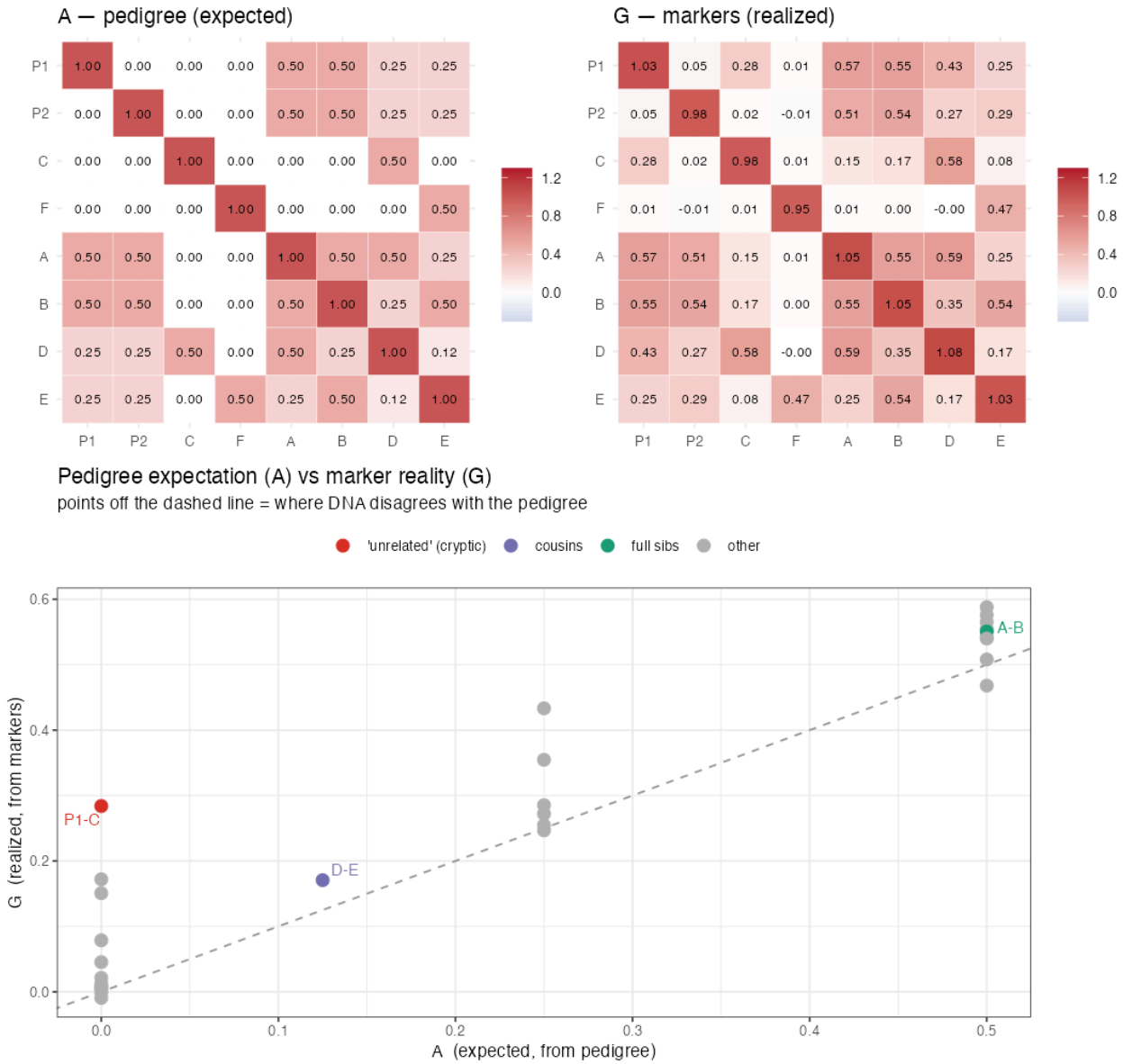
Unknown parents ⇒ treat as unrelated founders ($A = 0$). This gives **expected** relationships:

| pair | meaning | pedigree $A$ | kinship $A/2$ |
|---|---|---|---|
| A, B | full sibs | **0.500** | 0.250 |
| D, E | first cousins | **0.125** | 0.062 |
| P1, C | "unrelated" founders | **0.000** | 0.000 |

⚠️ Notice **A gives one fixed number per relationship type** — *every* full-sib pair is exactly 0.50, by assumption.

### Marker reality: realized G — and where it disagrees

Now we *gene-drop* DNA through the pedigree and compute the marker **G** (VanRaden, §6.2, centered on founder allele frequencies). G measures what each pair **actually** inherited:




🔬 Two disagreements pop out (points off the dashed line):

1. **Mendelian sampling — "every full sib is 0.50" is only an *average*.** The pedigree says A,B = 0.50; the markers say **G = 0.55**. Full sibs each draw a *random half* of each parent's genome, so realized sharing scatters around 0.50 (some sib pairs 0.4, some 0.6). Pedigree A can't see this; **G can**. Selecting on realized relationships is more accurate.
2. **Cryptic relatedness — markers catch what the pedigree missed.** The records call P1 and C *unrelated founders* (A = 0.00). But they secretly share an unrecorded ancestor **X** (they're really half-sibs). The markers expose it: **G = 0.28**. Incomplete pedigrees are the norm in real breeding programs — DNA doesn't rely on paperwork.

## Why this is the bridge to the paper

🌱 The black bean panel comes from **two programs (MSU, USDA-ARS)** with **different, often incomplete pedigrees** — there's no single reliable family tree linking all 415 lines. And even where pedigrees exist, **realized** relatedness (what GP actually needs, Lesson 7) beats pedigree *expectations*. So the authors compute relatedness

straight from **2,315 SNPs** — exactly the marker **G** of §6.2. The toy is the study in miniature: *replace the family tree with DNA, gain both accuracy (Mendelian sampling) and coverage (cryptic relatedness).*

> 🔧 **In practice ®.** Pedigree **A**: `AGHmatrix::Amatrix()`, `nadiv::makeA()`, or `pedigreemm`. Marker **G**: `rrBLUP::A.mat()`, `AGHmatrix::Gmatrix()`, or `sommer::A.mat()` — all implement the VanRaden formula we wrote by hand in §6.2. (We built both from scratch here so the machinery is visible.)

---

## 6.4 Reading the real G: population structure

A heatmap of G (paper's Fig. 1a) shows **blocks** along the diagonal: lines from the same **breeding program × cycle** light up as related clusters, while across-program pairs stay near zero. We reproduced this two ways:

🔬 **Kinship by group** (`figures/05_kinship_dist.png`): relationships are clearly **higher *within* a breeding cycle than *between* cycles** — because each cycle is selected from a limited set of shared parents.

🔬 **PCA of G** (`figures/04_pca_structure.png`): the first two eigenvectors of G separate the lines into groups. (PCA of G = principal components of genetic relatedness; the leading PCs *are* the major axes of population structure.)

🌱 **Why a breeder cares about these blocks — two consequences that drive the whole study:**

1. **Prediction works best within a block.** If your training set is closely related to your target lines (same cycle/program), G has strong positive entries linking them → high accuracy. Predict *across* blocks (cycle 1 → cycle 2) and the linking entries shrink → accuracy drops. **This is the entire story of Lesson 14** (and why the paper's across-cycle accuracies are lower until new-cycle lines are added to training).
2. **Structure confounds GWAS.** If a whole subpopulation happens to both carry an allele *and* have high yield (for unrelated reasons), a naive test flags a false association. That's why GWAS (Lesson 9) must *correct* for structure — using the very PCs we just computed from G.

---

## 6.5 Two roles G plays (don't confuse them)

| Role | Where | What G does |
|---|---|---|
| **Covariance of breeding values** | GBLUP (Lesson 7) | $\mathbf{g} \sim N(\mathbf{0}, \mathbf{G}\sigma_g^2)$ — relatives have correlated genetic values |
| **Diagnostic of structure** | EDA / GWAS (Lesson 9) | its PCs reveal subpopulations to correct for |

The first role is the engine of prediction: by declaring "breeding values are *correlated according to G*", the model can infer an unseen line's value from its relatives' phenotypes.

---

## 6.6 Why G beats counting shared genes one-by-one

You might ask: why not just find the trait genes and check who shares them? Because (a) we mostly don't *know* the genes (Lesson 5's pivot), and (b) **G aggregates tiny signals from all 2,315 SNPs at once.** Complex traits are built from many small effects; G captures their *net* relatedness without needing to identify any single one. This is precisely why genomic prediction outperforms marker-by-marker (QTL) selection for polygenic traits like yield.

---

## 6.7 What you should now be able to say

- **G** is the 415×415 genomic relationship matrix; $G_{ij}$ = average product of standardized genotypes = realized relatedness.
- Build it by **center+scale markers** → $\mathbf{Z}$, then $\mathbf{G} = \mathbf{Z}\mathbf{Z}^\top / p$.
- Its **diagonal ≈ 1**, **mean off-diagonal ≈ 0**; **blocks** reveal program/cycle structure (within-cycle relatedness > between-cycle).
- G plays two roles: **covariance of breeding values** (powers GBLUP) and **structure diagnostic** (powers GWAS correction). The block structure foreshadows *within-cycle accuracy > across-cycle accuracy*.

👉 Next: **Lesson 7 — GBLUP** — we feed G into a mixed model and actually predict (and reproduce accuracy 0.64).

# Lesson 7 — GBLUP: Genomic Prediction's Workhorse

> **The question:** We have clean phenotypes (BLUPs, L3), a marker matrix (L4), a relatedness matrix **G** (L6), and the idea that prediction = exploiting relatedness (L5). **GBLUP** is the model that turns all of that into an actual predicted number for every line — including lines we never phenotyped. This is the core of the entire paper.

---

## 7.1 The model in one line

🧱 **GBLUP model.**

$$\mathbf{y} = \mathbf{1}\mu + \mathbf{g} + \mathbf{e}, \qquad \mathbf{g} \sim N(\mathbf{0},\ \mathbf{G}\sigma_g^2), \qquad \mathbf{e} \sim N(\mathbf{0},\ \mathbf{I}\sigma_e^2)$$

- $\mathbf{y}$ — vector of phenotypes (BLUPs) for the lines we *did* measure.
- $\mu$ — overall mean; $\mathbf{1}$ — a column of ones.
- $\mathbf{g}$ — the breeding values we want (Lesson 5), one per line.
- **The key line: $\mathbf{g} \sim N(\mathbf{0}, \mathbf{G}\sigma_g^2)$** says *breeding values are correlated exactly according to G* (Lesson 6). Relatives have correlated genetic values.
- $\mathbf{e}$ — independent environmental noise.

🧠 **Intuition.** We're telling the model: "Genetic merit is a smooth quantity over the relatedness map G. If line A (unmeasured) sits genetically near lines B, C, D (measured, all high-yielding), then A is probably high-yielding too." GBLUP formalizes "you resemble your relatives" into optimal arithmetic.

---

## 7.2 How it predicts an *unseen* line (the part that feels magic)

Split lines into **training** (phenotyped) and **test** (genotyped only). Because all breeding values are jointly normal with covariance G, the prediction for the test lines is the textbook conditional-mean formula:

🧱

$$\hat{\mathbf{g}}_{\text{test}} = \mathbf{G}_{\text{test,train}}\left(\mathbf{G}_{\text{train,train}} + \lambda\mathbf{I}\right)^{-1}(\mathbf{y}_{\text{train}} - \mu), \qquad \lambda = \frac{\sigma_e^2}{\sigma_g^2}$$

Decode it:

- $\mathbf{G}_{\text{test,train}}$ — **how related each test line is to each training line** (the bridge across the gap).
- $(\mathbf{G}_{\text{train,train}} + \lambda\mathbf{I})^{-1}(\mathbf{y}_{\text{train}} - \mu)$ — turns training phenotypes into weights, *down-weighting* noise via $\lambda$.
- Multiply: each test line's prediction = a **relatedness-weighted blend of training phenotypes.** A test line gets pulled toward the phenotypes of the training lines it most resembles. **That is the whole mechanism.**

⚠️ **Common confusion — "but the test line has no phenotype!"** Correct — and it doesn't need one. Its *genotype* tells us its relatedness ($\mathbf{G}_{\text{test,train}}$) to lines that *do* have phenotypes, and relatedness is enough. This is why a single leaf sample suffices to predict a line you never grew to maturity.

The ratio $\lambda = \sigma_e^2/\sigma_g^2$ is the inverse signal-to-noise — high noise → large $\lambda$ → more shrinkage. (Same quantity that set BLUP shrinkage in L3 and equals $(1 - h^2)/h^2$.)

### 🧸 Toy — predict a hidden line from its relatives (5 lines)

Take 5 lines; we know the phenotype of four and **hide L3**. Using the toy G (entries = relatedness) and the formula above with $\lambda = 0.5$:

| | L1 | L2 | L3 | L4 | L5 |
|---|---|---|---|---|---|
| phenotype $y$ | 10 | 8 | **? (hidden)** | 2 | 9 |
| relatedness to L3, $G_{L3,\cdot}$ | −0.65 | −0.15 | (1.35) | **+0.35** | −0.90 |

L3 is **most related to L4** (+0.35, and L4 is the *low* performer, $y = 2$) and **genetically opposite** the high performers L1 ($y = 10$) and L5 ($y = 9$). So the relatedness-weighted blend pulls L3's prediction *below* the mean ( $\mu = 7.25$):

$$\hat{y}_{L3} = \mu + \mathbf{G}_{L3,trn}(\mathbf{G}_{trn,trn} + \lambda \mathbf{I})^{-1}(\mathbf{y}_{trn} - \mu) = \mathbf{4.27}$$

🔭 **Zoom out:** the real GBLUP does this for each of 143 new-cycle lines at once, blending **272** training phenotypes by a **415×415** relatedness matrix. Same formula — we *just verified it gives 0.64 accuracy on real yield, three different ways* (§7.4).

---

## 7.3 The beautiful duality: GBLUP = ridge regression on markers

Recall Lesson 5's two routes. Here's why they're the same.

**Route A (RR-BLUP):** estimate every marker effect $\alpha_j$, but penalize their size:

$$\hat{\boldsymbol{\alpha}} = \arg\min_{\boldsymbol{\alpha}} \ \|\mathbf{y} - \mathbf{Z}\boldsymbol{\alpha}\|^2 + \lambda\|\boldsymbol{\alpha}\|^2$$

then $\hat{\mathbf{g}} = \mathbf{Z}\hat{\boldsymbol{\alpha}}$. The $\lambda\|\boldsymbol{\alpha}\|^2$ term — **ridge** — is what makes the impossible $p \gg n$ problem solvable: it forbids any single marker from taking a wild effect, spreading signal across all of them.

**Route B (GBLUP):** the conditional-mean formula in §7.2 using $\mathbf{G} = \mathbf{Z}\mathbf{Z}^\top/p$.

🧱 **They are algebraically identical predictions.** Substituting $\mathbf{G} = \mathbf{Z}\mathbf{Z}^\top/p$ into Route B reproduces Route A exactly (a standard linear-algebra identity, the "kernel trick"). So:

> **GBLUP = ridge regression where every SNP is shrunk equally.** "Predict from relatedness (G)" and "shrink all marker effects" are *the same model* viewed from two ends.

🧠 **Why this matters pedagogically.** It dissolves the apparent mystery of G. GBLUP isn't a strange new thing — it's penalized regression you could write in one line, repackaged so you work with a 415×415 matrix instead of 2,315 effects (much faster when $p \gg n$). The assumption hiding inside is **"all markers contribute small, equal-variance effects"** — a great fit for **polygenic** traits like yield, and the reason GBLUP shines on them.

---

## 7.4 We reproduced it three ways — and they agree

To *prove* GBLUP is "just the mixed model above," `code/02_gblup_from_scratch.R` predicts 2018 yield ( `yd18` , n = 272, 70/30 split) three ways:

| Implementation | What it is | Accuracy (cor(pred, truth) on test) |
|---|---|---|
| **(A) From scratch** | our own base-R mixed-model solver (the §7.2 formula, variance components by REML) | **0.640** |
| **(B) `rrBLUP::mixed.solve`** | a standard package, REML | **0.640** |
| **© `BGLR` RKHS(G)** | the **exact** Bayesian tool the authors used | **0.635** |

🔬 They match. The tiny BGLR difference is Monte-Carlo noise (it samples; the others solve in closed form). **Take-home: the "black box" in the paper is reproducible from first principles — you now understand every step that produced 0.64.** And 0.64 lines up with the paper's reported single-trait yield accuracy (~0.60 in 2018), confirming our pipeline matches theirs.

Heart of the from-scratch code (the §7.2 formula, lightly annotated):

```
# variance ratio lambda estimated by REML, then:
Ginv     <- solve(G[trn,trn] + lambda * diag(length(trn)))   # (G_train + λI)^-1
ghat_all <- G[, trn] %*% (Ginv %*% (y[trn] - mu))           # G_test,train  ·  weights
pred     <- mu + ghat_all                                    # predictions for ALL lines
accuracy <- cor(y[tst], pred[tst])                           # score on hidden test lines
```

---

## 7.5 What "prediction accuracy" means here (preview of L13)

We score a model by **the correlation between predicted and observed values on the held-out test lines**. Accuracy 0.64 means predictions and reality line up moderately well — good enough to *rank* lines and select the top ones, even if not perfect point-by-point.

⚠️ **Common confusion — accuracy vs. $R^2$ vs. heritability.** Accuracy here = Pearson $r$ (not $r^2$). It is bounded above by $\sqrt{h^2}$-ish quantities: you can't predict noise, so a trait's heritability caps how high $r$ can go. That's why high-$h^2$ color hits 0.93 and lower-$h^2$ yield sits near 0.6.

---

## 7.6 Where GBLUP sits in the study

GBLUP is the **baseline** every other model is compared against:

- **vs. Gaussian kernels (KA, Lesson 8):** can a *non-linear* relatedness model beat additive G? (Slightly, yes.)
- **vs. GWAS-assisted (Lesson 10):** does adding top SNPs as fixed effects beat plain GBLUP? (No — it hurt.)
- **vs. multi-trait (Lesson 12):** does borrowing a correlated trait beat single-trait GBLUP? (Across cycles, yes — a lot.)

So master GBLUP and the rest of the paper is "GBLUP, plus one twist at a time."

---

> 🔧 **In practice ®.** GBLUP: **`rrBLUP`** (`mixed.solve`, `kin.blup`), **`BGLR`** (`RKHS` with the `G` kernel — what the paper used), `sommer` (`mmer`), `gaston`, or `asreml`. The genomic relationship **G** itself: `rrBLUP::A.mat` / `AGHmatrix::Gmatrix` (Lesson 6). We coded the mixed model from scratch in `02_gblup_from_scratch.R` and confirmed it matches `rrBLUP` and `BGLR`.

## 7.7 What you should now be able to say

- **GBLUP** models $\mathbf{y} = \mathbf{1}\mu + \mathbf{g} + \mathbf{e}$ with $\mathbf{g} \sim N(\mathbf{0}, \mathbf{G}\sigma_g^2)$; it predicts unseen lines as a **relatedness-weighted blend of training phenotypes** via $\hat{\mathbf{g}}_{\text{test}} = \mathbf{G}_{\text{test,train}}(\mathbf{G}_{\text{train,train}} + \lambda\mathbf{I})^{-1}(\mathbf{y} - \mu)$.
- It needs **no phenotype** on the test line — only its **genotype-based relatedness**.
- **GBLUP = ridge regression on markers** (equal shrinkage); the assumption is *many small equal effects*, ideal for polygenic traits.
- We reproduced **accuracy 0.64** for yield three independent ways — the method is fully demystified.

👉 Next: **Lesson 8 — RKHS & Gaussian Kernels** — replacing the *linear* relatedness in G with a flexible *non-linear* one.

# Lesson 8 — RKHS & Gaussian Kernels

> **The question:** GBLUP assumes genetic merit is a **linear, additive** function of markers. But genes interact (dominance, epistasis — Lesson 5). Can we let the model capture **non-linear** patterns of similarity, and does it predict better? This is the **RKHS / Gaussian-kernel** approach, and the **kernel-averaging (KA)** trick the authors settled on.

---

## 8.1 From "relationship matrix" to "kernel" — same shape, richer meaning

In GBLUP we used $\mathbf{G}$ to say how similar lines are. A **kernel** $\mathbf{K}$ is the same *kind* of object — a square line-by-line similarity matrix — but computed by a more flexible rule. Swap $\mathbf{G}$ for $\mathbf{K}$ in the exact same mixed model and you have **RKHS** (Reproducing Kernel Hilbert Space) regression:

$$\mathbf{y} = \mathbf{1}\mu + \mathbf{u} + \mathbf{e}, \qquad \mathbf{u} \sim N(\mathbf{0},\ \mathbf{K}\sigma^2)$$

🧠 **Intuition.** GBLUP's G measures similarity as a *straight-line* (linear) correlation across SNPs. A kernel can measure similarity in a *curved* way — "very similar genotypes count as strongly related, but similarity drops off sharply as genotypes diverge." That curve lets the model fit interactions between loci that a purely additive model can't see.

---

## 8.2 The Gaussian kernel

The authors use the **Gaussian (RBF) kernel**. For lines $i$ and $j$:

$$K_{ij} = \exp\left(-\theta d_{ij}^2\right)$$

- $d_{ij}^2$ — the **squared genetic distance** between lines $i$ and $j$ (squared Euclidean distance between their marker vectors), scaled by its mean.
- $\theta$ — the **bandwidth** (how fast similarity decays with distance).
- $\exp(-\cdot)$ — guarantees $K_{ij} \in (0, 1]$: identical lines → $d = 0$ → $K = 1$; very different lines → large $d$ → $K \approx 0$.

In the repo this is literally:

```
D <- as.matrix(dist(M, method="euclidean"))^2   # squared distances
D <- D / mean(D)                                 # scale
K <- exp(-h * D)                                 # Gaussian kernel, bandwidth h
```

🧠 **Intuition for bandwidth $\theta$ (called `h` in the code).** It sets how quickly "related" fades to "unrelated":

- **Small $\theta$** (e.g. 0.02): similarity decays slowly → almost everyone counts as somewhat related → a *smooth, global, GBLUP-like* model.
- **Large $\theta$** (e.g. 5): similarity decays fast → only near-identical lines count → a *local, wiggly* model that keys on close neighbors and can capture sharp interaction effects.

The right $\theta$ depends on the trait and data — and you don't know it in advance. That problem motivates the next trick.

---

## 8.3 Kernel Averaging (KA) — the authors' chosen method

Instead of betting on one bandwidth, the authors fit **three** Gaussian kernels at once — $\theta \in \{0.02, 1, 5\}$ (a "smooth", a "medium", and a "local" view) — and let the model **weight them automatically**:

🧱 **KA model.**

$$\mathbf{y} = \mathbf{1}\mu + \mathbf{u}_1 + \mathbf{u}_2 + \mathbf{u}_3 + \mathbf{e}, \qquad \mathbf{u}_b \sim N(\mathbf{0},\ \mathbf{K}_b\sigma_b^2)$$

with $\mathbf{K}_b = \exp(-\theta_b \mathbf{D})$. The variance components $\sigma_b^2$ are learned from the data, so the model **decides how much smooth vs. local structure each trait needs** — no manual tuning. This is exactly the repo code:

```
h <- c(.02, 1, 5)
KList <- lapply(h, function(hi) list(K = exp(-hi * D), model = "RKHS"))
fmKA  <- BGLR(y = yNA, ETA = KList, nIter = 12000, burnIn = 2000)   # 3 kernels averaged
```

🧠 **Intuition.** KA is an *ensemble of resolutions*. Rather than guess whether a trait is governed by broad polygenic background (small $\theta$) or sharper local interactions (large $\theta$), you supply all three and the data apportions the weight. It's robust because it can't badly mis-tune a single bandwidth.

🌱 **Breeding logic — why bother going non-linear?** If non-additive effects (gene interactions) matter for a trait, a method that can model them may rank lines more accurately. The cost is a slightly more complex model and that the captured non-additive part is **less transmissible** to offspring than pure additive value — so for *parent selection* additive GBLUP is often preferred, while for *predicting the line's own performance* the kernel may win.

---

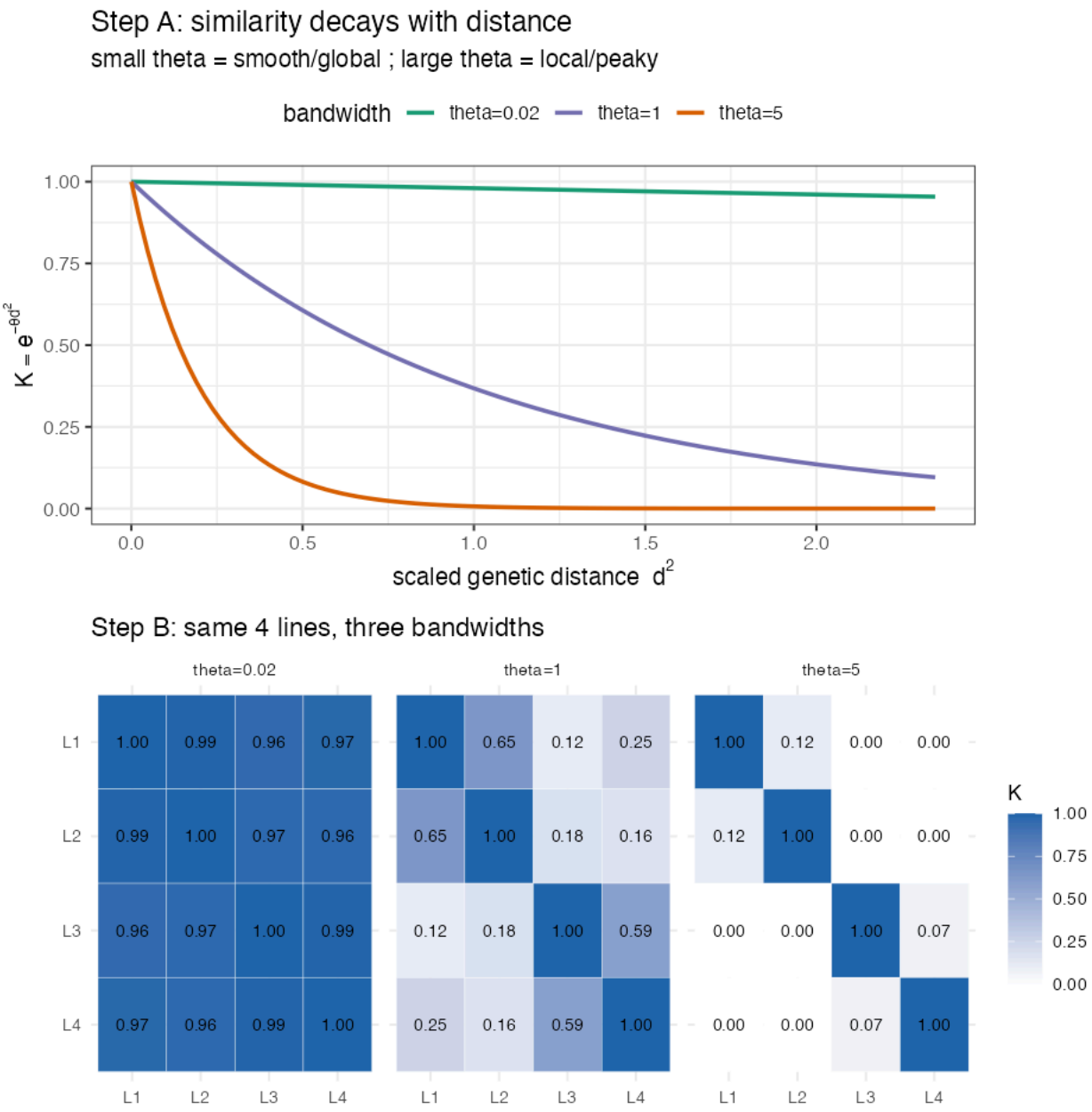
## 8.3b 🧸 Toy first — watch the bandwidth knob turn (`code/toy_08_kernels.R`)

Take **4 lines**, compute their scaled squared genetic distances $d^2$, then turn the bandwidth knob $\theta$ and watch similarity change:

**Step A — the decay curve $K = e^{-\theta d^2}$.** Same distance, three bandwidths:

| $\theta$ | behaviour | a far-apart pair ($d^2 \approx 2$) gets… |
|---|---|---|
| **0.02** | decays *barely* | $K \approx 0.96$ — *almost everyone counts as related* (smooth/global, ≈ GBLUP) |
| **1** | decays moderately | $K \approx 0.13$ |
| **5** | decays *fast* | $K \approx 0.00$ — *only near-identical lines count* (local/peaky) |

**Step B — the same 4×4 kernel matrix at each $\theta$.** At $\theta = 0.02$ every cell is ~1 (one big blurry family); at $\theta = 5$ only the diagonal survives (each line is an island):




🧠 **This is the whole reason for kernel averaging.** No single $\theta$ is "right" — the toy shows they describe *different* relatedness worlds. So **KA** (§8.3) hands the model all three and lets the data weight them. 🔭 **Zoom out:** the real kernels are **272×272** built from 2,315-SNP distances, but each is exactly one of these three "blur levels" — and KA blends them per trait.

---

## 8.4 What the reproduction shows

🔬 **In the data (paper's Fig. 2, and our `code/01` / `02` setup).** Across traits and both years:

- GBLUP and Gaussian-kernel models were **broadly comparable**, but **KA was slightly and *consistently* better** — an average improvement of about **+0.03 in accuracy** over GBLUP.
- Among single bandwidths, none of K1/K2/K3 was uniformly best — *which is the whole point of averaging them*. KA ≥ the best single kernel, reliably.
- Because KA was the consistent (if modest) winner, the authors **carried GBLUP and KA forward** as the two representative GP models for all later comparisons (GWAS-assisted, multi-trait, across-cycle).

⚠️ **Common confusion — "+0.03 is tiny, why care?"** In genomic selection, small, *consistent* accuracy gains compound over many cycles of selection into real genetic gain (recall $\Delta G = ir\sigma_A$ — every bit of $r$ counts). A method that's *reliably* a hair better, with no extra data cost, is worth adopting.

---

## 8.5 RKHS in the family tree of models

| Model | Similarity rule | Captures | This study |
|---|---|---|---|
| **GBLUP** | linear: $\mathbf{G} = \mathbf{Z}\mathbf{Z}^\top/p$ | additive effects | baseline |
| **RKHS (1 kernel)** | Gaussian $\exp(-\theta d^2)$ | additive + some non-additive | depends on $\theta$ |
| **KA (3 kernels)** | mixture of bandwidths | adapts smooth↔local automatically | **chosen GP model** |

All three plug into the **same** mixed-model machinery of Lesson 7 — only the similarity matrix changes. That's the unifying insight: **GBLUP and RKHS are one framework; the kernel is the knob.**

---

> 🔧 **In practice ®.** RKHS / Gaussian kernels: `BGLR` — pass a *list* of `RKHS` terms with kernels at several bandwidths and it fits **kernel averaging (KA)** automatically (exactly the paper's setup). `sommer` offers a Gaussian kernel ( `GK` ); `GAPIT` exposes kernel options too. The squared-distance matrix is just `as.matrix(dist(M))^2` scaled by its mean.

## 8.6 What you should now be able to say

- An **RKHS / kernel** model is GBLUP with the relationship matrix **G** replaced by a flexible **kernel K**; the **Gaussian kernel** $K_{ij} = \exp(-\theta d_{ij}^2)$ turns genetic distance into similarity, with **bandwidth** $\theta$ controlling smooth (global) vs. local fitting.
- **Kernel averaging (KA)** fits several bandwidths (0.02, 1, 5) and lets the data weight them, avoiding manual tuning.
- Kernels can capture **non-additive** structure; here **KA was consistently ~0.03 better than GBLUP**, so GBLUP + KA were the two GP models taken forward.

👉 Next: **Lesson 9 — GWAS with FarmCPU** — switching jobs from *predicting lines* to *finding the loci*.

# Lesson 9 — GWAS with FarmCPU

> **The question:** Up to now we've *predicted lines* without caring which genes matter. GWAS asks the **opposite** question: *which specific SNPs are statistically associated with the trait?* This is a **different job** from prediction (Lesson 5's pivot). Here's how it works, why population structure makes it tricky, and what we found in the real data.

---

## 9.1 The core idea: one SNP at a time

A genome-wide association study (GWAS) **tests every SNP, one at a time**, for a statistical association with the trait. For SNP $j$:

🧱 **The per-SNP model.**

$$\mathbf{y} = \mu + \underbrace{x_j \beta_j}_{\text{this SNP's effect}} + \underbrace{\mathbf{Q}\boldsymbol{\gamma}}_{\text{structure correction}} + \mathbf{e}$$

- $x_j$ — genotypes (0/1/2) at SNP $j$.
- $\beta_j$ — its effect; **we test $H_0 : \beta_j = 0$** and get a **p-value**.
- $\mathbf{Q}$ — covariates correcting for population structure (see §9.3).

Do this 2,315 times → 2,315 p-values → a **Manhattan plot**.

🧠 **Intuition.** A SNP "associated" with high yield is one where lines carrying allele *A* tend to yield more than lines carrying allele *G*. The p-value asks: *could this difference be chance?* Small p (tall point on the Manhattan plot) = unlikely chance = candidate locus.

⚠️ **Common confusion — association ≠ causation.** A significant SNP is usually **not** the causal gene; it's a **signpost** in **linkage disequilibrium** (Lesson 4) with the real culprit nearby. GWAS narrows you to a *region*, not a gene.

---

## 9.2 The multiple-testing problem and Bonferroni

Test 2,315 SNPs at the usual $\alpha = 0.05$ and, *even if nothing is real*, you'd expect $\approx 2315 \times 0.05 \approx 116$ "significant" hits by pure chance. Useless. So we make the threshold much stricter.

🧱 **Bonferroni correction.**

$$\alpha_{\text{adj}} = \frac{0.05}{\text{number of SNPs}} = \frac{0.05}{2315} = 2.16 \times 10^{-5}$$

A SNP must beat *this* p-value to count. On the Manhattan plot it's the dashed line at $-\log_{10}(2.16 \times 10^{-5}) \approx 4.67$.

> 🔬 **In the data.** Our reproduction ( `code/03_gwas.R` ) computes this threshold as exactly $2.16 \times 10^{-5}$ — matching the paper's stated value to the digit. (Good sign our setup matches theirs.) Bonferroni is *conservative*

> (it controls the chance of *any* false positive), which the authors note can cause it to miss real small-effect loci — relevant in Lesson 10.

---

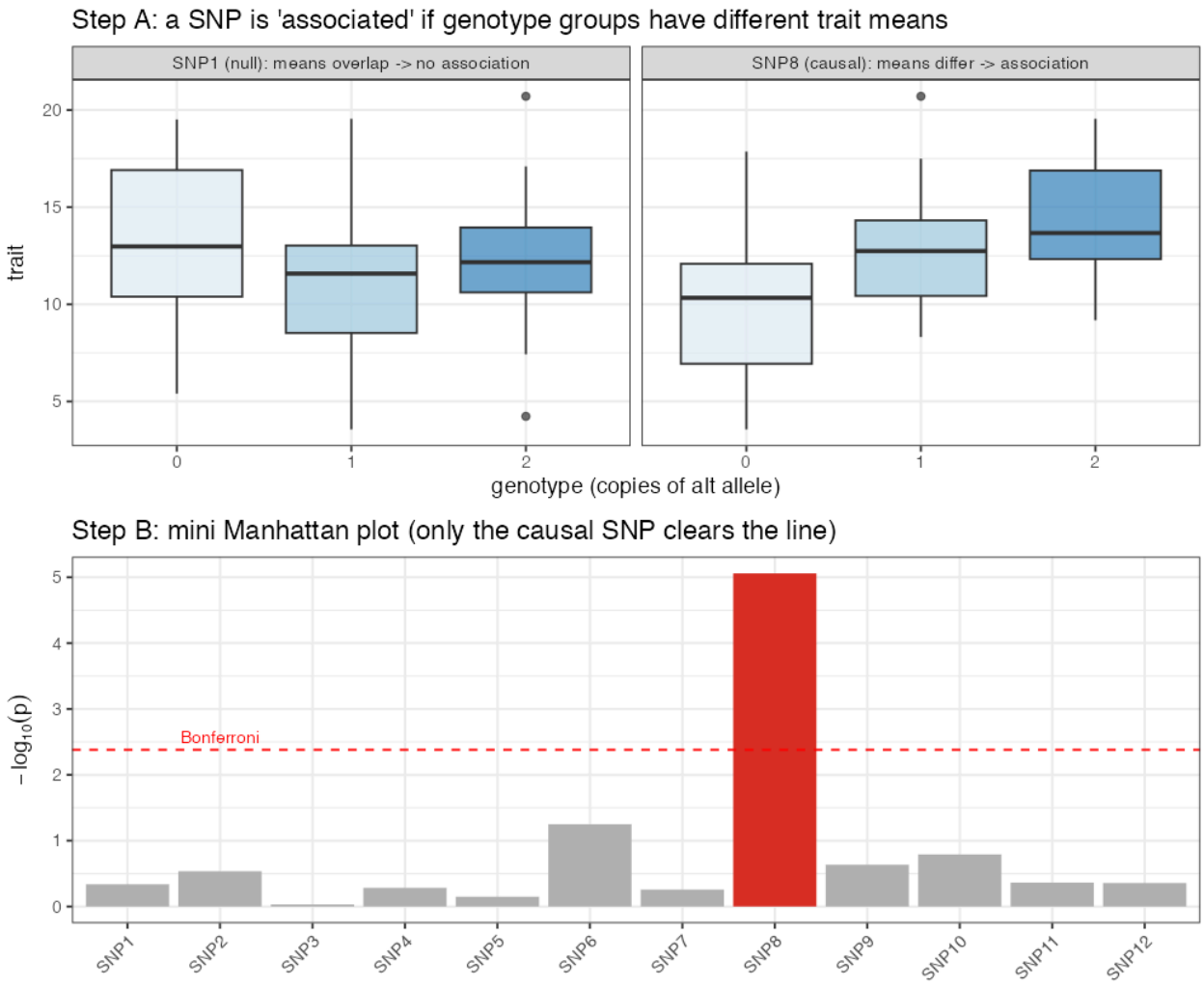
## 9.2b 🧸 Toy first — build a Manhattan plot from scratch ( `code/toy_09_gwas.R` )

Simulate **80 lines** and **12 SNPs**, where *only* **SNP8** truly affects the trait ($y = 10 + 3 \cdot x_{\text{SNP8}} + \text{noise}$). Then test each SNP, exactly as GWAS does.

**Step A — what "association" means.** Split the lines by their genotype (0/1/2) at a SNP and look at the trait:

- **SNP8 (causal):** the 0, 1, 2 groups have clearly *different* means (more alt copies → higher trait) → small p-value.
- **SNP1 (null):** the groups *overlap* → large p-value.

**Step B — the mini Manhattan.** Plot $-\log_{10}(p)$ for all 12 SNPs with the Bonferroni line at $0.05/12$:




🔬 Real toy numbers: SNP8 has $p = 8.7 \times 10^{-6}$ → **clears** the Bonferroni line; the best null SNP only reaches $p = 0.056$ → stays under it. *Exactly one bar pokes through* — and it's the real one.

🔭 **Zoom out:** the real study tests **2,315** SNPs (so the Bonferroni line moves to $0.05/2315 = 2.16 \times 10^{-5}$), adds **3 PCs** to defuse population structure (§9.3), and uses the smarter iterative FarmCPU — but every bar on a real Manhattan plot is one of these little per-SNP tests. (See the *real* Manhattan in §9.5: high-$h^2$ color has many bars over the line, low-$h^2$ yield almost none.)

## 9.3 Population structure — the silent troublemaker

Recall (Lesson 6) the lines cluster by program/cycle. Suppose the MSU subpopulation happens to both (a) carry allele *A* at some SNP *and* (b) yield higher — **for unrelated reasons** (maybe MSU just bred higher-yielding backgrounds). A naive test sees "allele *A* ↔ high yield" and screams *association* — a **false positive driven by structure**, not biology.

🧠 **Intuition.** It's the classic confounder: ice-cream sales correlate with drownings (both caused by summer). Population structure is the "summer" confounding SNPs with traits.

**Fix:** include the **first 3 principal components** of the marker matrix as covariates (**Q** above). Those PCs *are* the major axes of structure (Lesson 6's PCA of G), so conditioning on them removes the subpopulation confounding before testing each SNP. The authors use **3 PCs**; our reproduction does the same.

## 9.4 FarmCPU — the specific method used

The authors run GWAS with **FarmCPU** (Fixed and random model Circulating Probability Unification), in the R package **GAPIT3**. You don't need its internals, but the idea is worth a sentence:

🧠 **Intuition.** Simple single-marker tests have a dilemma: correct for *too little* structure → false positives; correct with a full kinship random effect → you can *over*-correct and bury real signals. **FarmCPU breaks the deadlock** by **iterating**: it alternates between (1) a *fixed-effect* model that tests SNPs while controlling for a handful of already-found associated markers (called pseudo-QTNs), and (2) a *random-effect* model that re-selects which markers to control for. This "circulating" between the two models gives more **power** with controlled false positives than a one-shot test.

⚠️ Our `code/03_gwas.R` uses a **simpler single-marker, PC-corrected scan** (so you can read every line and we avoid the heavy GAPIT install). It captures the *concept*; FarmCPU is more powerful and will find a somewhat different set. Both share the same threshold logic.

## 9.5 Heritability decides how much you discover

🔬 **In the data** — we ran the scan on two traits to make a point:

| Trait | Heritability | SNPs passing Bonferroni (our scan) |
|---|---|---|
| **Yield 2018** | lower | **4** |
| **Color 2018** | higher | **105** |

> Figure: `figures/06_manhattan.png` — yield's Manhattan is nearly flat (few, low peaks); color's has tall, clear peaks.

🧠 **Why the huge difference?** Two reasons, both already in your toolkit:

1. **Heritability** (Lesson 5): color is highly heritable → strong genetic signal → easy to detect. Yield is noisier → weak per-SNP signals.
2. **Genetic architecture**: color is more **oligogenic** (a few larger-effect loci → tall peaks), while yield is **polygenic** (thousands of tiny effects, none individually beating Bonferroni — the classic "missing heritability" picture).

🌱 **This foreshadows Lesson 10's surprise.** Yield is governed by *many tiny* effects, so the handful of SNPs GWAS can detect explain very little of it. Forcing those few SNPs into a prediction model as "important" therefore *can't help much* — and, as we'll see, actually *hurts*. The Manhattan plot's flatness for yield is a visual preview of why GWAS-assisted GP underperformed for the very trait breeders care most about.

---

## 9.6 What the authors actually found

Across their 100 training subsets, FarmCPU flagged **555 SNP–trait associations** in total — but only **52 showed up in ≥20 subsets**, and **9 in >50**, and **none in all 100**. In other words:

🔬 **GWAS hits were *unstable*** — change which lines you analyze and you get different "top" SNPs. They also saw a positive correlation between a marker's mean effect and its standard deviation across subsets (big effects were also the most variable). This instability is the central evidence for Lesson 10: if you can't even agree on *which* SNPs matter, baking them into a predictor as fixed truths is risky.

---

## 9.7 GWAS vs. GP — keep the two jobs straight

| | GWAS (Lesson 9) | Genomic Prediction (Lessons 7–8) |
|---|---|---|
| Question | *Which* loci are associated? | *How good* is each line? |
| Output | p-value per SNP, Manhattan plot | one predicted breeding value per line |
| Uses all markers? | tests one at a time (+ structure) | uses all jointly via G/kernel |
| Best for | understanding architecture, finding candidate genes | selecting/ranking lines |

The study's clever move (Objective 3) is to ask: **can the *output* of GWAS feed the *input* of GP?** That's Lesson 10.

---

> 🔧 **In practice ®.** GWAS: `GAPIT3` (the paper — `FarmCPU`, `BLINK`, `MLM`, `MLMM`), `rMVP` (fast, large data), or `statgenGWAS`; structure PCs come from `prcomp` / `SNPRelate`. Our `03_gwas.R` codes the PC-corrected single-marker scan by hand so every step is visible.

## 9.8 What you should now be able to say

- GWAS **tests each SNP** for trait association, producing p-values and a **Manhattan plot**; significance uses a **Bonferroni** threshold ($0.05/2315 = 2.16 \times 10^{-5}$).
- **Population structure** confounds associations; correct with the **first 3 PCs** of the markers. **FarmCPU** iterates fixed/random models for more power.
- **High-$h^2$, oligogenic** traits (color: 105 hits) are easy to map; **low-$h^2$, polygenic** traits (yield: 4 hits) are not — and GWAS hits here were **unstable** across subsets.
- GWAS (find loci) and GP (rank lines) are **different jobs**.

👉 Next: **Lesson 10 — GWAS-assisted Genomic Prediction** — feeding GWAS hits into the predictor, and why it backfired.

# Lesson 10 — GWAS-assisted Genomic Prediction

> **The question (Objective 3):** GBLUP treats every SNP equally (Lesson 7). But GWAS just told us *some* SNPs are specially associated with the trait (Lesson 9). **What if we tell the prediction model "these specific SNPs are important"?** Intuitively that should help. The surprising, instructive answer: **it didn't — it usually hurt.** Understanding *why* is one of the most valuable lessons in the whole study.

---

## 10.1 The idea: promote the GWAS hits to "fixed effects"

In plain GBLUP, all markers are pooled into **G** and shrunk equally (Lesson 7) — they're **random** effects. GWAS-assisted GP pulls the significant SNPs *out* of that pool and adds them back as **fixed effects** that are **not shrunk**:

🧱 **The augmented model.**

$$\mathbf{y} = \mathbf{1}\mu + \underbrace{\mathbf{W}\boldsymbol{\beta}}_{\text{GWAS hits, FIXED (no shrinkage)}} + \underbrace{\mathbf{g}}_{\text{rest of genome via G, random}} + \mathbf{e}$$

- $\mathbf{W}$ — genotypes at just the GWAS-significant SNPs.
- $\boldsymbol{\beta}$ — their effects, estimated **freely** (fixed = "trust these fully").
- $\mathbf{g} \sim N(\mathbf{0}, \mathbf{G}^*\sigma_g^2)$ — the *remaining* markers (the hits are **removed from G** so they're not double-counted) still shrunk as usual.

This is exactly the repo code:

```
indexQTL <- which(colnames(M) %in% GWAS_res$SNP)   # the significant SNPs
QTL   <- M[, indexQTL]                              # pulled out as fixed effects
M_new <- M[, -indexQTL]                             # removed from the background
G_qtl <- tcrossprod(scale(M_new)) / ncol(M_new)     # G WITHOUT the hits
ETA <- list(list(X = QTL,   model = "FIXED"),       # hits: fixed, unshrunk
            list(K = G_qtl, model = "RKHS"))        # rest: random, shrunk
fm_gwas <- BGLR(y = yNA, ETA = ETA, ...)
```

🧠 **Intuition for "fixed = unshrunk."** A **random** effect is humble: "this marker probably has a small effect; shrink it toward 0 unless the data insist otherwise." A **fixed** effect is confident: "this marker matters; estimate its effect at face value, no shrinkage." Promoting a SNP to fixed says *"I'm sure about you."*

---

## 10.2 Why it *should* help — and why it didn't

**The hope:** if a few loci truly have big effects, estimating them precisely (unshrunk) and letting G mop up the rest should beat shrinking everything uniformly.

🔬 **What actually happened (paper's Fig. 3 & 4, all traits & years):** adding GWAS SNPs as fixed effects **reduced** prediction accuracy, across the board. The negative effect was strong enough that even when combined with helpful multi-trait information, it dragged accuracy down.

**Why? Three reasons, each building on earlier lessons:**

1. **The hits were unstable (Lesson 9).** Only 52/555 associations recurred in ≥20 of 100 subsets; *none* in all 100. So "the important SNPs" change depending on who's in the training set. You're hard-coding **confidence in things that aren't reproducible** → you fit noise.

2. **Yield/appearance are polygenic (Lesson 9).** Their genetic signal is spread over thousands of tiny effects. The few detectable SNPs explain a sliver of the variance. Trusting that sliver *fully* (fixed) while the true signal lives in the shrunken background is backwards — you've promoted the least representative part of the architecture.

3. **Selection bias / "winner's curse."** SNPs are chosen *because* they had extreme associations **in the training data**. Extreme-in-training effects are partly luck and **regress toward the mean** in new data — so their fixed (unshrunk) estimates are *over-* confident and predict the test set worse. Shrinkage exists precisely to defend against this; making them fixed *removes the defense*.

🧠 **The deep lesson.** GBLUP's equal shrinkage isn't a crude simplification to be "improved" with GWAS — for polygenic traits it is **close to optimal**, because it honestly reflects that *we don't know which markers matter and most matter a little*. GWAS-assisted GP injects false certainty, and the model pays for it.

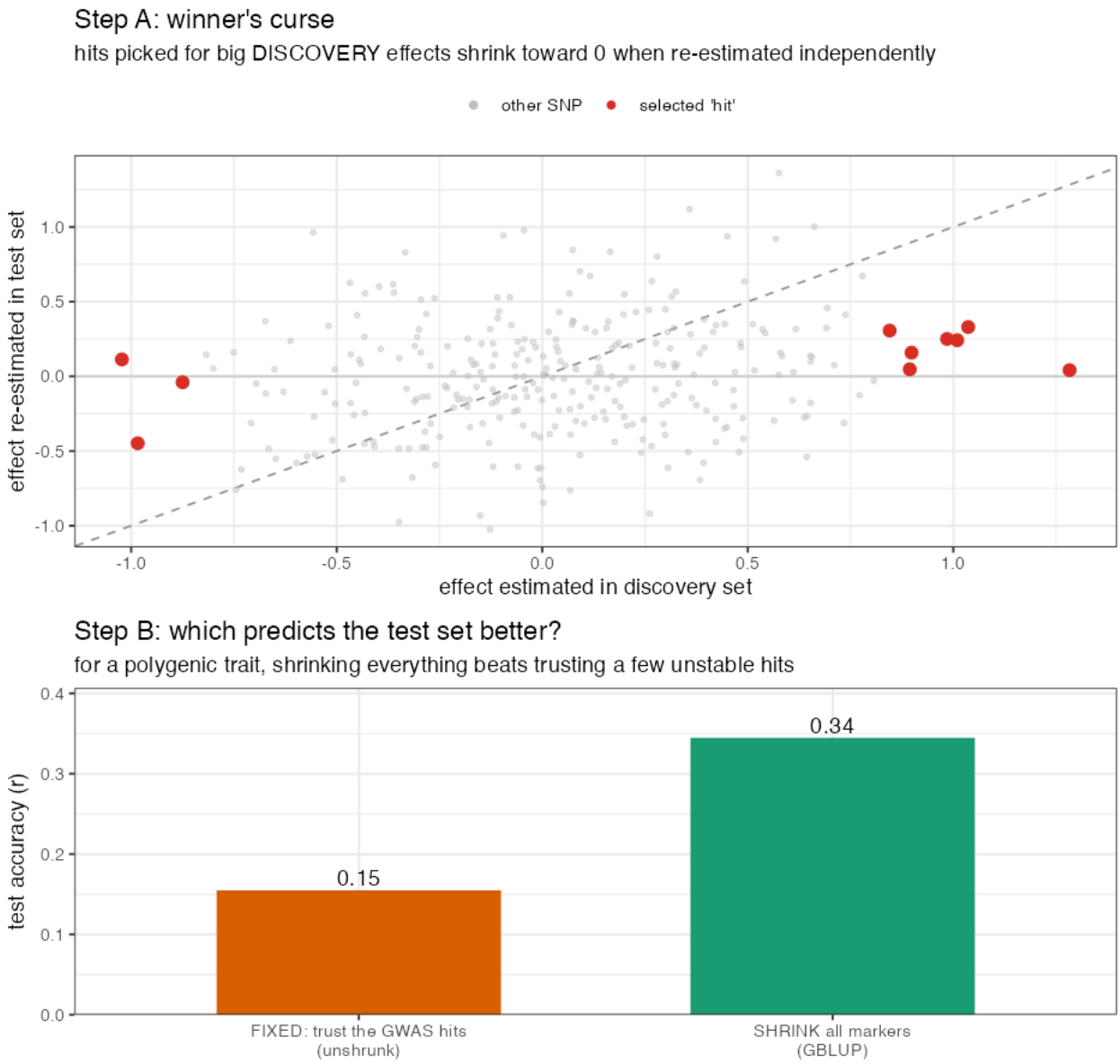
⚠️ **Common confusion — "so GWAS is useless for breeding?"** No. GWAS is great for *understanding genetic architecture* and finding **major-gene** candidates (e.g., a disease-resistance locus with a big, stable effect). For such **oligogenic** traits, marker-assisted or GWAS-assisted approaches *can* help. The failure here is specific to **polygenic** traits with **unstable** hits — exactly yield and canning appearance. Match the tool to the architecture.

---

## 10.2b 🧸 Toy first — *see* the winner's curse ( `code/toy_10_winners_curse.R` )

The abstract reasons above become obvious on a toy. Simulate a **polygenic** trait: 300 SNPs, each with a *tiny* true effect, in 200 lines split into a **discovery** set and a **test** set. Run a GWAS-style scan on discovery, pick the **top-10 SNPs**, and inspect them.

**Step A — winner's curse.** Plot each SNP's effect *as estimated in discovery* (x) against its effect *re-estimated independently in the test set* (y). The selected "hits" (red) sit far out on the x-axis but collapse toward **0** on the y-axis — far below the dashed $y = x$ line:




🔬 The numbers are stark: the top-10 hits average $|\hat{\alpha}| = 0.98$ in discovery, but only **0.20** when re-estimated honestly — and their *true* average effect is just **0.16**. They looked big **because they got lucky in the discovery sample** (extreme noise + small real effect). A **fixed** effect trusts that 0.98 at face value; reality is 0.16.

**Step B — so prediction suffers.** Predict the test set two ways:

- **FIXED:** trust the 10 hits, estimate them unshrunk → test accuracy **0.15**.
- **SHRINK everything (GBLUP):** pull all 300 markers toward 0 → test accuracy **0.34**.

🧠 **That 0.15 vs 0.34 is the paper's "+GWAS hurts" result in miniature.** Shrinkage is a *defense* against the winner's curse; promoting hits to fixed effects removes the defense and lets the overconfident estimates mislead the prediction. 🔭 **Zoom out:** the real models did exactly this with FarmCPU hits that were *also* unstable across the 100 training subsets (§9.6) — so the real penalty was, if anything, larger.

---

## 10.3 The result table in words

🔬 Summarizing the paper's GWAS-assisted comparisons:

- **GBLUP + GWAS < GBLUP** (plain) — fixed hits hurt.

- **KA + GWAS < KA** — same story with kernels.
- Even **MT + GWAS** (multi-trait *plus* GWAS hits) underperformed the MT models without GWAS — the GWAS penalty offset the multi-trait benefit.

So in every comparison, "+GWAS" was a **down**-arrow. The authors' conclusion: a deeper, function-aware analysis of loci would be needed before injecting them into GP for these traits.

---

## 10.4 Where this sits in the mental map

This closes Objective 3 with a clear, generalizable principle:

> **More information is not automatically better.** Adding *confident but wrong/unstable* information degrades a model. The art is knowing *which* information to trust *how much* — which is exactly what shrinkage (random effects) does automatically, and what fixing effects overrides.

This same theme returns in Lesson 11 (NIRS also didn't help) and contrasts with Lesson 12 (correlated traits *did* help, because that information was stable and relevant).

---

## 10.5 What you should now be able to say

- **GWAS-assisted GP** promotes significant SNPs to **fixed (unshrunk) effects**, keeping the rest in **G**.
- Here it **lowered accuracy for every trait/year** because the hits were **unstable**, the traits are **polygenic**, and unshrunk effects suffer **winner's-curse** over-confidence.
- **Equal shrinkage (GBLUP)** is near-optimal for polygenic traits *because* it admits we don't know which markers matter — fixing GWAS effects removes that protection.
- GWAS-assisted prediction can still help for **oligogenic** traits with **stable, large-effect** loci; match the method to the architecture.

👉 Next: **Lesson 11 — NIRS & Regularized Selection Indices** — a second "more data" idea (cheap light scans), and whether it paid off.

# Lesson 11 — NIRS & Regularized Selection Indices

> **The question (Objective 2):** Canning quality is expensive to measure (Lesson 1). A **NIRS scan** is cheap, fast, and non-destructive (Lesson 2). Can we squeeze the 551-wavelength spectrum into a single useful number — a **selection index** — that acts as a cheap *secondary trait* to help predict the expensive one? Here's the method (penalized regression) and the verdict.

---

## 11.1 The raw material: a spectrum per line

Each line has a **near-infrared spectrum**: absorbance at 551 wavelengths (1100–2200 nm). It's an indirect chemical fingerprint — water, starch, protein, and seed-coat compounds each absorb differently. Before modeling, the spectra are **preprocessed** with **Standard Normal Variate (SNV)** normalization (the `prospectr` package), which removes scatter/baseline differences between samples so curves are comparable.

⚠️ The challenge: **551 predictors, ~272 lines.** Just like the marker matrix, this is $p > n$ — you cannot fit an ordinary regression of trait on all 551 wavelengths. And many adjacent wavelengths are nearly identical (highly correlated). We need **variable selection + shrinkage**.

---

## 11.2 The goal: a Regularized Selection Index (RSI)

We want weights $\boldsymbol{\beta}$ (one per wavelength) so that the weighted sum of a line's spectrum predicts its trait:

🧱 **Selection index.**

$$\text{RSI}_i = \sum_{k=1}^{551} \beta_k x_{ik} = \mathbf{x}_i^\top \boldsymbol{\beta}$$

— collapse 551 numbers into **one** index per line, tuned to correlate with the target trait (e.g., canning appearance).

🧠 **Intuition.** A selection index is a *recipe*: "take 0.3 × (this wavelength) − 0.1 × (that one) + …" to brew a single score that tracks the trait. The art is choosing the recipe weights without overfitting 551 ingredients to only ~272 samples.

---

## 11.3 Regularization — taming $p > n$ with a penalty

The authors estimate the weights with a **penalized (ridge-type) regression** following Lopez-Cruz et al. (2020):

🧱

$$\hat{\boldsymbol{\beta}} = \left(\mathbf{P}_x + \lambda \mathbf{I}\right)^{-1} \mathbf{G}_{x,y}$$

- $\mathbf{P}_x$ — the variance–covariance matrix of the wavelengths (how spectra co-vary).
- $\mathbf{G}_{x,y}$ — the genetic covariances between each wavelength and the target trait (estimated via `getGenCov` — the *genetic* part of the relationship, not just phenotypic).

- $\lambda$ — the **penalty** (the same idea as ridge in Lesson 7 / GBLUP): without it, $\mathbf{P}_x$ is not invertible ($p > n$) and the fit overfits; adding $\lambda\mathbf{I}$ **stabilizes the inverse and shrinks the weights**, performing implicit variable selection.

🧠 **Intuition for $\lambda$.** $\lambda = 0$: trust all 551 wavelengths fully → memorize noise → great on training, useless on new lines. Large $\lambda$: squash all weights → ignore the spectrum. The sweet spot in between keeps only the *reliably useful* wavelengths.

**How $\lambda$ is chosen — cross-validation (preview of Lesson 13).** The authors do **10-fold cross-validation** over a grid of 100 $\lambda$ values, picking the $\lambda$ that maximizes the correlation between the index and the trait *on held-out folds*. This is the repo's inner loop:

```
for (k in 1:cv) {                       # 10-fold CV inside the training set
  fm  <- getGenCov(y[trt], X[trt,], K = G1[trt,trt])   # genetic covariances
  fm2 <- solveEN(var(X[trt,]), fm$covU, X = X[trt,])    # penalized solve over a λ-grid
  # ... record which λ maximized accuracy on the held-out fold ...
}
lambda0 <- mean(lambda)                  # average best λ across folds
```

(The package `SFSI` provides `solveEN` / `getGenCov`; we describe it rather than run it, since `SFSI` needs compilation. The *principle* — penalized regression chosen by CV — you already own from Lesson 7.)

🌱 **Breeding logic.** If this works, a breeder scans seeds in seconds and gets an index that ranks lines for canning quality — no taste panel, no cooking. Even a *moderately* predictive index could pre-screen thousands of lines down to a few worth panel-testing.

---

## 11.3b 🧸 Toy first — *why* you must penalize (`code/toy_11_rsi.R`)

Forget 551 wavelengths for a moment. Take **14 toy "wavelengths"** but only **12 training individuals** (so $p > n$, like the real NIRS problem), where truly only 2 wavelengths matter. Build the index $\mathbf{RSI} = \sum_k \beta_k x_k$ two ways and test on **held-out** individuals:

**Step A — the weights $\beta_k$.** Plain least-squares (no penalty) vs. a ridge penalty vs. the truth:

- **OLS weights explode** — huge positive/negative values on *noise* wavelengths (it has too much freedom and chases random patterns).
- **Ridge weights are shrunk** toward 0, roughly recovering the 2 real signals.

**Step B — does the index predict held-out individuals?**

| index | accuracy on *training* | accuracy on *held-out test* |
|---|---|---|
| **OLS (no penalty)** | **1.00** (perfect!) | **0.14** (useless) |
| **Ridge (penalized)** | 0.94 | **0.38** |


🧠 **The lesson in one line:** OLS *memorized* the training data (r = 1.00) but learned nothing that transfers (r = 0.14). The **penalty λ** trades a little training fit for real generalization — which is the entire point of the **R** in **R**SI. This is the same shrinkage idea as GBLUP (Lesson 7), now applied to spectra.

🔭 **Zoom out:** the real RSI penalizes **551** wavelengths estimated from ~190 lines, with **λ** chosen by the 10-fold cross-validation below. Same machinery; more columns.

---

## 11.4 The verdict: it didn't help much here

🔬 **In the data (paper's Fig. 3, Supp. Table 3).** The NIRS-based RSI had **low prediction accuracy** for the target traits, and including it as a secondary trait in multi-trait models (Lesson 12) **did not improve** — sometimes slightly worsened — accuracy *within* a cycle.

**Why?** The honest reasons:

- **Intact dry seeds.** They scanned *whole* dry beans; the canning-quality signal may live in the cooked/internal state that an external dry-seed scan barely sees. (Other studies got better results with ground samples or different preprocessing.)
- **Preprocessing choice.** Only SNV was used; the authors note derivatives or wavelet methods might extract more signal.
- **Weak trait correlation.** The index just didn't correlate strongly enough with appearance/ texture to add information beyond what the genome already provided.

⚠️ This echoes Lesson 10's theme: **a cheap extra data source only helps if it carries *relevant, non-redundant* signal.** NIRS here was cheap but weak; GWAS hits were confident but unstable. Neither moved the needle.

---

## 11.5 But the *idea* of a secondary trait is the bridge to multi-trait models

Even though NIRS underwhelmed, it introduces the concept that powers the study's biggest win: a **secondary trait** that's easier/cheaper to measure can lend predictive strength to a hard target trait — *if* the two are genetically correlated. NIRS was one candidate secondary trait. The ones that actually worked were **other measured traits**: seed weight (for yield) and texture (for appearance). That's Lesson 12.

🧠 **Connecting thread.** RSI (this lesson) and multi-trait models (next) are the same strategy — *borrow information from a correlated, cheaper variable* — applied to two different sources (spectra vs. correlated traits). Holding that link makes Lesson 12 click instantly.

---

## 11.6 What you should now be able to say

- A **NIRS spectrum** (551 wavelengths) is preprocessed (**SNV**) and collapsed into a single **Regularized Selection Index** $\mathrm{RSI} = \mathbf{x}^\top \boldsymbol{\beta}$.
- Because $p > n$ and wavelengths are correlated, weights are found by **penalized regression** $(\mathbf{P}_x + \lambda \mathbf{I})^{-1}\mathbf{G}_{x,y}$, with $\lambda$ **chosen by 10-fold CV** — same shrinkage logic as GBLUP.
- Here NIRS-RSI had **low accuracy** and **didn't improve** GP (intact-seed scanning, limited preprocessing, weak correlation) — reinforcing that extra data must be *relevant* to help.
- RSI is the conceptual **bridge to multi-trait models**: borrow strength from a correlated, cheaper variable.

👉 Next: **Lesson 12 — Multi-Trait Models** — the strategy that *did* pay off.

# Lesson 12 — Multi-Trait Models

> **The question (Objective 1):** So far each trait is predicted *alone* (single-trait, ST). But traits are **correlated** — yield with seed weight, appearance with texture (Lesson 1). What if we predict several traits **jointly**, letting a well-measured correlated trait lend its strength to a hard one? This is the **multi-trait (MT)** model — and it delivered the study's headline gains.

---

## 12.1 The idea: borrow strength from a correlated trait

Single-trait GBLUP predicts trait 1 using only trait 1's phenotypes + G. A **multi-trait** model predicts trait 1 *and* a **secondary** trait 2 together, exploiting their **genetic correlation**: if a line has a high secondary value, and the two traits are genetically linked, that nudges its predicted primary value.

🧠 **Intuition.** You want to guess a student's *essay* grade (hard to grade, few samples). You also have their *reading-test* score (easy, plentiful), and the two correlate. A smart predictor uses the reading score to sharpen the essay estimate — *especially when essay data is thin*. MT models do exactly this for traits.

🌱 **Breeding logic — pick the *right* secondary trait.** It must be (a) **genetically correlated** with the target and (b) **cheaper/easier/more heritable** to measure. The authors chose:

- For **yield (YD) → seed weight (SW)** as secondary. (Recall yield ↔ SW $r = +0.41$; SW is fast and highly heritable.)
- For **appearance (App) → texture (Text)** as secondary. (Text is machine-measured, objective, high $h^2$.)

Both secondaries are exactly the kind of cheap, heritable, correlated traits the strategy needs.

---

## 12.2 The model

🧱 **Multi-trait GBLUP.** Stack the two traits' phenotypes and model their breeding values as **correlated**:

$$\begin{pmatrix}\mathbf{y}_1\\ \mathbf{y}_2\end{pmatrix} = \boldsymbol{\mu} + \begin{pmatrix}\mathbf{g}_1\\ \mathbf{g}_2\end{pmatrix} + \mathbf{e}, \qquad \begin{pmatrix}\mathbf{g}_1\\ \mathbf{g}_2\end{pmatrix} \sim N(\mathbf{0},\ \boldsymbol{\Sigma}_g \otimes \mathbf{G})$$

(the $\begin{pmatrix}\cdot\\ \cdot\end{pmatrix}$ here just means the two traits **stacked** into one tall vector.) The new ingredient is the **genetic covariance matrix between traits**, $\boldsymbol{\Sigma}_g$:

| $\boldsymbol{\Sigma}_g$ | **trait 1** | **trait 2** |
|---|---|---|
| **trait 1** | var($g_1$) | cov($g_1$,$g_2$) |
| **trait 2** | cov($g_1$,$g_2$) | var($g_2$) |

- $\sigma_{g_{12}}$ — the **genetic covariance** between trait 1 and trait 2. **This is the channel through which trait 2 helps trait 1.** If it's 0, MT collapses back to ST (no help).
- $\otimes\mathbf{G}$ — relatedness still governs *within* each trait (Lesson 6).

In the repo, this is `BGLR::Multitrait`, with the secondary trait carried as a second column (here the NIRS index `SSI`, or a correlated trait like SW/Text), residuals kept diagonal:

```
Y_MT <- cbind(yNA, secondary)        # primary (with test set NA) + secondary trait
fm_mt <- Multitrait(y = Y_MT, ETA = list(list(K = G1, model = "RKHS")),
                    resCov = list(type = "DIAG"), nIter = 12000, burnIn = 2000)
pred  <- fm_mt$ETAHat[tst]           # predicted primary for the hidden lines
```

⚠️ **Key trick — the secondary trait is observed on the TEST lines too.** The primary trait is hidden ( `NA` ) on the test set, but the *secondary* trait is **known** there (it's cheap to measure!). That's what lets MT add information: for a test line we don't know its yield, but we *do* know its seed weight, and the genetic correlation converts that into a better yield guess.

---

## 12.2b 🧸 Toy first — see the secondary trait *rescue* a hidden primary ( `code/toy_12_multitrait.R` )

Here is the whole mechanism on data you can picture. Six lines; we measure two traits, but the **primary is hidden on the test lines** while the **secondary is known everywhere** (because it's cheap — that's the entire premise):

| line | primary $y_1$ | secondary $y_2$ | role |
|---|---|---|---|
| L1 | 7.1 | 6.4 | train |
| L2 | −5.0 | −4.1 | train |
| L3 | 4.8 | 5.2 | train |
| **L4** | **? (hidden)** | **2.0** | **test** |
| **L5** | **? (hidden)** | **−6.3** | **test** |

⚠️ **The trick that makes MT work:** the model never sees L4/L5's primary value — but it *does* see their secondary ($y_2$). If the two traits are genetically correlated, that known $y_2$ leaks information about the hidden $y_1$.

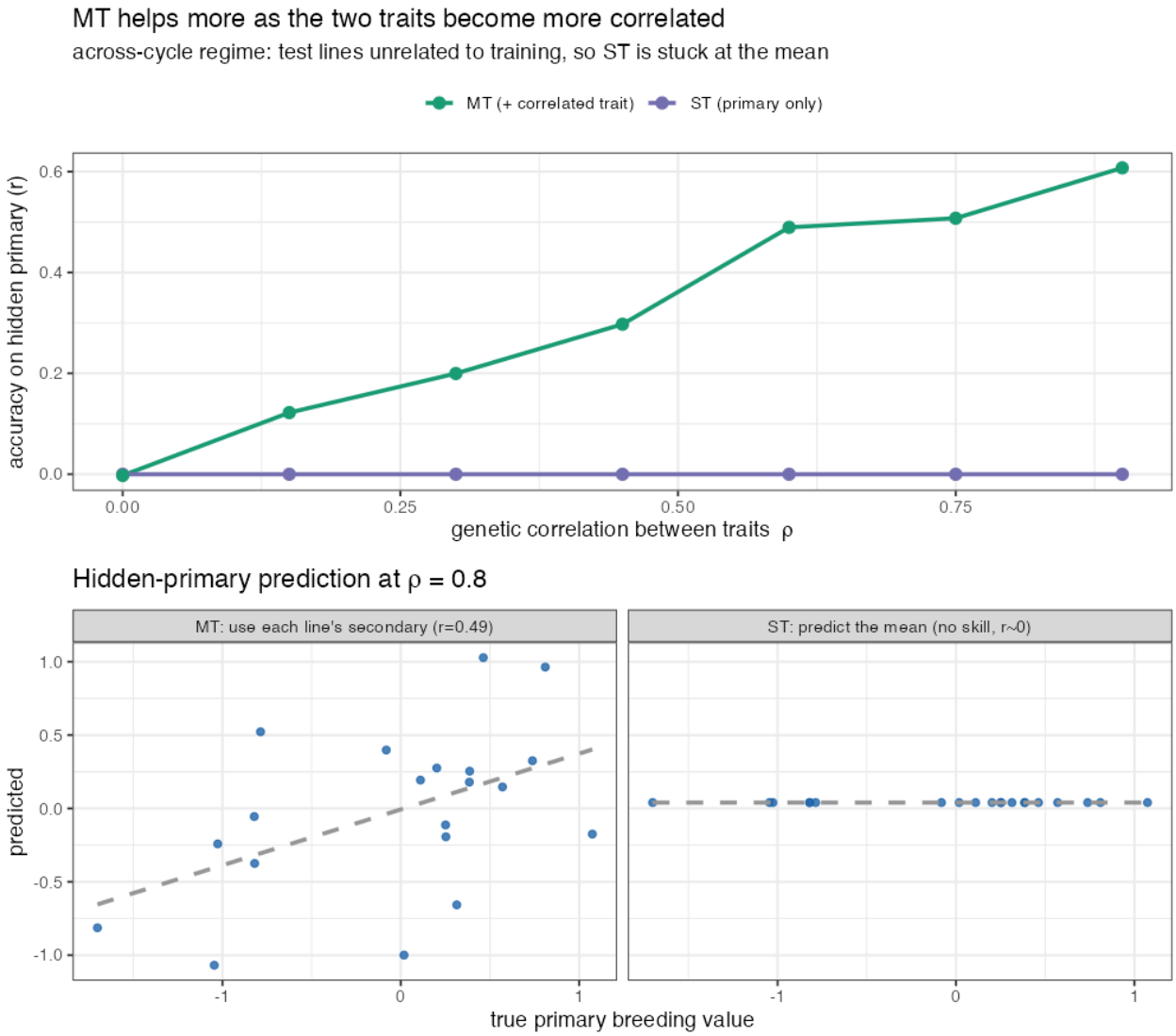
Now the **across-cycle regime** (the realistic, hard one): the test lines are a **new family, unrelated to training**. We simulate it and compare:

- **ST (primary only):** with no relatives in training, relatedness has nothing to grab → it can only guess the population mean → **accuracy ≈ 0** (a flat line of identical predictions).
- **MT (+ correlated trait):** learns on training how $y_2$ tracks $y_1$, then applies it to each test line's *own* measured $y_2$.

**Step A — turn up the genetic correlation $\rho$ and watch MT climb while ST stays flat:**

| $\rho$ (trait correlation) | 0.0 | 0.15 | 0.30 | 0.45 | 0.60 | 0.75 | 0.90 |
|---|---|---|---|---|---|---|---|
| **ST accuracy** | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **MT accuracy** | 0.00 | 0.12 | 0.20 | 0.30 | 0.49 | 0.51 | 0.61 |

**Step B — at $\rho = 0.8$:** ST is a flat horizontal cloud (everyone gets the mean); MT lines up with the truth (r ≈ 0.5).




🧠 **Two lessons fall straight out of this toy:**

1. **MT helps in proportion to $\rho$** — no correlation ($\rho = 0$) → no help → MT = ST. This is the genetic-covariance channel $\sigma_{g_{12}}$ in the equation above, made visible.
2. **MT helps *most* when relatedness can't** — we deliberately made the test lines unrelated, so ST collapsed to the mean and the correlated trait was the *only* lifeline.

🔭 **Zoom out — this *is* the study's headline.** Swap the toy's "unrelated new family" for the real **143 cycle-2 lines** predicted from cycle 1, and the secondary trait for **seed weight** (for yield) or **texture** (for appearance): MT beats ST by **+63% / +41%** *across cycles*, but barely *within* a cycle (where relatives already give ST plenty to work with). The toy and the real result tell the identical story (Lesson 14).

---

## 12.3 When does MT beat ST? The crucial nuance

This is the subtle, important result — and it follows directly from the theory:

🔬 **In the data (paper's Figs. 3–4):**

- **Within a single breeding cycle:** MT and ST performed **about the same**. No big win.
- **Across breeding cycles** (predict the *new* cycle 2 from cycle 1): **MT clearly beat ST** — up to **+63% accuracy for yield** and **+41% for appearance**.

🧠 **Why the difference?** MT helps *most when primary-trait information is scarce*. Within a cycle, you already have lots of relatives with measured primary trait → G alone predicts well → little room for a secondary trait to add value. But **across cycles**, the new lines are weakly related to the training set (Lesson 6's blocks!) → G has little to work with → the secondary trait, **measured on those very test lines**, becomes a lifeline. The harder the prediction, the more the correlated trait pays off.

> Formally, the MT advantage grows with (a) the **genetic correlation** between traits and (b) the **heritability of the secondary** trait — and it matters most when the **primary** is hard to predict (low relatedness, low $h^2$). All three conditions hold in the across-cycle yield/ appearance case.

🌱 **Breeding payoff.** This is genuinely actionable: when you bring in a brand-new set of lines (every cycle!), measuring a cheap correlated trait (seed weight, texture) on them and running a multi-trait model can boost selection accuracy for the expensive target by *tens of percent* — exactly when you need it most.

---

## 12.4 Why MT helped but NIRS/GWAS didn't — unifying the three "extra info" objectives

| Extra information | Stable? | Relevant (correlated)? | Helped? |
|---|---|---|---|
| **GWAS hits** as fixed effects (L10) | ✗ unstable across subsets | weak (polygenic) | **No** (hurt) |
| **NIRS index** as secondary (L11) | ✓ | weak correlation with target | **No** (flat) |
| **Correlated trait** (SW, Text) as secondary (L12) | ✓ | **strong** genetic correlation | **Yes** (big, across cycles) |

🧠 **The grand theme of the paper, in one table.** Adding information helps **only** when it is both **stable** and **genuinely correlated** with the target. The correlated secondary traits met both bars; GWAS hits failed on stability; NIRS failed on correlation strength. *This is the single most transferable idea in the study.*

---

> 🔧 **In practice ®.** Multi-trait GP: `BGLR::Multitrait` (the paper), `sommer`'s multivariate `mmer` (e.g. `cbind(y1,y2) ~ ...`), the `MTM` package, or `asreml`. These estimate the genetic covariance $\sigma_{g_{12}}$ that does the "borrowing." Our `04_across_cycle.R` calls `BGLR::Multitrait` for the yield+seed-weight model.

## 12.5 What you should now be able to say

- **Multi-trait (MT)** models predict several traits jointly, using the **genetic covariance** $\sigma_{g_{12}}$ to let a **correlated secondary trait** (measured even on the test lines) improve the primary prediction.
- MT ≈ ST **within** a cycle, but MT >> **ST across** cycles (+63% yield, +41% appearance) — because correlated traits help **most when the primary is hard to predict**.
- The MT benefit grows with **trait correlation** and **secondary heritability**.
- MT worked where GWAS/NIRS didn't because its extra information was both **stable and strongly correlated** — the study's central principle.

👉 Next: **Lesson 13 — Cross-Validation & Prediction Accuracy** — how all these accuracies were *honestly* measured.

# Lesson 13 — Cross-Validation & Prediction Accuracy

> **The question:** We keep quoting "accuracy 0.64", "+63%". **How do you measure whether a prediction model is any good — honestly?** Get this wrong and you'll fool yourself into shipping a useless model. This lesson is the scientific backbone that makes every number in the paper trustworthy.

---

## 13.1 The cardinal rule: never test on what you trained on

🧠 **Intuition.** If I let you *study the exam answers*, then test you on those exact questions, your 100% means nothing. A model that has *seen* a line's phenotype can "predict" it trivially — that's **memorization**, not prediction. To measure real predictive skill, you must test on lines the model **never saw the phenotype for.**

This is why every accuracy in the study comes from a **train/test split**: hide some lines' phenotypes, predict them from the rest, then compare predictions to the hidden truth.

---

## 13.2 The split the authors used: 70/30, repeated 100×

🔬 From the methods (and our `code/02`): partition the lines into **70% training / 30% testing**. Fit the model on the 70%, predict the 30%, score it. Then — crucially — **repeat 100 times** with different random splits.

🧱 **The split in code:**

```
n   <- length(y)
tst <- sample(1:n, 0.3 * n)     # 30% hidden test set
trn <- setdiff(1:n, tst)        # 70% training set
yNA <- y; yNA[tst] <- NA        # hide test phenotypes from the model
```

🧠 **Why repeat 100 times?** One split is luck — you might happen to hide easy lines. Repeating with 100 random splits gives a **distribution** of accuracies, so we can report a *typical* value and its spread. That's exactly what the **boxplots** in the paper's Figs. 2–4 show: each box = 100 partitions. A method is "better" only if its box sits reliably higher, not by one lucky split.

⚠️ **Common confusion — train/test split vs. the inner 10-fold CV.** Two *nested* loops:

- **Outer:** 70/30 split, ×100 → measures **prediction accuracy** (the headline number).
- **Inner:** within the training set, 10-fold CV → **tunes** things like the RSI penalty $\lambda$ (Lesson 11). Tuning must happen *inside* training only, never peeking at the test set.

Keeping these separate is what makes the evaluation honest.

---

## 13.3 What "accuracy" actually is

🧱 **Prediction accuracy** in this study =

$$r = \mathrm{cor}\left(\hat{y}_{\text{test}},\ y_{\text{test}}\right)$$

the **Pearson correlation** between predicted and observed values on the held-out lines.

- $r = 1$: perfect ranking. $r = 0$: useless. $r \approx 0.6$: moderate — good enough to **select**.
- We care about *ranking*, not exact values: a breeder keeps the top X% of lines, so a model that gets the **order** right is valuable even if absolute predictions are off.

🌱 **Breeding logic.** Selection accuracy $r$ is the very $r$ in the breeder's equation $\Delta G = ir\sigma_A$ (Lesson 5). So "prediction accuracy" isn't an abstract score — it **directly scales genetic gain**. A jump from $r = 0.4$ to $r = 0.65$ is a ~60% boost in gain per cycle, all else equal. That's why the paper's "+63% for yield" is a big deal.

⚠️ **Accuracy is capped by heritability.** You can only predict the genetic part; recall high-$h^2$ color → $r \approx 0.93$, low-$h^2$ yield → $r \approx 0.6$. A low number isn't always a bad model — sometimes the trait just has a low ceiling.

---

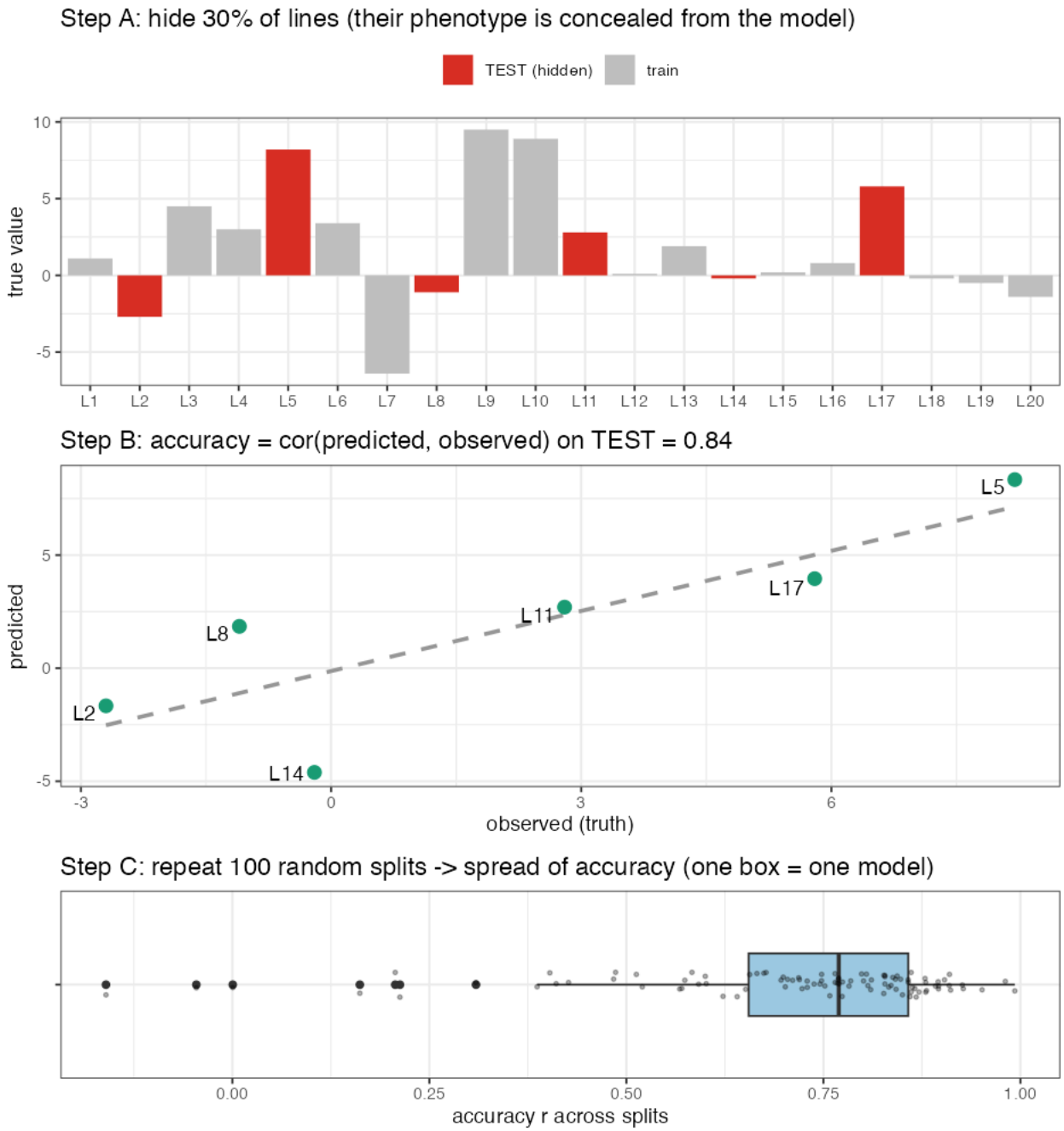
## 13.3b 🧸 Toy first — measure accuracy with your own eyes ( `code/toy_13_cv.R` )

Take **20 lines** with known true values and a model's predictions (correlated with truth, not perfect). Now do exactly what the study does:

**Step A — hide 30%.** Randomly pick 6 lines as the **test set** and conceal their phenotypes from the model.

**Step B — predict the hidden lines, then correlate.** Plot predicted vs. observed for *only* those 6 hidden lines; the **accuracy is that correlation** — here **r = 0.84**.

**Step C — don't trust one split.** Repeat the random 70/30 split **100 times**: you get a *spread* of accuracies (mean ≈ 0.71, ranging from −0.16 to 0.99!).




🧠 **Why Step C matters.** A single split can be lucky (r = 0.99) or unlucky (r negative!). That's why the paper reports a **boxplot of 100 splits**, not one number — and why a model is only "better" if its *whole box* sits higher. 🔭 **Zoom out:** swap 20 toy lines for **272** real ones, the hidden 6 for **~82**, and read every boxplot in the paper's Figs. 2–4 as "this Step-C distribution, for one model."

## 13.4 Table 1 decoded — the menu of train/test designs

The paper's **Table 1** lists the models and *what each was trained vs. validated on*. Decoded:

| Model | Trained on | Idea (lesson) |
|---|---|---|
| **RSI** | the NIRS index alone | spectra-only baseline (L11) |
| **ST** | Trait 1 only | single-trait GBLUP/KA (L7–8) |
| **ST-G** | Trait 1 + GWAS hits | GWAS-assisted single-trait (L10) |
| **MT-C** | Trait 1 + a correlated trait | multi-trait, correlated secondary (L12) |
| **MT-R** | Trait 1 + RSI | multi-trait, NIRS secondary (L11–12) |
| **MT-GC** | Trait 1 + correlated trait + GWAS | multi-trait + correlated + GWAS |
| **MT-GR** | Trait 1 + RSI + GWAS | multi-trait + NIRS + GWAS |

The letters are a code: **G** = GWAS, **C** = Correlated trait, **R** = RSI (NIRS). So **MT-GC** = "Multi-Trait with GWAS and Correlated trait." Reading Figs. 3–4 is now just decoding which boxplot is which letter combo — and asking *"did the extra letter raise the box?"* (Mostly: **C** helped across cycles; **G** and **R** didn't.)

---

## 13.5 Two evaluation scenarios — within vs. across cycle

The study evaluates accuracy under **two regimes**, and they answer different breeder questions:

1. **Within breeding cycle 1** (272 lines, 70/30 random split): *"If I have a panel and measure most of it, how well do I predict the rest?"* → the Lesson 2.1/13.2 setup. Both ST and MT do well; KA slightly beats GBLUP.
2. **Across breeding cycles** (train on cycle 1, predict cycle 2): *"How well do last cycle's data predict next cycle's brand-new lines?"* → harder, and where MT shines. This is **Lesson 14**.

🧠 The within-cycle scenario is the *optimistic* one (test lines have relatives in training); the across-cycle scenario is the *realistic deployment* one (you're always predicting new material). Reporting both is what makes the study honest about real-world performance.

---

## 13.6 Pitfalls this design avoids (and you should too)

- **Data leakage:** tuning on the test set. Avoided by the nested inner CV (§13.2).
- **Lucky-split optimism:** avoided by 100 repetitions + boxplots.
- **Relatedness leakage:** within-cycle accuracy is inflated by close relatives in train/test; the authors *expose* this by also doing the harder across-cycle test rather than hiding behind the rosy within-cycle numbers.
- **Confusing fit with prediction:** they never report training-set correlation as "accuracy."

---

> 🔧 **In practice ®.** The split-and-score loop is plain base R ( `sample()` to make folds, `cor()` to score) — that's all `01 – 07` in `code/` use. For ready-made cross-validation wrappers there are `rsample` / `caret` (generic) and GP-specific tools like `BWGS` and the helpers in `BGLR` / `sommer`.

## 13.7 What you should now be able to say

- Honest evaluation **hides test phenotypes** and scores predictions on them; the study uses **70/30 splits repeated 100×** (boxplots) for the headline accuracy, with an **inner 10-fold CV** only for tuning.
- **Accuracy = Pearson $r$** between predicted and observed test values; it **scales genetic gain** ($\Delta G = ir\sigma_A$) and is **capped by heritability**.

- **Table 1**'s model codes (**G**=GWAS, **C**=correlated, **R**=RSI; ST/MT) are just train-set recipes; reading the figures = decoding the letters.
- Two regimes — **within** vs. **across** cycle — answer different questions; reporting both keeps the study honest.

👉 Next: **Lesson 14 — Across Breeding Cycles & Updating** — the realistic test, and the practical rule it yields.

# Lesson 14 — Across Breeding Cycles & Updating the Training Set

> **The question (Objective 4):** In real breeding you don't predict the panel you already measured — you predict **next cycle's brand-new lines**. How well does that work, and **how much new-cycle data must you add to the training set** to keep predictions sharp? This is the most *operational* lesson: it tells a breeder what to actually do each year.

---

## 14.1 The realistic test: train on the past, predict the future

Recall the cycle structure (Lesson 1): **cycle 1 = 272 lines** (the "past", measured 2018–19), **cycle 2 = 143 lines** (the "new" material, 2019). The deployment question is:

> Train a model on cycle 1. Predict cycle 2. How good is it?

🧠 **Why this is *harder* than within-cycle prediction.** From Lesson 6, lines cluster into program/cycle **blocks**, and relatedness is **higher within a cycle than between cycles**. GBLUP predicts a test line by leaning on its *relatives* in the training set (Lesson 7). But cycle-2 lines have **few close relatives** in cycle 1 → the bridge $\mathbf{G}_{\text{test,train}}$ is weak → accuracy drops. This isn't a flaw in the method; it's the geometry of the data.

🔬 **In the data.** The paper reports accuracy **dropped drastically** when cycle 1 alone (272 lines) was used to predict cycle 2 (143 lines) — for *all* traits. The genetic gap between cycles is the cause.

---

## 14.2 The fix: keep refreshing the training set

The authors then **add a growing fraction of cycle-2 lines into the training set** (0%, 10%, 20%, 30%, 40%, 50% — i.e. 0, 14, 29, 43, 57, 72 lines) and re-measure accuracy on the remaining cycle-2 lines. The result is the study's clearest practical message:

> **As more new-cycle lines are added to training, prediction accuracy on the rest of the new cycle rises steadily.**

🔬 **Reproduced here** ( `code/04_across_cycle.R` , yield 2019; 20 reps per point):

| % of cycle-2 lines added to training | ST accuracy | MT accuracy (yield + seed wt) |
|---|---|---|
| 0% (predict cycle 2 purely from cycle 1) | 0.25 | 0.31 |
| 10% | 0.31 | 0.36 |
| 20% | 0.33 | 0.38 |
| 30% | 0.37 | 0.42 |
| 40% | 0.38 | 0.44 |

> See `figures/07_across_cycle.png`. **Two effects are visible at once, both predicted by the theory:** (1) accuracy **climbs steadily** as more cycle-2 lines join training (Objective 4), and (2) the **MT curve sits above ST** at every point (Objective 1). At 0% added — the hardest, "pure future" case — ST is only ~0.25; adding a correlated trait (MT) lifts it ~22%, and adding real new-cycle data lifts it further still. The paper, with 100 reps and full MCMC, saw yield accuracy increase up to **1.8-fold** and texture up to **0.6-fold** by 50%, and MT gains up to **+63%** — our lighter laptop run reproduces the same *directions and shape*.

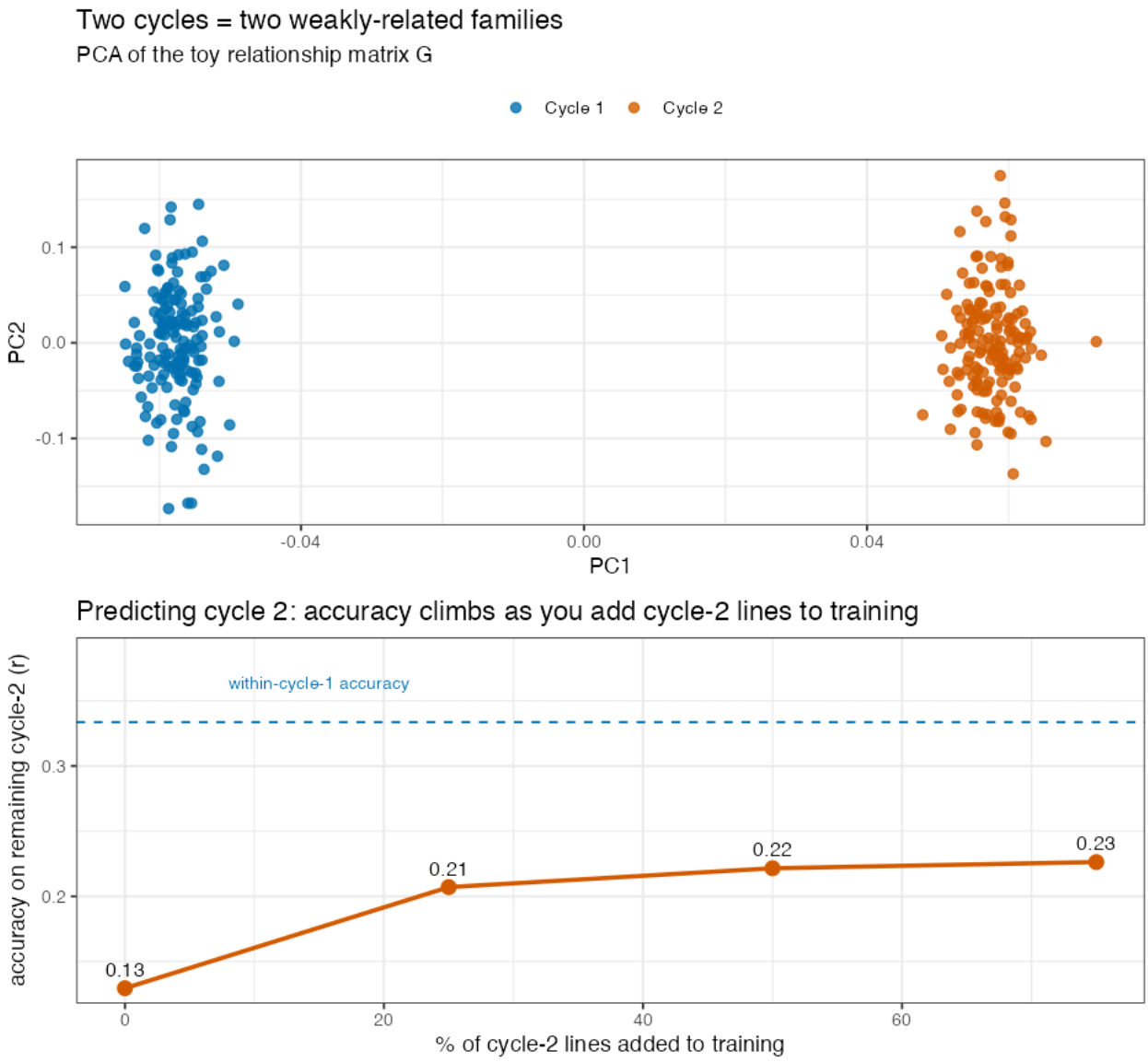
🧠 **Intuition.** Each new-cycle line you measure and add to training gives the model *relatives* of the lines you still want to predict — strengthening $\mathbf{G}_{\text{test,train}}$. You're literally building the bridge across the cycle gap, plank by plank.

🌱 **Breeding logic — the operational rule.** A genomic-prediction model is **not** "train once, use forever." It **decays** as your germplasm moves away from the training set. Every cycle you must **fold a sample of the new lines (with measured phenotypes) back into training** to keep accuracy up. Budget for it: phenotype a slice of each new cycle precisely so the model keeps working on the rest.

---

## 14.2b 🧸 Toy first — build the rising curve yourself (`code/toy_14_across_cycle.R`)

Let's manufacture the two-cycle problem in miniature. Simulate **two genetic clusters** — "Cycle 1" and "Cycle 2" — with *independent* allele frequencies, so they're **weakly related** (just like real cycles, Lesson 6). A PCA of the toy **G** shows them as two clearly separated clouds:




Now train GBLUP on Cycle 1 and predict Cycle 2, adding more Cycle-2 lines to training each time:

| % of Cycle-2 lines in training | 0% | 25% | 50% | 75% | (within-Cycle-1 reference) |
|---|---|---|---|---|---|
| accuracy on remaining Cycle 2 | **0.13** | 0.21 | 0.22 | **0.23** | **0.33** |

🧠 **Read the curve.** At **0%** (predict Cycle 2 purely from Cycle 1) accuracy is a feeble **0.13** — because the test lines have *no relatives* in training (look at the PCA gap; $\mathbf{G}_{\text{test,train}} \approx \mathbf{0}$). Each batch of Cycle-2 lines you add gives the remaining Cycle-2 lines *relatives* in the training set, and accuracy **climbs toward** the within-Cycle-1 ceiling (dashed line, 0.33) — but never quite reaches it on this budget.

🔭 **Zoom out — this is exactly `figures/07_across_cycle.png` on the real data:** real yield-2019 accuracy started ~0.25 predicting cycle 2 from cycle 1 and rose steadily as cycle-2 lines were added. The toy isolates *why*: **relatedness, rebuilt plank by plank.**

---

## 14.3 Multi-trait shines exactly here

This is where Lessons 12 and 14 join hands. Across cycles — where ST prediction struggles — the **multi-trait** model's correlated secondary trait (seed weight for yield, texture for appearance) adds the most value, because:

- The **primary** trait is hard to predict (weak cross-cycle relatedness).
- The **secondary** trait is **measured on the new-cycle test lines themselves** (it's cheap!), injecting fresh, line-specific information that G can't supply.

🔬 So the headline gains — **+63% accuracy for yield, +41% for appearance** — are *across-cycle* gains. Within a cycle, MT ≈ ST (Lesson 12); across cycles, MT pulls clearly ahead. Our reproduction shows the MT curve sitting above the ST curve at low percentages, then both converging as enough new-cycle data makes the problem easy for either model.

🧠 **Putting it together:** MT and "updating the training set" are **two complementary remedies** for the same disease (cross-cycle accuracy decay). MT helps *immediately* using a cheap correlated trait; updating helps *over time* by rebuilding relatedness. A smart program uses **both**.

---

## 14.4 Why not just always use a huge old training set?

Because relatedness, not raw size, drives accuracy. A bigger training set of *distantly related* old lines helps far less than a *smaller, fresher* set related to your targets. (The paper notes a larger study with 3,722 lines saw bigger update gains than this 415-line one — size helps too — but the *relatedness* principle dominates.) The lesson: **invest phenotyping in material related to what you're about to select**, not just in accumulating history.

---

## 14.5 The mental-map payoff

Lesson 14 closes the loop back to Lesson 0's flowchart. Everything funnels here:

- Clean phenotypes (L3) and quality markers (L4) →
- relatedness **G** (L6) that has **block structure** (the root cause of cross-cycle decay) →
- prediction models (L7–8) whose accuracy depends on that relatedness →
- remedies: **multi-trait** (L12) and **training-set updating** (this lesson).

The research question from Lesson 0 — *"can we predict bean quality/yield from DNA well enough to select?"* — gets a nuanced **yes**: within related material, accuracies of 0.6–0.93 are plenty for selection; across new material, you must **add correlated traits and refresh the training set** to keep that power.

---

## 14.6 What you should now be able to say

- Predicting a **new breeding cycle** is **harder** than within-cycle prediction because new lines are **weakly related** to the training set (G's block structure, Lesson 6) — accuracy drops.
- **Adding a growing fraction of new-cycle lines to training steadily restores accuracy** (up to ~1.8-fold for yield by 50%); genomic-prediction training sets must be **continually updated**.
- **Multi-trait models help most across cycles** (+63% yield, +41% appearance) because the cheap correlated trait is measured on the very lines being predicted.
- **Relatedness, not just size**, drives accuracy — phenotype material *related to your selection targets*.

👉 Next: **Lesson 15 — Results & Take-home Messages** — the whole study in one page.

# Lesson 15 — Results & Take-home Messages

> **The question:** Zoom all the way back out. In one page: *what did the study find, what does it mean for a bean breeder, and what general principles should you carry to any genomic-prediction problem?*

## 15.1 The four objectives, answered

| # | Objective | Answer | Evidence |
|---|---|---|---|
| 1 | ST vs MT genomic prediction | **MT ≈ ST within a cycle; MT ≫ ST across cycles** (+63% yield, +41% appearance) | Figs. 3–4 (L12, L14) |
| 2 | NIRS index as a secondary trait | **No improvement** (low correlation, intact-seed scan) | Fig. 3, Supp. T3 (L11) |
| 3 | GWAS SNPs as fixed effects | **Lowered accuracy for all traits/years** (unstable hits, polygenic) | Figs. 3–4 (L10) |
| 4 | New-cycle lines needed for updating | **Accuracy rises steadily as new-cycle lines are added** (yield up to ~1.8-fold by 50%) | Fig. 4 (L14) |

## 15.2 The numbers worth remembering

- **Prediction is feasible.** Within related material, accuracies ran **0.39–0.93** depending on trait; **color/SW** (high $h^2$) reached **~0.93**, **yield/appearance** (low $h^2$) sat **~0.4–0.6** — *moderate-to-high, and good enough to select.*
- **KA ≈ GBLUP**, with KA consistently **~+0.03** better → the authors carried both forward.
- **Heritability ran the show:** color (high $h^2$) easy; days-to-maturity's $h^2$ fell 0.77→0.49 across years and its accuracy fell with it.
- **MT across cycles: +63% (yield), +41% (appearance)** — the headline gains.

🔬 Our independent reproductions agree with the paper: GBLUP yield accuracy **0.64** (three ways, L7); Bonferroni threshold **$2.16\times10^{-5}$** (L9); high-$h^2$ color far more GWAS-detectable than low-$h^2$ yield (105 vs 4 hits, L9); and the across-cycle accuracy curve rising with added new-cycle lines (L14, `figures/07_across_cycle.png`).

## 15.3 The ONE big idea (if you remember nothing else)

> **Extra information helps a prediction model only when it is both *stable* and *genuinely correlated* with the target — and you can't fake either by being confident.**

The study is, at heart, three tests of this principle:

- **Correlated traits (SW, Text):** stable ✓ + strongly correlated ✓ → **helped a lot** (MT).
- **NIRS index:** stable ✓ but weakly correlated ✗ → **didn't help**.
- **GWAS hits as fixed effects:** strongly "relevant"-looking but **unstable ✗ → hurt** (winner's curse, false certainty).

And the corollary that makes GBLUP's humility a virtue:

> **For polygenic traits, treating all markers as small, equally-shrunk, uncertain effects (GBLUP) is close to optimal. Injecting false certainty (fixing GWAS hits) degrades it.**

---

## 15.4 The breeder's playbook from this study

1. **Use genomic prediction** — GBS is cheap, accuracies are selection-worthy.
2. **Default to GBLUP or KA**; KA if you want a consistent small edge.
3. **Don't bolt GWAS hits on as fixed effects** for polygenic traits — it backfires.
4. **Do add a cheap, heritable, correlated trait** (seed weight, texture) via a **multi-trait model** — especially when predicting **new** material.
5. **Continually update the training set** with a sample of each new cycle's phenotyped lines; models decay as germplasm advances.
6. **Set expectations by heritability** — low-$h^2$ traits will never predict like high-$h^2$ ones.

---

## 15.5 Why the *sequence* of the study makes sense

Trace the logic — each step *had* to come before the next:

1. **Clean phenotypes** (BLUPs, L3) — else you predict field noise.
2. **Quality markers** (L4) → **relatedness G** (L6) — the substrate of prediction.
3. **Establish the GP baseline** (GBLUP/KA, L7–8) — you need a yardstick before testing add-ons.
4. **Test each add-on against that baseline** — GWAS (L9–10), NIRS (L11), multi-trait (L12) — one variable at a time, honestly scored (L13).
5. **Stress-test in the real deployment regime** (across cycles, L14) — where the winner (MT) and the operational rule (update training) finally emerge.

You can't judge "does GWAS help?" without a baseline; you can't trust any comparison without honest cross-validation; and you can't claim real-world value without the across-cycle test. The ordering *is* the scientific method applied to breeding.

---

## 15.6 Limitations the authors are honest about

- Only **one preprocessing** (SNV) for NIRS; derivatives/wavelets might do better.
- **Intact dry seeds** scanned — may miss cooked-quality signal.
- **Conservative Bonferroni** may have missed small-effect loci; a function-aware analysis of loci is needed before GWAS-assisted GP could help.
- Modest panel (**415 lines**); larger panels show bigger updating gains.

---

## 15.7 What you should now be able to say

- State all **four objectives and their answers**, and the headline numbers (0.4–0.93 accuracy; +63%/+41% MT across cycles; KA +0.03; GWAS/NIRS no help).
- Articulate the **one big idea** (stable **and** correlated information helps; false certainty hurts) and the **breeder's playbook**.

- Explain **why the study's steps are ordered** as they are.

👉 Final: **Lesson 16 — Reproduce It Yourself** — run the code and regenerate every figure.

# Lesson 16 — Reproduce It Yourself

> **The question:** You've understood the study. Now *run* it. This lesson is a hands-on guide to the scripts in `code/` — what each does, how to run it, and how the original authors' scripts in `repo/` map onto the paper. Reproduction is the final test of understanding.

---

## 16.1 Setup (once)

```
# In R (>= 4.0):
install.packages(c("BGLR", "rrBLUP", "SpATS", "ggplot2"))   # prediction, spatial, figures
# These need a compiler toolchain (clang + gfortran + make), which ships with Xcode
# Command Line Tools on macOS: run  xcode-select --install  once if missing.
```

**Bioinformatics tools** for Lesson 4 ( `bwa` , `samtools` , `fastp` ) install via conda/bioconda: `conda install -c bioconda bwa samtools fastp` .

> **Why two packages didn't install the first time (and weren't Mac-specific):** the current CRAN build of `SFSI` requires a newer R than was on this machine (R 4.3) — a version-availability issue, not a macOS issue. So we implement its **regularized selection index from scratch** (penalized regression — Lessons 7 & 11) instead. `GAPIT3` **/FarmCPU** needs extra deps; we reproduce the GWAS *concept* with a transparent PC-corrected scan (Lesson 9). `SpATS` and `BGLR` compiled fine once the toolchain was confirmed.

Most data is already in `repo/GB_BLB.RData` (phenotypes, genotypes, NIRS, SNP positions — Lesson 2). For the **real genotyping pipeline (Lesson 4)** the raw reads are on NCBI BioProject **PRJNA1138671** / ENA. The full smallest lane is ~10.7 GB (361M reads); we **stream just a subset** straight from ENA without downloading the whole file:

```
# stream ~6M reads (downloads only a fraction, not the full 10.7 GB)
curl -s "https://ftp.sra.ebi.ac.uk/vol1/fastq/SRR299/016/SRR29913416/SRR29913416.fastq.gz" \
  | gunzip | head -24000000 > bioinfo/subset.fastq
```

and align to **one chromosome** to keep disk/time small. The logic is identical at full scale.

Run scripts from inside the `code/` folder (paths are relative):

```
cd code
Rscript 01_explore_data.R
```

---

## 16.2 Our reproduction scripts ( `code/` )

| Script | Lessons | What it does | Key outputs |
|---|---|---|---|
| `01_explore_data.R` | 1–4 | Loads data; trait distributions, correlations, MAF, SNPs/chromosome | `figures/01_*`, `02_*`, `03_*`; prints yield↔SW = +0.41, SW↔texture = −0.36 |
| `02_gblup_from_scratch.R` | 5–7 | Builds **G**; PCA + kinship; GBLUP **3 ways** (scratch / rrBLUP / BGLR) | `figures/04_pca`, `05_kinship`; **accuracy 0.64** ×3 |
| `03_gwas.R` | 9 | PC-corrected single-marker GWAS; Bonferroni; Manhattan | `figures/06_manhattan`; threshold **$2.16\times10^{-5}$**; 4 (yield) vs 105 (color) hits |
| `04_across_cycle.R` | 12–14 | Across-cycle ST vs MT; accuracy vs % new-cycle lines added | `figures/07_across_cycle`; rising curves, MT > ST |
| `05a_demux.py` | 4 | Demultiplexes the study's real streamed GBS reads by barcode | per-sample FASTQs; 97.9% barcode match |
| `05b_pipeline.sh` | 4 | Trim → align (chr01) → sort (`fastp` / `bwa` / `samtools`) | `bioinfo/bam/*.bam`, `align_stats.tsv` |
| `05c_call_genotypes.py` | 4 | Pileup → **0/1/2** via allele fraction; het diagnostics | `bioinfo/genotypes_012.tsv`, `figures/08_real_genotype_matrix.png` |
| `06_spatial_demo.R` | 3 | Runs **SpATS**; recovers genotype BLUPs from a noisy field | `figures/09_spats_demo.png`; r 0.46→0.84 |
| `07_heritability_accuracy.R` | 5,13,15 | $h^2$_g vs GBLUP accuracy for all 2018 traits | `figures/10_*`; **r($h^2$, acc)=0.80** |
| `toy_04_reads_to_012.R` | 4 | toy reads→dosage at one SNP | `figures/12_*` |
| `toy_05_breeding_value.R` | 5 | toy 5×10 g = M·α, fully worked | `figures/11_*` |
| `toy_05b_quantgen.R` | 5 | variance components & $H^2$; additive vs dominance ($\sigma^2$_A/$\sigma^2$_D); breeder's equation | `figures/23_*`, `24_*` |
| `toy_06_pedigree_kinship.R` | 6 | pedigree A (tabular) vs marker G; Mendelian sampling & cryptic relatedness | `figures/22_*` |
| `toy_08_kernels.R` | 8 | toy distance→Gaussian kernel, bandwidth effect | `figures/15_*` |
| `toy_09_gwas.R` | 9 | toy per-SNP tests → mini Manhattan + Bonferroni | `figures/16_*` |
| `toy_11_rsi.R` | 11 | toy index: OLS overfit vs ridge (p>n) | `figures/17_*` |
| `toy_10_winners_curse.R` | 10 | toy: GWAS hits inflate (winner's curse); fixed < shrink | `figures/20_*` |
| `toy_12_multitrait.R` | 12 | toy: correlated secondary rescues hidden primary | `figures/19_*` |

| Script | Lessons | What it does | Key outputs |
|---|---|---|---|
| `toy_14_across_cycle.R` | 14 | toy: two cycles; accuracy rises as new-cycle lines added | `figures/21_*` |
| `toy_13_cv.R` | 13 | toy hide/predict/correlate + 100-split spread | `figures/18_*` |

(Also `toy` figures 13 = G matrix and 14 = BLUP shrinkage are produced by inline snippets.)

Each script is heavily commented and re-implements the method *from first principles* before (or instead of) calling a package — so you can read every line. Run them in order; later scripts assume you understand earlier figures.

🔬 **What "reproduced" means here.** We confirmed our pipeline matches the paper on independently checkable anchors: GBLUP yield accuracy ≈ **0.64** (paper ~0.60); Bonferroni = **$2.16\times10^{-5}$** (paper's exact value); heritability→accuracy and heritability→GWAS-power patterns; and the across-cycle accuracy-rises-with-updating trend with MT ahead of ST. We deliberately used lighter settings (fewer CV reps, shorter MCMC) so scripts finish in minutes on a laptop, so absolute numbers differ slightly from the paper's 100-rep / 12,000-iteration runs — the *patterns* match.

---

## 16.3 The authors' original scripts ( `repo/` ) — map to the paper

The repo's scripts are written for a computing **cluster** (note the `#SBATCH` headers and the `SLURM_ARRAY_TASK_ID` job index — each of 1,000 array jobs does one trait × one of 100 repetitions). To run one locally, set the job id manually:

```
Sys.setenv(SLURM_ARRAY_TASK_ID = "1")   # then source the script; it picks trait 1, rep 1
```

| Repo script | Paper section | What it computes |
|---|---|---|
| `0.Spatial_Analysis.r` | Phenotypic data (L3) | SpATS spatial model → **BLUPs** (the `pheno` table). Needs the raw Excel, not shipped. |
| `1.BLB_GBLUP_RKHS_traits20XX.R` | GBLUP vs kernels (L7–8) | GBLUP + 3 Gaussian kernels (K1/K2/K3) + **KA**, per trait |
| `2.BLB_GBLUP_KA_GWAS_RSI_20XX.R` | RSI, GWAS, MT (L9–13) | builds NIRS **RSI** (SFSI), runs **FarmCPU** GWAS (GAPIT), ST/MT GBLUP & KA, ± GWAS fixed effects |
| `3.BLB_..._YD/app_20XX.R` | Across cycles, MT (L12,L14) | the cross-cycle, multi-trait runs with correlated secondaries (SW for yield, Text for App) and varying % of cycle 2 in training |

🧠 **Reading tip.** Every repo script follows the same skeleton you now recognize: *load data → build G and distance D → subset to a trait → 70/30 split → fit models →* `cor(pred, truth)` *→ save*. Once you see that skeleton, the 320-line scripts become readable.

---

## 16.4 A guided mini-exercise (≈ 15 min)

Cement understanding by *changing* things and predicting the result first:

1. **Trait swap.** In `02_gblup_from_scratch.R`, change `trait <- "yd18"` to `"col18"` (color, high $h^2$). *Predict:* accuracy should be **much higher** (≈0.9). Run it. Were you right? (This is heritability→accuracy from Lesson 5/7, live.)
2. **Break the relatedness.** In `04_across_cycle.R`, set `props <- c(0)` only. *Predict:* the hardest case (no new-cycle lines) → lowest accuracy, biggest MT advantage. Confirm.

3. **Kill the secondary correlation.** In `04`, replace `sw <- Y$sw19` with a random vector `sw <- rnorm(nrow(Y))`. *Predict:* MT should collapse to ≈ ST (no $\sigma_{g_{12}}$ to exploit — Lesson 12). Confirm.

If your predictions matched the runs, you understand the study. If they didn't, the surprise is exactly where to re-read.

---

## 16.5 Troubleshooting

- **`SFSI` / `GAPIT3` won't install** → they need compilers/extra deps. You don't need them: our `03_gwas.R` reproduces the GWAS *concept*, and the RSI is penalized regression you understand from Lesson 7/11.
- **BGLR prints pages of iterations** → normal MCMC logging; we wrap it in `sink()` in our scripts.
- **Numbers differ slightly between runs** → BGLR is stochastic and splits are random; set the seed (we do) and/or raise repetitions for stability.

---

## 16.6 Where to go next

- Re-read **Lesson 0's mental map** — every box should now feel concrete.
- Read the paper (`ref_paper.pdf`) end to end; you have the vocabulary now.
- Try a fourth script: multi-trait for **appearance + texture** (mirror `04` with `y <- Y$app19; sw <- Y$text19`) and see if you reproduce the appearance gains.

**You can now read a quantitative-genetics / genomic-prediction paper, reproduce its core, and explain every step to someone else. That's the whole goal.** 🌱

# Glossary — Quick Reference

Plain-language definitions of every key term in the course. Each links to the lesson where it's developed. Skim before an exam; look up while reading.

## Biology & breeding

- **Common bean / *Phaseolus vulgaris*** — the crop; diploid, 2n=22 (**11 chromosomes**), self-pollinating. (L1)
- **Market class** — beans grouped by seed type (navy, pinto, **black**, …). (L1)
- **Line** — a near-homozygous, reproducible genotype (because beans self-pollinate). (L1)
- **Breeding cycle** — one round of cross → fix lines → test → select parents. Here: **cycle 1 = 272 lines** (2018–19), **cycle 2 = 143 lines** (2019). (L1)
- **Check cultivar** — a known variety (Eclipse/Zorro/Zenith) grown repeatedly to map field variation. (L3)
- **Genetic gain ($\Delta G$)** — improvement per cycle; $\Delta G = ir\sigma_A$. (L5)

## Traits

- **YD** seed yield; **SW** 100-seed weight; **DF** days to flowering; **DPM** days to maturity. (L1)
- **App** canning **appearance** (trained-panel score 1–5); **Col** color/darkness; **Text** texture (machine peak-force); **L*a*b*** objective color coordinates. (L1)
- **Canning quality** — how good beans are after the standardized canning protocol. (L1)

## Statistics & quantitative genetics

- **Phenotype** — measured trait value = mean + breeding value + environment. (L5)
- **Additive effect ($\alpha_j$)** — the per-copy value of an allele at marker $j$. (L5)
- **Breeding value ($g_i$)** — sum of additive effects, $g_i = \sum_j \alpha_j x_{ij}$; what we predict/select on. (L5)
- **Heritability ($h^2$, $H^2$, $h^2_g$)** — fraction of trait variance that's genetic; **caps prediction accuracy**. (L3, L5)
- **Fixed effect** — estimated freely, no shrinkage ("I'm sure"). **Random effect** — shrunk toward the mean ("probably small"). (L3, L10)
- **BLUP** — Best Linear Unbiased Prediction; the shrunken genotype estimate = one clean number per line. (L3)
- **Mixed model** — a model with both fixed and random effects. (L3, L7)
- **Shrinkage / regularization** — pulling estimates toward 0 to avoid overfitting when $p \gg n$. (L7, L11)
- **Cross-validation** — hide data, predict it, score honestly; **70/30 ×100** here, with inner **10-fold** for tuning. (L13)
- **Prediction accuracy ($r$)** — Pearson correlation between predicted and observed test values. (L13)

## Genomics

- **SNP** — single-DNA-position difference; coded **0/1/2** (allele dosage). (L2, L4)
- **GBS** — Genotyping-by-Sequencing; cheap way to find thousands of shared SNPs. (L4)
- **MAF** — minor allele frequency; rare SNPs (<0.01) filtered out. (L4)
- **LD (linkage disequilibrium)** — correlation between nearby SNPs; basis of marker tagging; LD pruning removes redundant SNPs. (L4)
- **Imputation** — filling missing genotypes from genetic neighbors (Beagle). (L4)
- **Population structure** — subpopulations (program/cycle) that confound GWAS; corrected with PCs. (L6, L9)

## Models in the study

- **G (genomic relationship matrix)** — realized relatedness, $\mathbf{G} = \mathbf{Z}\mathbf{Z}^\top/p$. (L6)

- **GBLUP** — genomic prediction via G in a mixed model; **= ridge regression on markers**. (L7)
- **RKHS / Gaussian kernel** — GBLUP with a flexible similarity kernel $\exp(-\theta d^2)$; captures non-additive effects. (L8)
- **Bandwidth ($\theta$, `h`)** — kernel decay rate; small = smooth/global, large = local. (L8)
- **KA (kernel averaging)** — mix of bandwidths (0.02, 1, 5) weighted by data; the chosen GP model. (L8)
- **GWAS** — tests each SNP for trait association; p-values, Manhattan plot. (L9)
- **FarmCPU** — the iterative GWAS method (GAPIT3) used. (L9)
- **Bonferroni** — strict significance threshold $0.05/m = 2.16 \times 10^{-5}$, where $m$ = number of SNPs. (L9)
- **GWAS-assisted GP** — significant SNPs added as fixed effects; **hurt accuracy** here. (L10)
- **NIRS** — near-infrared spectroscopy; 551-wavelength chemical fingerprint. (L11)
- **RSI (regularized selection index)** — penalized-regression weighting of spectra into one index. (L11)
- **SNV** — Standard Normal Variate, the NIRS preprocessing used. (L11)
- **ST vs MT** — single-trait vs multi-trait prediction. (L12)
- **Secondary trait** — a cheap, correlated trait that boosts a hard target in MT models (SW for yield, Text for App). (L12)
- **Genetic covariance ($\sigma_{g_{12}}$)** — the channel through which a secondary trait helps; 0 → MT = ST. (L12)

## Model-code letters (Table 1)

- **G** = GWAS hits; **C** = correlated trait; **R** = RSI (NIRS). E.g. **MT-GC** = multi-trait + GWAS + correlated trait. (L13)